



# Appunti di Linguaggi di Programmazione

Anno 2022/2023

Autore:  
Dessì Leonardo

Revisori:  
Corda Erica  
Dessì Matteo  
Giuffrida Simone



# Indice

| I          | Parte 1                                      |           |
|------------|----------------------------------------------|-----------|
| <b>1</b>   | <b>Introduzione</b>                          | <b>7</b>  |
| <b>1.1</b> | <b>Introduzione</b>                          | <b>7</b>  |
| 1.1.1      | L'evoluzione dei linguaggi                   | 7         |
| 1.1.2      | Nascita dei linguaggi di programmazione      | 8         |
| <b>2</b>   | <b>Analisi lessicale: Linguaggi regolari</b> | <b>11</b> |
| <b>2.1</b> | <b>Letture di un programma</b>               | <b>11</b> |
| <b>2.2</b> | <b>Linguaggi</b>                             | <b>12</b> |
| 2.2.1      | Operazioni sui linguaggi                     | 13        |
| <b>2.3</b> | <b>Espressioni regolari</b>                  | <b>14</b> |
| <b>2.4</b> | <b>Automi finiti</b>                         | <b>17</b> |
| <b>2.5</b> | <b>Grammatica</b>                            | <b>20</b> |
| <b>3</b>   | <b>Analisi Semantica</b>                     | <b>27</b> |
| <b>3.1</b> | <b>Introduzione alla semantica</b>           | <b>27</b> |
| 3.1.1      | Paradigma di programmazione                  | 28        |
| <b>3.2</b> | <b>Semantica operativa</b>                   | <b>28</b> |
| <b>4</b>   | <b>Espressioni aritmetiche non tipizzate</b> | <b>31</b> |
| <b>4.1</b> | <b>Introduzione</b>                          | <b>31</b> |
| <b>4.2</b> | <b>Sintassi</b>                              | <b>33</b> |
| <b>4.3</b> | <b>Stili di semantica</b>                    | <b>34</b> |
| <b>4.4</b> | <b>Evaluation</b>                            | <b>35</b> |
| 4.4.1      | Big-step semantic                            | 41        |
| <b>5</b>   | <b>Lambda Calcolo non tipizzato</b>          | <b>43</b> |
| <b>5.1</b> | <b>Introduzione</b>                          | <b>43</b> |

|            |                                             |           |
|------------|---------------------------------------------|-----------|
| <b>5.2</b> | <b>Basi sul lambda-calcolo</b>              | <b>44</b> |
| 5.2.1      | Sintassi concreta e astratta                | 45        |
| 5.2.2      | Variabili e Metavariabili                   | 46        |
| 5.2.3      | Scope (o ambiente)                          | 46        |
| 5.2.4      | Semantica operativa                         | 46        |
| <b>5.3</b> | <b>Programmazione con il lambda-calcolo</b> | <b>49</b> |
| 5.3.1      | Argomenti multipli                          | 49        |
| 5.3.2      | Booleani di Church                          | 49        |
| 5.3.3      | Coppie(Pairs)                               | 51        |
| 5.3.4      | Numerali di Church                          | 51        |
| <b>5.4</b> | <b>Formalità</b>                            | <b>53</b> |
| 5.4.1      | Sintassi                                    | 53        |
| 5.4.2      | Sostituzione                                | 54        |
| 5.4.3      | Semantica operativa                         | 56        |

## II

## Parte 2

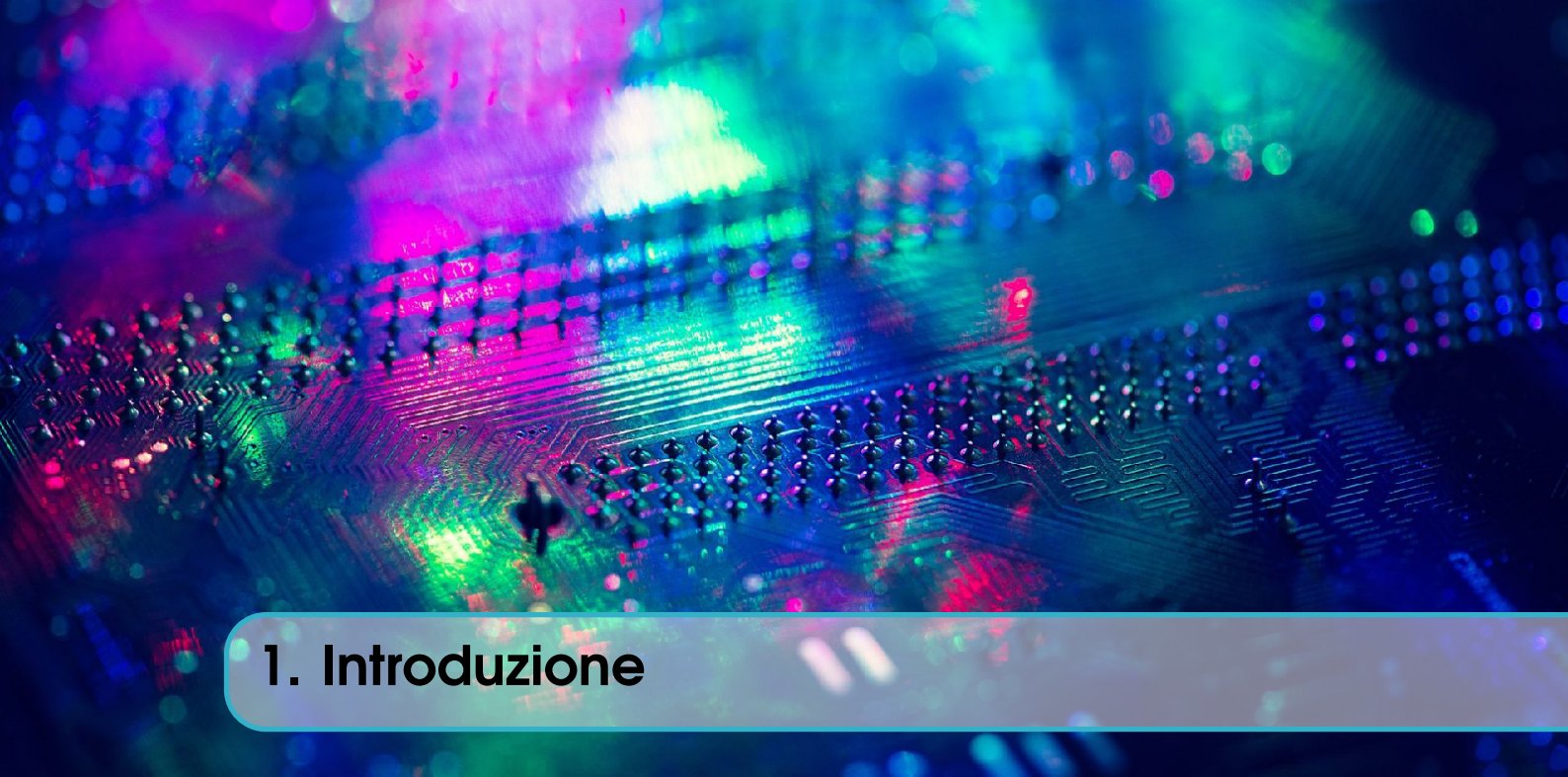
|          |                                           |           |
|----------|-------------------------------------------|-----------|
| <b>6</b> | <b>Macchina SECD</b>                      | <b>59</b> |
| 6.1      | Introduzione                              | 59        |
| 6.2      | Descrizione della macchina                | 59        |
| 6.3      | L'interprete                              | 60        |
| <b>7</b> | <b>Espressioni aritmetiche tipizzate</b>  | <b>63</b> |
| 7.1      | Tipi                                      | 63        |
| 7.2      | La relazione di tipizzazione              | 64        |
| 7.3      | Sicurezza = Progresso + Preservazione     | 66        |
| <b>8</b> | <b>Lambda Calcolo tipizzato semplice</b>  | <b>67</b> |
| 8.1      | Tipi di funzione                          | 67        |
| 8.2      | La relazione di tipizzazione              | 68        |
| 8.3      | Vaneggi del prof                          | 71        |
| <b>9</b> | <b>Linguaggio imperativo</b>              | <b>73</b> |
| 9.1      | Lo stato                                  | 73        |
| 9.2      | Il cuore di un linguaggio imperativo: IMP | 73        |
| 9.2.1    | Espressioni                               | 74        |
| 9.2.2    | Comandi                                   | 74        |

|             |                                                                    |            |
|-------------|--------------------------------------------------------------------|------------|
| <b>10</b>   | <b>Ambiente e Memoria</b>                                          | <b>79</b>  |
| <b>10.1</b> | <b>Ambiente</b>                                                    | <b>79</b>  |
| 10.1.1      | Binding                                                            | 81         |
| 10.1.2      | Blocchi                                                            | 81         |
| 10.1.3      | Tipi d'ambiente e operazioni                                       | 82         |
| <b>10.2</b> | <b>Scope</b>                                                       | <b>83</b>  |
| 10.2.1      | Scope statico e dinamico                                           | 86         |
| <b>10.3</b> | <b>Ambiente locale</b>                                             | <b>88</b>  |
| <b>10.4</b> | <b>Gestione della memoria</b>                                      | <b>89</b>  |
| 10.4.1      | Allocazione statica                                                | 89         |
| 10.4.2      | Allocazione dinamica: la pila                                      | 89         |
| <b>10.5</b> | <b>Memoria e pila degli RdA</b>                                    | <b>90</b>  |
| <b>10.6</b> | <b>Ambiente e pila degli RdA</b>                                   | <b>91</b>  |
| 10.6.1      | Implementazione dello scope statico                                | 91         |
| <b>11</b>   | <b>Il linguaggio IMP con i blocchi e le dichiarazioni (locali)</b> | <b>93</b>  |
| <b>11.1</b> | <b>Il linguaggio IMP con i blocchi e le dichiarazioni (locali)</b> | <b>93</b>  |
| 11.1.1      | Dichiarazioni                                                      | 93         |
| 11.1.2      | Comandi                                                            | 94         |
| <b>12</b>   | <b>Parametri, astrazione e higher order</b>                        | <b>97</b>  |
| <b>12.1</b> | <b>Programmi strutturati</b>                                       | <b>97</b>  |
| <b>12.2</b> | <b>Astrazione</b>                                                  | <b>98</b>  |
| 12.2.1      | Sottoprogrammi e parametri                                         | 98         |
| <b>12.3</b> | <b>Higher order</b>                                                | <b>102</b> |
| <b>13</b>   | <b>Linguaggio Funzionale</b>                                       | <b>107</b> |
| <b>14</b>   | <b>Esercizi</b>                                                    | <b>109</b> |
|             | <b>Index</b>                                                       | <b>118</b> |

# Parte 1

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione .....</b>                       | <b>7</b>  |
| 1.1      | Introduzione                                    |           |
| <b>2</b> | <b>Analisi lessicale: Linguaggi regolari ..</b> | <b>11</b> |
| 2.1      | Lettura di un programma                         |           |
| 2.2      | Linguaggi                                       |           |
| 2.3      | Espressioni regolari                            |           |
| 2.4      | Automi finiti                                   |           |
| 2.5      | Grammatica                                      |           |
| <b>3</b> | <b>Analisi Semantica .....</b>                  | <b>27</b> |
| 3.1      | Introduzione alla semantica                     |           |
| 3.2      | Semantica operativa                             |           |
| <b>4</b> | <b>Espressioni aritmetiche non tipizzate ..</b> | <b>31</b> |
| 4.1      | Introduzione                                    |           |
| 4.2      | Sintassi                                        |           |
| 4.3      | Stili di semantica                              |           |
| 4.4      | Evaluation                                      |           |
| <b>5</b> | <b>Lambda Calcolo non tipizzato .....</b>       | <b>43</b> |
| 5.1      | Introduzione                                    |           |
| 5.2      | Basi sul lambda-calcolo                         |           |
| 5.3      | Programmazione con il lambda-calcolo            |           |
| 5.4      | Formalità                                       |           |





# 1. Introduzione

## 1.1 Introduzione

**Definizione 1.1.1 — Linguaggio di programmazione.** I linguaggi di programmazione sono linguaggi ingegnerizzati che permettono di insegnare alla macchina a condividere informazioni algoritmiche.

Un programma, matematicamente, è considerabile come una "funzione" che trasforma input in output.

Nei linguaggi di programmazione tutto viene costruito o **matematicamente** o **ingegneristicamente**.

Ad esempio una macchina di Turing è un dispositivo matematico composto da un insieme di istruzioni e da un'automata a stati che gestisce la testina. Essa è importante in quanto è il primo dispositivo di calcolo in grado di descrivere ogni funzione e ogni programma concepibile (secondo la tesi di Church-Turing). Tutto quello che è intuitivamente calcolabile, lo è anche con la macchina di Turing.

Nella programmazione **imperativa** è importante capire qual è l'istruzione che verrà eseguita dopo quella corrente. Quello imperativo è il primo paradigma creato, in quanto è quello più intuitivo.

Mentre con il paradigma **funzionale** ci si concentra su quale sarà il risultato della successiva applicazione delle funzioni su un input.

Tramite linguaggi che implementano il paradigma ad **oggetti** invece, i calcoli vengono eseguiti facendo comunicare oggetti(istanze di classi) che interagiscono tramite metodi.

### 1.1.1 L'evoluzione dei linguaggi

Cosa determina l'evoluzione dei linguaggi:

- **hardware:** avere a disposizione nuovi tipi di hardware consente anche di realizzare nuovi linguaggi
- **applicazioni:** lo sviluppo di nuove applicazioni(dalle prime solo numeriche, ad applicazione dove si elabora informazione non numerica)
- **metodologie:** le metodologie di programmazione influenzano lo sviluppo
- **implementazione:** come implementiamo i costrutti influenza lo sviluppo dei linguaggi, ci rendiamo conto se sono praticamente realizzabili

- **teoria:** lo sviluppo teorico influenza i linguaggi (ad esempio i tipi)

### 1.1.2 Nascita dei linguaggi di programmazione

I linguaggi di programmazione nascono con la macchina di Von Neumann (macchina a programma memorizzato). I programmi sono un particolare tipo di dato rappresentato nella memoria della macchina. La macchina possiede un interprete capace di fare eseguire il programma memorizzato, e quindi implementare un qualunque algoritmo descrivibile nel “linguaggio macchina”. Un qualunque linguaggio macchina dotato di semplici operazioni primitive per effettuare la scelta e per iterare (O simili) è Turing-equivalente, cioè può descrivere tutti gli algoritmi.



I linguaggi di programmazione **hanno tutti lo stesso potere espressivo**, ma la caratteristica distintiva importante è il quanto costa esprimere (legato al livello di astrazione definito dal linguaggio)

#### Dagli anni 50

Due linguaggi importanti furono **Fortran** e **Cobol**.

- Notazioni ad alto livello orientate rispettivamente al calcolo scientifico (numerico) ed alla gestione di dati (anche su memoria secondaria)
- Astrazione procedurale (sottoprogrammi, ma con caratteristiche molto simili a costrutti forniti dai linguaggi macchina)
- nuove operazioni e strutture dati (per esempio gli array su Fortran, e i records in Cobol)

nulla di significativamente diverso dai linguaggi macchina.

#### Dagli anni 60

nascita delle cose che studiamo:

- formalizzazione degli aspetti sintattici (BNF)
- primi risultati semantici basati sul  $\lambda$ -calcolo (teoria delle funzioni), introdotti negli anni 20 da Alonso Church (formalizzò la teoria delle funzioni). Inizia a nascere la possibilità di creare l'ambiente.

Caratteristiche comuni tra i linguaggi:

- introduzione dell'**ambiente**, vera *astrazione procedurale* con ricorsione
- argomenti procedurali e per nome

I linguaggi che nacquero furono:

- **Algol'60:** primo linguaggio imperativo veramente ad alto livello con scoping statico e gestione della memoria a stack.
- **Lisp:** primo linguaggio funzionale, direttamente ispirato al  $\lambda$ -calcolo, scoping dinamico, strutture dati dinamiche, gestione dinamica della memoria a heap con garbage collector, interpretato.
- **Simula'67:** primo linguaggio pseudo object-oriented, nasce la classe (aggiunta), estensione di Algol'60 orientato alla simulazione discreta.

#### Dagli anni 70

Studi sulle metodologie di programmazione, nascono i tipi di dati astratti, modularità, formalizzazione di classi e oggetti. Nasce la programmazione di sistema in linguaggi ad alto livello: eccezioni e concorrenza (vari threads).

I linguaggi che nacquero furono:



- **Pascal**: estensione di Algol'60 con la definizione di tipi(non astratti), l'uso esplicito di puntatori e la gestione dinamica della memoria a heap (senza garbage collector)
- **C**: Pascal + moduli + tipi astratti + eccezioni + semplice interfaccia per interagire con il sistema operativo.
- **ADA**: come sopra + concorrenza + costrutti per la programmazione in tempo reale. Quest'ultima cosa fu un progetto ambizioso, anche dal punto di vista semantico, con una grande enfasi sulla semantica statica (proprietà verificabili dal compilatore)
- **Smalltalk**: Pascal + incapsulamento e information hiding (classi che sono ora primitive e non aggiunte)

Fino ad ora abbiamo la programmazione imperativa (eseguiamo la prossima istruzione), la logica e la risoluzione alla base di un calcolo e infine un linguaggio di programmazione:

- **Prolog**: implementazione di un frammento del calcolo dei predicati del primo ordine, strutture dati molto flessibili (termini) con calcolo effettuato dall'algoritmo di unificazione, computazioni non-deterministiche, gestione della memoria a heap con garbage collector

nasce anche la programmazione funzionale. Il Lisp anche se potente è complicato da usare. I linguaggi funzionali che nacquero furono:

- **ML**: implementazione del  $\lambda$ -calcolo tipato, definizione di nuovi tipi ricorsivi, i valori dei nuovi tipi sono termini, che possono essere visitati con un meccanismo di pattern matching (versione semplificata dell'unificazione), scoping statico (a differenza di Lisp), semantica statica molto potente (inferenza e controllo dei tipi), gestione della memoria a heap con garbage collector
- **Haskell**: Come ML ma con regola di valutazione lazy

### Dagli anni 80

Nasce il Personal Computer e i seguenti linguaggi:

- **C++**: C + classi e oggetti (allocati sulla heap, ancora senza garbage collector)
- **CLP** (Constraint Logic Programming): Prolog + calcolo su domini diversi (anche numerici) con opportuni algoritmi di soluzione di vincoli nascono i calcoli per la concorrenza, ma non si evolvono ancora in linguaggi di programmazione

### Dagli anni 90

Negli anni 90 nasce:

- **Java**: possiede molte caratteristiche dei linguaggi imperativi(essenzialmente tutte quelle del linguaggio più avanzato del filone, cioè C++). Possiede alcune caratteristiche dei linguaggi del filone logico (gestione della memoria con garbage collector). Utilizza il meccanismo delle classi e dell'ereditarietà per ridurre il numero di meccanismi primitivi. Quasi tutto viene realizzato con classi predefinite nelle librerie. Ha un'implementazione mista, tipica del filone logico, che ne facilita la portabilità e lo rende particolarmente adatto ad essere integrato nelle applicazioni di rete.

### Dal 2000

Nel 2000 nascono i primi veri linguaggi concorrenti, come Erlang. Molti linguaggi special purpose, e tentativi di proporre linguaggi che abbiano molte caratteristiche (ad esempio, **Scala**)





## 2. Analisi lessicale: Linguaggi regolari

### 2.1 Lettura di un programma

Un programma solitamente è scritto utilizzando un determinato alfabeto, il quale lo rappresentiamo con la lettera  $\Sigma$ . Si hanno due strategie per trasformare il linguaggio che abbiamo utilizzato per scrivere il programma in un linguaggio macchina, il quale è direttamente comprensibile dalla macchina:

- Il **compilatore**: è un programma che traduce un programma nel linguaggio ad alto livello in un programma equivalente in linguaggio macchina. Si dicono programmi equivalenti quei programmi in cui con gli stessi input vengono prodotti gli stessi output;
- L'**interprete**: è un programma che legge un'istruzione alla volta del programma scritto in un linguaggio ad alto livello ed effettua le operazioni del linguaggio macchina corrispondenti al suo significato, quindi traduce linea per linea.

Un programma sorgente viene scritto mediante un editor che può produrre un file fatto di istruzioni e costrutti consentiti dal linguaggio di programmazione in uso. Il testo reale effettivo del file viene scritto usando caratteri di un particolare insieme o sottoinsieme di un set (alfabeto); pertanto, un sorgente può essere considerato come un flusso di caratteri terminanti con un marcatore di fine file EOF (End Of File).

Un file sorgente viene inviato come flusso ad un **lexer** carattere per carattere da una qualche interfaccia di input. Il lavoro del lexer è impacchettare il flusso di caratteri (altrimenti) insignificante in gruppi che, elaborati dal parser, acquistano significato. Ogni carattere o gruppo di caratteri raccolto in questo modo viene detto token.

**Definizione 2.1.1 — Token.** Un token è un'informazione astratta che rappresenta una stringa del testo in ingresso. Più precisamente, un token è una coppia: la sua prima componente è il **nome del token**, cioè un simbolo astratto che rappresenta una specifica categoria sintattica (ad esempio gli identificatori, gli operatori aritmetici, una parola riservata, ecc.). La seconda componente del token è il **valore del token** ed è costituita da una sequenza di simboli del testo di ingresso (ad esempio lo specifico identificatore, lo specifico operatore, ecc.).

I token sono componenti del linguaggio di programmazione in questione come parole chiave, identificatori, simboli ed operatori. Di solito il lexer rimuove commenti e spaziatura dal programma, ed ogni altro contenuto che non ha un valore semantico per l'interpretazione del programma.

Il lexer converte il flusso di caratteri in un flusso di token che hanno un significato individuale stabilito dalle regole del lexer. Il flusso di token generato dal lexer viene ricevuto in ingresso dal parser.

Un lexer di solito genera errori riguardanti le sequenze dei caratteri che non riesce a mettere in corrispondenza con gli specifici tipi di token definiti dalle sue regole. Un lexer raggruppa sequenze di caratteri che riconosce nel flusso dotate di un valore semantico individuale. Esso non considera il loro valore semantico nel contesto dell'intero programma poiché questo è dovere del parser.

I linguaggi sono descritti attraverso **grammatiche**.

**Definizione 2.1.2 — Grammatica.** Una grammatica è sostanzialmente un insieme di regole che descrivono o prescrivono la formazione di delle parole di un linguaggio, che, dal punto di vista della programmazione, sono le frasi sintatticamente corrette di un linguaggio di programmazione. Quindi, per noi, una grammatica non è altro che un insieme di regole per generare le parole del linguaggio.

Una grammatica determina esattamente quello che definisce un dato token e quali sequenze di token sono considerate valide. Il parser organizza i token che riceve in sequenze ammesse definite della grammatica del linguaggio. Se il linguaggio viene usato esattamente come definito nella grammatica, il parser sarà in grado di riconoscere i pattern che costituiscono specifiche strutture del discorso e raggrupparle insieme opportunamente. Se il parser incontra una sequenza di token che corrisponde a nessuna delle sequenze consentite, solleverà un errore e forse proverà a recuperarlo facendo alcune assunzioni sulla natura di tale errore.

Il parser genera una o più tavole dei simboli che contengono informazione riguardante i token incontrati, ad esempio se il token è il nome di una procedura o meno oppure se aveva qualche valore specifico. Le tavole dei simboli sono impiegate nella generazione del codice oggetto e nel controllo sui tipi, ad esempio, in modo che un intero non venga assegnato ad una stringa ecc. Un parser di solito genera errori che attengono alle sequenze di token che non riesce a far corrispondere alle specifiche strutture sintattiche consentite, come stabilito dalla grammatica. I lexer e i parser sono entrambi riconoscitori: i lexer riconoscono sequenze di caratteri, i parser riconoscono sequenze di token e le rendono non ambigue.

## 2.2 Linguaggi

Si consideri un insieme finito  $A$ , che chiamiamo alfabeto. Con  $A^*$  indichiamo tutte le parole di lunghezza finita sull'alfabeto  $A$ . Le parole sono una sequenza, eventualmente vuota, di simboli dell'alfabeto. Sia  $A = \{a, b\}$ , allora  $a$  è una parola, così come  $abbab$ .

**Definizione 2.2.1 — Parola vuota.** Con  $\varepsilon$  indichiamo la parola vuota, quella cioè in cui non c'è alcun simbolo dell'alfabeto.

**Definizione 2.2.2 — Concatenazione di parole.** Date due parole  $w, w' \in A^*$ , con  $w \cdot w'$  indichiamo la concatenazione delle due parole, che significa che scriviamo la parola  $w'$  alla destra della parola  $w$ , formando così una nuova parola. Osserviamo che si tratta di un'operazione non commutativa, per la quale la stringa vuota costituisce l'elemento neutro:  $\varepsilon w = w\varepsilon = w$ .

**Definizione 2.2.3 — Lunghezza di una parola.** Con  $|w|$  indichiamo la lunghezza di una parola, che è definita nel seguente modo:

$$|\varepsilon| = 0 \quad |a \cdot w| = |w| + 1$$

**Definizione 2.2.4 — Prefisso di una parola.** Sia data una parola  $w \in A^*$ , un prefisso di  $w$  è una parola  $w' \in A^*$  tale che esiste una parola  $w'' \in A^*$  e  $w = w' \cdot w''$ .

**Definizione 2.2.5 — Suffisso di una parola.** Un suffisso di una parola  $w \in A^*$  è una parola  $w'' \in A^*$  tale che esiste una parola  $w' \in A^*$  e  $w = w' \cdot w''$ .

**Definizione 2.2.6 — Infisso di una parola.** Un infisso di una parola  $w \in A^*$  è una parola  $w \in A^*$  tale che esistono due parole  $w', w'' \in A^*$  e  $w = w' \cdot w \cdot w''$ .

Consideriamo  $A = \{a, b\}$ , allora un linguaggio può essere l'insieme delle parole di lunghezza pari in cui compaiono il doppio di  $a$  rispetto alle  $b$ . Alcune parole di questo linguaggio sono  $aaaabb$ ,  $abbaaaa$ ,  $baabaa$  e così via. È bene ricordare che un linguaggio può contenere infinite parole, ma tutte le parole sono di lunghezza finita. In generale un linguaggio sarà un insieme di parole che soddisfano una certa proprietà, ad esempio, se scriviamo

$$\mathcal{L} = \{w \in A^* \mid \exists k \in \mathbb{N}. |w| = 2k + 1\}$$

indichiamo il linguaggio di tutte le parole che hanno lunghezza dispari. Se è chiaro dal contesto spesso indichiamo una proprietà con la lettera  $P$ , quindi scriviamo

$$\mathcal{L} = \{w \in A^* \mid P(w)\}$$

### 2.2.1 Operazioni sui linguaggi

Sui linguaggi sono definite varie operazioni, alcune insiemistiche, altre che riguardano le parole. Le più importanti sono l'unione, l'intersezione, il complemento, la differenza (per quanto riguarda le operazioni insiemistiche), la concatenazione e la chiusura di Kleene (per quelle che sono operazioni sulle parole). Ne diamo una breve descrizione facendo riferimento alle varie proprietà. assumiamo che  $\mathcal{L} = \{w \in A^* \mid P_i(w)\}$ . Abbiamo che:

$$\mathcal{L}_1 \cup \mathcal{L}_2 = \{w \in A^* \mid P_1(w) \text{ or } P_2(w)\} \quad (\text{Unione})$$

$$\mathcal{L}_1 \cap \mathcal{L}_2 = \{w \in A^* \mid P_1(w) \text{ and } P_2(w)\} \quad (\text{Intersezione})$$

$$\overline{\mathcal{L}} = \{w \in A^* \mid \neg P(w)\} \quad (\text{Negazione})$$

$$\mathcal{L}_1 \setminus \mathcal{L}_2 = \{w \in A^* \mid P_1(w) \text{ and } \neg P_2(w)\} \quad (\text{Differenza})$$

$$\mathcal{L}_1 \cdot \mathcal{L}_2 = \{w \in A^* \mid \exists w_1, w_2 \in A^*. w = w_1 \cdot w_2, w_1 \in \mathcal{L}_1 \text{ and } w_2 \in \mathcal{L}_2\} \quad (\text{Concatenazione})$$

**Definizione 2.2.7 — Chiusura di Kleene.** Iterando la concatenazione possiamo definire la chiusura di Kleene, indicata con  $\mathcal{L}^*$  e costituita da tutte le concatenazioni finite di elementi di  $\mathcal{L}$ . Iniziamo col definire

$$\mathcal{L}^0 = \{\varepsilon\}$$

$$\mathcal{L}^{i+1} = \{\mathcal{L}^i \cdot \mathcal{L}\}$$

$\mathcal{L}^*$  è definito come:

$$\mathcal{L}^* = \bigcup_{i=0}^{\infty} \mathcal{L}^i$$

Quando si ha  $\mathcal{L}^+$  indichiamo  $\mathcal{L} \cdot \mathcal{L}^*$  (chiusura positiva).

■ **Esempio 2.1** Dato un insieme  $\mathcal{L}$  (finito o infinito) di stringhe,  $\mathcal{L}^*$  è sempre un insieme infinito che contiene la stringa vuota  $\varepsilon$ , tutti gli elementi di  $\mathcal{L}$ , e tutte le stringhe che si ottengono mettendo in sequenza un numero finito di volte gli elementi di  $\mathcal{L}$ . Se  $\mathcal{L}$  contiene solo simboli (cioè tutti i suoi elementi sono di lunghezza 1),  $\mathcal{L}^*$  è l'insieme di tutte le possibili stringhe finite su  $\mathcal{L}$ , è cioè il più grosso linguaggio (rispetto all'inclusione insiemistica) definito sui simboli di  $\mathcal{L}$ .

**Esercizio 2.1** Considerate l'alfabeto  $A = \{0, 1\}$ , descrivete il linguaggio in cui le parole, viste come numeri, sono multipli di 2 ed hanno almeno 3 1 consecutivi. Provate a formalizzarlo e a dare degli esempi di parole del linguaggio.

**Soluzione:**

$$(0 + 1)^* \cdot 111 \cdot (0 + 1)^* \cdot 0$$

**Esercizio 2.2** Considerate il linguaggio

$$\mathcal{L} = \{w \in \{0, 1\}^* \mid w = w_1 \cdot w_2 \wedge |w_1| = 2|w_2|\}$$

sull'alfabeto  $\{0, 1\}$ . Descrivete la proprietà che caratterizza le parole del linguaggio e fornite alcuni esempi. La parola 0000 appartiene ad  $\mathcal{L}$ ? Se sì, verificate il perchè, se no, discutete come la proprietà viene violata.

**Soluzione:** Osservando la struttura del linguaggio si può notare che:

- se  $w_1$  è vuota, allora  $w_2$  deve essere vuota, e quindi  $w$  è vuota;
- se  $|w_1| = 1$  allora  $|w_2| = 2$  e quindi  $|w| = 1 + 2 = 3$
- se  $|w_1| = 2$  allora  $|w_2| = 4$  e quindi  $|w| = 2 + 4 = 6$
- se  $|w_1| = 3$  allora  $|w_2| = 6$  e quindi  $|w| = 3 + 6 = 9$
- se  $|w_1| = 4$  allora  $|w_2| = 8$  e quindi  $|w| = 4 + 8 = 12$
- ...

Si può notare che il linguaggio descritto è quello in cui la lunghezza di tutte le parole sono multipli di 3.

## 2.3 Espressioni regolari

Possiamo ora introdurre la nozione di espressione regolare, che ci serve per specificare il pattern associato ad un token. Ad ogni espressione regolare è associato in modo univoco un linguaggio, che costituisce l'insieme dei lessemi corrispondenti a quella espressione regolare. I simboli sotto-segnati vengono utilizzati per dire che la variabile è una espressione regolare.

**Definizione 2.3.1 — Espressione regolare.** Consideriamo un alfabeto  $A$ . Ad ogni simbolo dell'alfabeto  $a \in A$  associamo  $\underline{a}$ . Una espressione regolare (*e.r.*) su  $A$  è definita induttivamente come segue:

1.  $\emptyset, \varepsilon$  e  $\underline{a}$  sono espressioni regolari, dove  $a \in A$ ;
2. se  $r_1$  ed  $r_2$  sono espressioni regolari allora anche  $r_1 + r_2$  ed  $r_1 \cdot r_2$  sono espressioni regolari;
3. se  $r$  è un'espressione regolare anche  $r^*$  e  $(r)$  sono espressioni regolari;
4. niente altro è una espressione regolare.



Questo si può riscrivere anche in notazione BNF(Backus-Naur Form):

$$r ::= \underline{\quad} \mid \varepsilon \mid \underline{a} \mid r + r \mid r \cdot r \mid (r)$$

Se consideriamo  $A = \{0, 1\}$  una espressione regolare è, ad esempio, la seguente  $(0 \cdot \underline{1} + 0)^* \cdot 1$ . Spesso scriveremo  $(0\underline{1} + 0) \cdot \underline{1}$  omettendo, quando è chiaro dal contesto, il  $\cdot$  cioè la concatenazione, e ometteremo anche la “sottolineatura”, a meno che non si creino ambiguità.

Come detto prima ad ogni espressione regolare è associato in modo univoco un linguaggio. Consideriamo, ad esempio l’espressione regolare  $(0 + 1)^* 00$ . Il linguaggio denotato da questa espressione regolare è il linguaggio di tutte le parole che terminano con due 0. Per capire questo dobbiamo associare ad ogni espressione regolare un linguaggio.

**Definizione 2.3.2 — Linguaggio associato a una espressione regolare.** Il linguaggio  $\mathcal{L}$  associato ad una espressione regolare  $r$  è definito induttivamente come segue:

1.  $\mathcal{L}(\emptyset) = \emptyset$
2.  $\mathcal{L}(\varepsilon) = \{\varepsilon\}$
3.  $\mathcal{L}(a) = \{a\}$
4.  $\mathcal{L}(r_1 + r_2) = \mathcal{L}(r_1) \cup \mathcal{L}(r_2)$
5.  $\mathcal{L}(r_1 \cdot r_2) = \mathcal{L}(r_1) \cdot \mathcal{L}(r_2)$
6.  $\mathcal{L}(r^*) = \mathcal{L}(r)^*$
7.  $\mathcal{L}((r)) = \mathcal{L}(r)$

Osserviamo che il linguaggio di tutte le parole su un certo alfabeto  $A$  è dato dall’espressione regolare  $\sum_{a \in A} \underline{a}$ .

■ **Esempio 2.2** Consideriamo  $A = \{0, 1\}$ , troviamo linguaggio associato alla seguente espressione regolare

$$(0 + 1)^*$$

Utilizzando le operazioni definite precedentemente otteniamo

$$\mathcal{L}((0 + 1)^*) \stackrel{6}{\iff} (\mathcal{L}(0 + 1))^* \stackrel{4}{\iff} (\mathcal{L}(0) \cup \mathcal{L}(1))^* \stackrel{3}{\iff} (\{0\} \cup \{1\})^* = \{0, 1\}^*$$

**Esercizio 2.3** Considerate l’espressione regolare  $(00 + 11) \cdot 00(1 + \varepsilon)$ . Descrivere il linguaggio da essa denotato. Fornire almeno tre parole che appartengono al linguaggio ed uno che non appartiene.

Soluzione:

1.  $L((00 + 11) \cdot 00 \cdot (1 + \varepsilon))$
2.  $L(00 + 11) \cdot L(00 \cdot (1 + \varepsilon))$
3.  $L(00 + 11) \cdot L(00) \cdot L(1 + \varepsilon)$
4.  $L(00) \cup L(11) \cdot L(00) \cdot L(1) \cup L(\varepsilon)$
5.  $\{00\} \cup \{11\} \cdot \{00\} \cdot \{1\} \cup \{\varepsilon\}$
6.  $\{00, 11\} \cdot \{00\} \cdot \{1, \varepsilon\}$

**Definizione 2.3.3 — Linguaggio regolare.** Un linguaggio  $\mathcal{L}$  è detto regolare se esiste una espressione regolare  $r$  tale che  $\mathcal{L}(r) = \mathcal{L}$ .

La classe dei linguaggi regolari è quindi quella per cui la proprietà del linguaggio può essere in qualche modo espressa con una espressione regolare. Non esiste un modo unico per denotare un

linguaggio regolare con una espressione regolare, abbiamo quindi bisogno di dire quando due espressioni regolari sono equivalenti.

**Definizione 2.3.4 — Equivalenza di espressioni regolari.** Due espressioni regolari  $r$  ed  $r'$  sono equivalenti, scritto come  $r \equiv r'$ , se e solo se  $L(r) = L(r')$ . Ossia se rappresentano lo stesso linguaggio.

Ad esempio, si vede chiaramente che  $a + a \equiv a$ . Tra le equivalenze note abbiamo:

$$\begin{array}{lcl|lcl} r_1 + r_2 & \equiv & r_2 + r_1 & r_1 + (r_2 + r_3) & \equiv & (r_1 + r_2) + r_3 \\ r_1 \cdot (r_2 \cdot r_3) & \equiv & (r_1 \cdot r_2) \cdot r_3 & r_1 \cdot (r_2 + r_3) & \equiv & r_1 \cdot r_2 + r_1 \cdot r_3 \\ r + \emptyset & \equiv & r & r \cdot \varepsilon & \equiv & r \\ \varepsilon r & \equiv & r & r \cdot \emptyset & \equiv & \emptyset \\ \emptyset \cdot r & \equiv & \emptyset & (r^*)^* & \equiv & r^* \end{array}$$

Osserviamo anche che se  $\mathcal{L}(r) \subseteq \mathcal{L}(r')$  allora  $r + r' \equiv r'$

**Esercizio 2.4** Provare che  $(0110 + 01)(10)^* \equiv 01(10)^*$  ■

**Esercizio 2.5** Date due espressioni regolari  $r_1$  ed  $r_2$ , provare che  $(r_1^* + r_2^*)^* \equiv (r_1 + r_2)^*$  ■

**Esercizio 2.6** Date due espressioni regolari  $r_1$  ed  $r_2$ , provare che  $(r_1^* \cdot r_2^*)^* \equiv (r_1 + r_2)^*$  ■

**Esercizio 2.7** Data l'espressione regolare  $r$ , provare che  $(r^*)^* \equiv r^*$ . ■

Le espressioni regolari vengono usate anche per stabilire se una certa parola soddisfa un certo pattern, ad esempio, se si vuole verificare che la parola  $w$  contiene la sequenza 001 possiamo vedere se  $w \in \mathcal{L}((0 + 1)^*001(0 + 1)^*)$ , dato che il linguaggio denotato dall'espressione regolare  $(0 + 1)^*001(0 + 1)^*$  è fatto da tutte le parole su  $\{0, 1\}$  dove appare il pattern 001.

■ **Esempio 2.3** Fissiamo l'alfabeto  $A = \{a, b\}$ .

- L'espressione  $ab + b$  definisce il linguaggio  $\{ab, b\}$ .
- $a(a + b)b$  definisce il linguaggio  $\{aab, abb\}$ .
- $a^*$  definisce  $\{\varepsilon, a, aa, aaa, \dots\} = \{a^n \mid n \geq 0\}$ , cioè il linguaggio di tutte le stringhe finite costituite da sole  $a$ .
- $b^*a \mid c$  definisce  $\{c\} \cup \{b^n a \mid n \geq 0\}$

■ **Esempio 2.4** Sia  $A = \{0, 1\}$  un alfabeto e data l'espressione regolare  $r$ :

$$r = 00 \cdot (1 + 00)^* \cdot 01 \cdot (1 + 11)^* \cdot (0 + \varepsilon)$$

Analizzare il linguaggio descritto da  $r$ .

Le parole appartenenti al linguaggio dovranno rispettare le seguenti restrizioni:

- Deve iniziare con due zeri  
 $L(00) = \{00\}$
- Poi ci può essere una sequenza di 00 e 1  
 $L((1 + 00)^*) = (L(1 + 00))^* = (L(1) \cup L(00))^* = \{1, 00\}^*$
- poi deve esserci 01  
 $L(01) = \{01\}$

- Poi può esserci una sequenza di 1  
 $L((1 + 11)^*) = L(1^*) = \{1\}^*$
- Infine ci può essere uno zero oppure un carattere vuoto  
 $L(0 + \varepsilon) = \{0, \varepsilon\}$

Possiamo ora osservare che un linguaggio finito è certamente regolare, dato che ogni sua parola può essere descritta da un'espressione regolare e l'unione di tutte queste caratterizza precisamente il linguaggio. Un linguaggio finito è, ad esempio, quello di tutte le keywords di un linguaggio di programmazione.

**Esercizio 2.8** Considerato il linguaggio  $\{fido, pimpa, fischio, piroetta, fitta\}$ . Scrivere un'espressione regolare che lo denota. ■

## 2.4 Automi finiti

Fino ad ora abbiamo visto come denotare un linguaggio regolare e come descriverlo con le espressioni regolari, ma non abbiamo considerato come riconoscerlo. Per questo introduciamo la nozione di automa a stati finiti che abbrevieremo spesso con FA (da Finite Automaton), premettendo eventualmente o la D di deterministico o la N di non-deterministico.

**Definizione 2.4.1 — Automa a stati finiti deterministico.** Un automa a stati finiti deterministico (DFA) è una quintupla

$$\mathcal{A} = (Q, A, \delta, q_0, F)$$

dove:

- $Q$  è un insieme finito di stati;
- $A$  è un alfabeto
- $\delta = Q \times A \rightarrow Q$  è la funzione di transizione
- $q_0 \in Q$  è lo stato iniziale
- $F \subseteq Q$  è l'insieme degli stati finali

Un automa ha anche un nastro di input dove viene scritta la parola che deve essere accettata. Se un automa si trova in uno stato  $q$  e la testina del nastro d'input legge il simbolo  $a$  appartenente all'alfabeto, allora l'automato va nello stato  $q'$  e la testina si sposta sul simbolo successivo, dove  $q' = \delta(q, a)$ . L'automato accetta la parola se la legge tutta e termina in uno stato finale.

Formalmente estendiamo la funzione  $\delta$  in modo che il suo dominio sia non  $Q \times A$  ma  $Q \times A^*$  chiamandola  $\delta^*$  in questo modo:

- $\delta^*(q, \varepsilon) = q$
- $\delta^*(q, aw) = \delta^*(\delta(q, a), w)$ .

**Definizione 2.4.2 — Linguaggio accettato da un DFA.** Sia  $\mathcal{A} = (Q, A, \delta, q_0, F)$  un DFA. Il linguaggio accettato da  $\mathcal{A}$ , che chiamiamo  $\mathcal{L}(\mathcal{A})$  è l'insieme  $\{w \in A^* \mid \delta^*(q_0, w) \in F\}$ . Ossia l'insieme di tutte le parole che portano l'automato in uno stato finale.

Un FA può essere anche non deterministico

**Definizione 2.4.3 — Automa a stati finiti non-deterministico.** Un automa a stati finiti non-deterministico (NFA) è la quintupla

$$\mathcal{A} = (Q, A, \delta, q_0, F)$$

dove:

- $Q$  è un insieme finito di stati;
- $A$  è un alfabeto
- $\delta = Q \times A \rightarrow 2^Q$  è la funzione di transizione
- $q_0 \in Q$  è lo stato iniziale
- $F \subseteq Q$  è l'insieme degli stati finali

In questo caso la funzione di transizione specifica in quali stati si può andare leggendo  $a$  ed essendo nello stato  $q$ . Anche la funzione  $\delta^*$  cambia leggermente dato che ora deve restituire un insieme di stati, quindi

- $\delta^*(q, \varepsilon) = \{q\}$
- $\delta^*(q, aw) = \bigcup_{q' \in \delta(q, a)} \delta^*(q', w)$ .

Infine abbiamo che il linguaggio accettato è:

**Definizione 2.4.4 — Linguaggio accettato da un NFA.** Sia  $\mathcal{A} = (Q, A, \delta, q_0, F)$  un NFA. Il linguaggio accettato da  $\mathcal{A}$ , che chiamiamo  $\mathcal{L}(\mathcal{A})$  è l'insieme  $\{w \in A^* \mid (\delta^*(q_0, w) \cap F) \neq \emptyset\}$ . Ossia l'insieme di tutte le parole che portano l'automa in almeno uno stato finale.

Il risultato importante è che ogni NFA può essere trasformato in un DFA che accetta lo stesso linguaggio.

**Teorema 2.4.1** Dato un automa NFA  $\mathcal{A}$ , esiste un  $\mathcal{A}'$  effettivamente costruibile da  $\mathcal{A}$  tale che  $\mathcal{L}(\mathcal{A}) = \mathcal{L}(\mathcal{A}')$

Osservando che un DFA può essere trasformato in un NFA semplicemente  $\delta(q, a) = q'$ , abbiamo che il non-determinismo non aggiunge potere espressivo. La prova di questo teorema è semplice. Partiamo dall'NFA  $\mathcal{A} = (Q, A, \delta, q_0, F)$ , costruiamo un DFA

$$\mathcal{D}(\mathcal{A}) = (2^Q, A, \hat{\delta}, \{q_0\}, \hat{F})$$

dove

- $\hat{\delta}(X, a) = \bigcup_{q \in X} \delta(q, a)$
- $\hat{F} = \{X \subseteq Q \mid (X \cap F) \neq \emptyset\}$ .

Questo automa è certamente deterministico ed è facile vedere che, per ogni parola  $w \in A^*$ , se  $(\delta^*(q_0, w) \cap F) \neq \emptyset$  allora  $\hat{\delta}^*(\{q_0\}, w) \in \hat{F}$ .

Dato un DFA  $\mathcal{A}$ , ne esiste uno che accetta il medesimo linguaggio e che ha almeno meno stati di  $\mathcal{A}$ . Questo implica che se la misura è il numero degli stati, allora ne esiste uno minimo.

Infine citiamo i risultati che collegano FA ed espressioni regolari.

**Teorema 2.4.2** Sia  $r$  una espressione regolare e  $L(r)$  il linguaggio ad esso associato. Allora esiste un DFA  $\mathcal{A}$  tale che  $\mathcal{L}(\mathcal{A}) = L(r)$ .

**Corollario 2.4.3** Sia  $\mathcal{A}$  una DFA e  $\mathcal{L}(\mathcal{A})$  il linguaggio ad esso associato. Allora esiste una espressione regolare  $r$  tale che  $\mathcal{L}(\mathcal{A}) = L(r)$ .

■ **Esempio 2.5** Supponiamo di voler trovare l'espressione regolare che descriva il seguente linguaggio:

$$\mathcal{L} = \{w \in 0, 1^* \mid w \text{ termina con il simbolo } 1\} = \{0, 1\}^* \cdot \{1\}$$

Per farlo ci basta pensare a come sono strutturate le parole che devono essere accettate dal linguaggio:

- le parole devono terminare col simbolo 1;
- la parte precedente all'1 finale può essere una qualsiasi concatenazione di 0 e 1.

Alcune parole che appartengono (e non) al linguaggio sono:

|                                  |                       |
|----------------------------------|-----------------------|
| $\varepsilon \notin \mathcal{L}$ | $1 \in \mathcal{L}$   |
| $0 \notin \mathcal{L}$           | $01 \in \mathcal{L}$  |
| $11 \in \mathcal{L}$             | $001 \in \mathcal{L}$ |
| $100 \notin \mathcal{L}$         | $101 \in \mathcal{L}$ |

Possiamo quindi trovare l'espressione regolare, che sarà

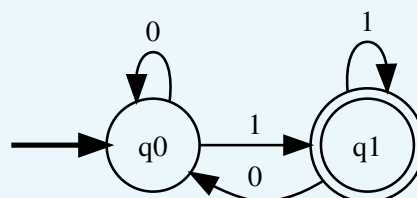
$$r = (0|1)^*1$$

Possiamo verificare se è giusto utilizzando le proprietà:

$$\begin{aligned}
 L((0|1)^*1) &= L((0|1)^*) \cdot L(1) \\
 &= (L(0) \cup L(1))^* \cdot L(1) \\
 &= (L(0)^* \cup L(1)^*) \cdot L(1) \\
 &= (\{0\}^* \cup \{1\}^*) \cdot \{1\} \\
 &= \{0, 1\}^* \cdot \{1\} \\
 &= \mathcal{L}
 \end{aligned}$$

L'uguaglianza è vera, quindi abbiamo verificato che l'espressione regolare trovata descrive il linguaggio.

Il DFA che descrive il linguaggio è il seguente:



ed è rappresentato dalla funzione di transizione seguente:

$$\begin{aligned}
 \delta(q_0, 0) &= q_0 \\
 \delta(q_0, 1) &= q_1 \\
 \delta(q_1, 0) &= q_0 \\
 \delta(q_1, 1) &= q_1
 \end{aligned}$$

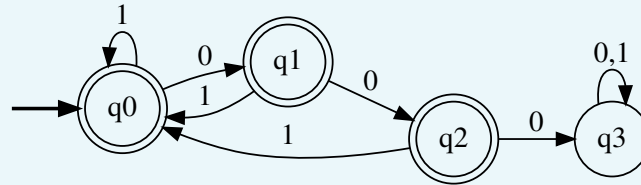
**Esercizio 2.9** Trovare l'espressione regolare che rappresenta il seguente linguaggio

$$L = \{w \in A^* \mid w \text{ non contiene mai 3 zeri di seguito}\}$$

**Soluzione** L'espressione regolare che rappresenta il linguaggio è

$$(0 + \varepsilon) \cdot (10 + 1 + 01 + 101)^* \cdot (0 + \varepsilon)$$

ed è rappresentata dal seguente DFA:



## 2.5 Grammatica

Diamone prima una definizione informale:

una grammatica è sostanzialmente un insieme di regole che descrivono o prescrivono la formazione di parole di un linguaggio, che, dal punto di vista della programmazione, sono le frasi sintatticamente corrette di un linguaggio di programmazione. Quindi, per noi, una grammatica non è altro che un insieme di regole per generare le parole del linguaggio. Prima di tutto formalizziamo la nozione di produzione.

**Definizione 2.5.1 — Produzione su un alfabeto.** Sia  $A$  un alfabeto. Una produzione su  $A$  è una coppia  $(\alpha, \beta)$  tale che

1.  $\alpha, \beta \in A^*$
2.  $\alpha \neq \varepsilon$

L'idea che sta dietro a questa nozione è banale. Devo riscrivere il lato sinistro della produzione ( $\alpha$ ) con in lato destro ( $\beta$ ), all'interno di una parola dove compare, come infisso, il lato sinistro della produzione. La seconda condizione impone che non si possa riscrivere la parola vuota, come sarà chiaro nel seguito. La produzione  $(\alpha, \beta)$  viene normalmente scritta come  $\alpha \rightarrow \beta$ . Osserviamo che le produzioni su  $A$  non sono altro che una relazione su  $A^* \times A^*$ .

Le produzioni che considereremo saranno tali che  $|\alpha| = 1$ , cioè si riscrive un solo simbolo.

**Definizione 2.5.2 — Grammatica.** Una grammatica su  $A$  è la quadrupla  $G = (\mathbf{V}, A, \mathcal{P}, S)$  dove

- $\mathbf{V}$  è un insieme finito di simboli non terminali, e  $A$  è un insieme finito di simboli terminali tali che  $\mathbf{V} \cap A = \emptyset$
- $\mathcal{P}$  è un insieme di produzioni su  $\mathbf{V} \cup A$  tali che il primo elemento della coppia appartiene sempre a  $\mathbf{V}$ , quindi  $\mathcal{P} \subseteq (\mathbf{V} \cup A)^* \times (\mathbf{V} \cup A)^*$  tale che per ogni  $\alpha \rightarrow \beta \in \mathcal{P}$  abbiamo che  $|\alpha| = 1$  e  $\alpha \in \mathbf{V}$
- $S \in \mathbf{V}$  è il simbolo iniziale

Prima di formalizzare cosa è una derivazione in una grammatica, diamone un esempio:

■ **Esempio 2.6** Consideriamo la grammatica:

$$G = (\{S\}, \{0, 1\}, \{S \rightarrow 0S0, S \rightarrow 1S1, S \rightarrow \varepsilon\}, S)$$



come la usiamo? L'idea è di derivare parole su  $A^*$  applicando le regole definite dalle produzioni, partendo dal simbolo iniziale  $S$ . Supponiamo di voler derivare la parola 011110. Dato che le produzioni delle grammatiche che stiamo considerando riscrivono solo simboli non terminali, dobbiamo trovare all'interno della parola che stiamo riscrivendo il simbolo non terminale che è il lato sinistro di una qualche produzione. Quindi da  $S$  deriviamo  $0S0$  usando la produzione  $S \rightarrow 0S0$ , poi, da questa, deriviamo  $01S10$  usando la produzione  $S \rightarrow 1S1$ , poi deriviamo  $011S110$  usando la medesima produzione  $S \rightarrow 1S1$  ed infine deriviamo  $011110$  usando la produzione  $S \rightarrow \varepsilon$ .

Data una grammatica  $G = (\mathbf{V}, A, \mathcal{P}, S)$ , un passo di derivazione è definito come segue:

**Definizione 2.5.3 — Passo di derivazione.** Sia  $G = (\mathbf{V}, A, \mathcal{P}, S)$  una grammatica e  $a \in (\mathbf{V} \cup A)^+$ . Un passo di derivazione  $\alpha \Rightarrow \gamma$  usando una produzione  $V : i \rightarrow \alpha_i$  è definito come segue:

- $\alpha = \beta V_i \delta$ , con  $\beta, \delta \in (\mathbf{V} \cup A)^*$
- $\gamma = \beta \alpha_i \delta$

■ **Esempio 2.7** Per spiegare meglio la reale applicazione di un passo di derivazione viene presentato un esempio. Utilizzando il precedente esempio abbiamo:

$$0S0 \xrightarrow{S \rightarrow 1S1} 01S10$$

In questo caso il passo di derivazione è definito come segue:

- $\alpha = 0S0$
- $\beta = 0$
- $V = S$
- $\delta = 0$
- $\gamma = 01S10$

Una derivazione non è altro che la chiusura riflessiva e transitiva della relazione " $\Rightarrow$ "

**Definizione 2.5.4 — Derivazione di parole.** Sia  $G = (\mathbf{V}, A, \mathcal{P}, S)$  una grammatica e  $a \in (\mathbf{V} \cup A)^+$ . Deriviamo da  $\alpha \in (\mathbf{V} \cup A)^+$  la parola  $\beta \in (\mathbf{V} \cup A)^+$  se e solo se  $\alpha \xRightarrow{*} \beta$ .

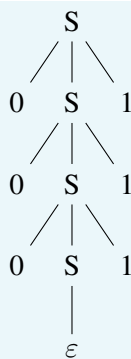
Per indicare che deriviamo  $\beta$  da  $\alpha$  scriveremo  $\alpha \xRightarrow{*} \beta$ . Chiaramente questa definizione implica che esiste una sequenza finita di passi che consentono di arrivare da  $\alpha$  a  $\beta$  con passi di derivazione successivi. Vediamo un altro esempio:

■ **Esempio 2.8** Data la grammatica

$$G = (\{S\}, \{0, 1\}, \{S \rightarrow 0S1, S \rightarrow \varepsilon\}, S)$$

deriviamo a partire da  $S$ :

$$\begin{array}{ll} S \Rightarrow 0S1 & \text{usando } S \rightarrow 0S1 \\ 0S1 \Rightarrow 00S11 & \text{usando } S \rightarrow 0S1 \\ 00S11 \Rightarrow 000S111 & \text{usando } S \rightarrow 0S1 \\ 000S111 \Rightarrow 000111 & \text{usando } S \rightarrow \varepsilon \end{array}$$



Osserviamo che abbiamo derivato la parola  $0^3 1^3$ , e possiamo immaginare che il linguaggio generato da questa grammatica sia il linguaggio di tutte le parole tali che contengono tanti 0 quanti 1 e tutti gli 0 stanno a sinistra degli 1.

Possiamo ora formalizzare quale è il linguaggio generato da una grammatica  $G$

**Definizione 2.5.5 — Linguaggio generato da una grammatica.** Sia  $G = (\mathbf{V}, A, \mathcal{P}, S)$  una grammatica. Il linguaggio generato da  $G$  è l'insieme delle parole fatte da soli simboli terminali derivabili da  $S$ . Quindi

$$\mathcal{L}(G) = \{w \in A^* \mid S \xRightarrow{*} w\}$$

Sia  $G = (\{S\}, \{0, 1\}, \{S \rightarrow 0S1, S \rightarrow \varepsilon\}, S)$  la grammatica. Si può facilmente vedere che  $\mathcal{L}(G) = \{0^i 1^i \mid i \geq 0\}$ .

Abbiamo formalizzato cosa è un passo di derivazione, e tramite questa nozione abbiamo anche stabilito cosa è un linguaggio. Vediamo ora una caratterizzazione alternativa alla nozione di derivazione, che sono gli alberi sintattici. Ricordiamo che un albero è un grafo etichettato con un vertice designato, detto radice e tale che ogni vertice è raggiungibile dalla radice con un unico cammino.

**Definizione 2.5.6 — Grafo.** Un grafo etichettato su insieme di etichette  $\mathcal{E}$  è la quadrupla  $(V, A, v_{in}, \ell)$  dove:

- $X$  è un insieme di vertici
- $A \subseteq X \times X$  è insieme di archi
- $v_{in} \in V$  è un vertice detto radice
- $\ell : V \rightarrow \mathcal{E}$  è una funzione di etichettatura

Definiamo cosa è un cammino in un albero:

**Definizione 2.5.7 — Cammino di un albero.** Sia  $(V, A, v_{in}, \ell)$  un grafo, un cammino  $\rho$  su tale grafo è una sequenza di vertici  $v_0, \dots, v_n$  tale che  $\forall 1 \leq i \leq n. (v_{i-1}, v_i) \in A$  del cammino  $\rho$  diciamo che parte da  $v_0$  ed arriva a  $v$ .

Possiamo ora dare la nozione di albero:

**Definizione 2.5.8 — Albero.** Sia  $(V, A, v_{in}, \ell)$  un grafo, allora  $(V, A, v_{in}, \ell)$  è un albero se e solo se per tutti i vertici  $v \in V$  esiste un unico cammino da  $v_{in}$  a  $v$

Di un albero diciamo che un vertice  $v$  è un nodo interno se esiste un vertice  $v'$  tale che  $(v, v') \in A$ , mentre  $v$  è una foglia se non è un nodo interno. Dato un nodo interno  $v$ , allora chiamiamo  $v'$  nodo figlio. Dato un vertice  $v$ , i suoi figli sono l'insieme  $\{v' \mid (v, v') \in A\}$ , e questo insieme è indicato con  $\text{succ}(v)$ . Osserviamo che una foglia  $v$  è tale che  $\text{succ}(v) = \emptyset$ . Per noi sarà importante

sottolineare che per ogni nodo interno (che quindi ha dei figli) l'ordine con cui sono elencati i figli (scritti da sinistra verso destra) è rilevante.

Possiamo formalizzare la nozione di albero sintattico

**Definizione 2.5.9 — Albero sintattico.** Sia  $G = (\mathbf{V}, A, \mathcal{P}, \mathcal{S})$  una grammatica. Un albero sintattico di  $G$  è un albero  $(V, A, v_{in}, \ell)$  etichettato su  $\mathbf{V} \cup A \cup \{\varepsilon\}$  tale che

- esiste una relazione  $\prec \subseteq V \times V$
- ogni vertice (interno o foglia) ha un'etichetta in  $\mathbf{V} \cup A \cup \{\varepsilon\}$
- ogni vertice interno è etichettato con una variabile in  $\mathbf{V}$
- ogni foglia è etichettata con un simbolo da  $V \cup A \cup \{\varepsilon\}$
- se un vertice  $v$  è etichettato con  $X$  ed i suoi figli  $\text{succ}(v) = \{v_1, \dots, v_n\}$  sono etichettati con  $X_1, X_2, \dots, X_n$  allora
  - $\text{succ}(v) \neq \emptyset$
  - $\prec^* \cap (\text{succ}(v) \times \text{succ}(v))$  è un ordine totale
  - c'è una produzione  $X \rightarrow X_1 X_2 \dots X_n$  tale che  $\ell(v_i) = X_i$  e  $\forall i \leq j \leq n. V_i \prec^* v_j$

Perchè abbiamo introdotto la nozione di albero sintattico? L'idea è che se leggiamo le foglie da sinistra a destra possiamo trovare una stringa derivabile dal simbolo non terminale che è nella radice.

Collegiamo gli alberi sintattici alle derivazione

**Teorema 2.5.1** Sia  $G = (\mathbf{V}, A, \mathcal{P}, \mathcal{S})$  una grammatica. Allora  $w$  è derivabile da  $X \in \mathbf{V}$  se e solo se esiste un albero sintattico in cui leggiamo  $w$  e che ha come etichetta del nodo radice  $X$

Avendo mostrato che sia l'albero sintattico che le derivazioni sono equivalenti abbiamo che, data una grammatica, ci sono vari modi per ottenere le parole appartenenti al linguaggio generato dalla grammatica:

- costruire una derivazione
- costruire un albero sintattico

I due metodi sono equivalenti

#### Esercizio 2.10

1. Scrivere una grammatica che genera tutte le parole palindrome (cioè tali che  $w = w^R$ ) su  $\{0, 1\}$
2. Sia  $G$  una grammatica in cui nessuna produzione è della forma  $A \rightarrow \varepsilon$ . Mostrare che se  $w \in \mathcal{L}(G)$ ,  $|w| = n$  e ha una derivazione di  $m$  passi, allora  $w$  ha un albero sintattico di  $n + m$  nodi

■

#### Esercizio 2.11 Scrivere una grammatica che genera $\{a^i b^j\}$

##### Soluzione

Una grammatica che accetta parole di parole con  $|a| = i$  e  $|b| = j$  sarebbe il seguente

$$G = \{\{S\}, \{a, b\}, \mathcal{P}, S\}$$

con

$$\mathcal{P} = \left\{ \begin{array}{ll} S & \rightarrow aS \mid A, \\ A & \rightarrow bA \mid \varepsilon \end{array} \right\}$$

Ad esempio, se proviamo a generare una parola con questa grammatica otteniamo

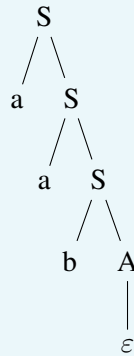


Figura 2.1: Albero di aab

**Esercizio 2.12** Scrivere una grammatica che genera

$$\{w \in \{0, 1\}^* \mid w \text{ interpretato come numero binario divisibile per } 8\}$$

**Soluzione**

Un numero in binario è divisibile per 8 se le ultime 3 cifre meno significative sono zeri. Quindi il linguaggio da generare è quello associato alla seguente espressione regolare

$$(0 + 1)^*000$$

Questo linguaggio è rappresentabile dalla grammatica  $G = \{\{S\}, \{0, 1\}, \mathcal{P}, S\}$  con

$$\mathcal{P} = \left\{ \begin{array}{ll} S & \rightarrow 0S \mid 1S \mid 0A, \\ A & \rightarrow 0B, \\ B & \rightarrow 0 \end{array} \right\}$$

**Esercizio 2.13** Scrivere una grammatica che genera tutte le parole su  $\{0, 1\}$  che hanno lo stesso numero di zero e di uno.

**Soluzione**

Una possibile grammatica che genera tutte le parole su 0, 1 che hanno lo stesso numero di zero e di uno è la seguente:

$$G = \{\{S\}, \{0, 1\}, \mathcal{P}, S\}$$

con  $\mathcal{P}$

$$\mathcal{P} = \left\{ \begin{array}{ll} S & \rightarrow 0B \mid 1A, \\ A & \rightarrow 0 \mid 0S \mid 1AA \\ B & \rightarrow 1 \mid 1S \mid 0BB \end{array} \right\}$$

Questa grammatica genera il linguaggio

$$\mathcal{L} = \{w \in \{0, 1\}^* \mid |0| = |1|\}$$

Questo esercizio è un po' difficile da immaginare. I vari simboli non terminali possono essere interpretati nel seguente modo:

- $S$  è una sorta di stato in cui è possibile aggiungere un 0 o un 1. Se aggiungiamo un 0, andiamo allo stato  $B$ , se aggiungiamo un 1, andiamo allo stato  $A$
- $A$  è considerabile come una sorta di stato in cui abbiamo già aggiunto un 1, e dobbiamo quindi compensare con uno 0. Se aggiungiamo uno 0, possiamo terminare la parola o continuare allo stato  $S$ . Se aggiungiamo un ulteriore 1, dobbiamo aggiungere un altro 0, e quindi aggiungiamo un ulteriore  $A$  alla parola
- $B$  è considerabile come una sorta di stato in cui abbiamo già aggiunto un 0, e dobbiamo quindi compensare con uno 1. Se aggiungiamo uno 1, possiamo terminare la parola o continuare allo stato  $S$ . Se aggiungiamo un ulteriore 0, dobbiamo aggiungere un altro 1, e quindi aggiungiamo un ulteriore  $B$  alla parola.

**Esercizio 2.14** Considerata la grammatica  $G = \{\{S\}, \{0, 1\}, \mathcal{P}, S\}$  con

$$\mathcal{P} = \left\{ \begin{array}{ll} S & \rightarrow 0S \mid 1S \mid 0A \mid 1A, \\ A & \rightarrow 0B, \\ B & \rightarrow 0C, \\ C & \rightarrow 0 \end{array} \right\}$$

Trovare gli alberi sintattici di 001000 e 11000

### Soluzione

Per poter trovare gli alberi sintattici bisogna prima analizzare le produzioni del linguaggio. Il simbolo iniziale è  $S$ , a partire da esso possiamo concatenare 0, 1 oppure  $A$ . Se viene concatenato  $A$ , saremo obbligati a utilizzare le produzioni di  $A$ , che portano obbligatoriamente alla concatenazione di 3 zeri e alla conclusione della parola, in quanto:

$$A \rightarrow 0B \rightarrow 00C \rightarrow 000$$

Quindi in breve il linguaggio generato dalla grammatica  $G$  è quello associato alla seguente espressione regolare

$$(0 + 1)^* 000$$

Gli alberi sintattici sono i seguenti:

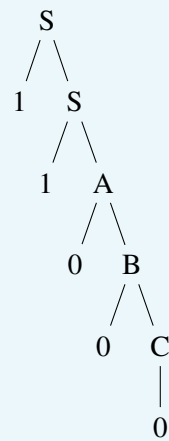


Figura 2.2: albero di 11000

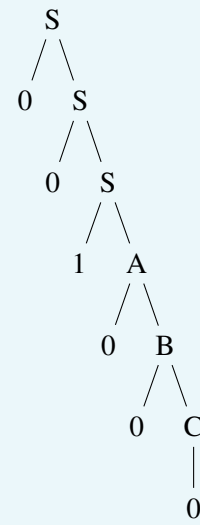


Figura 2.3: albero di 001000



## 3. Analisi Semantica

### 3.1 Introduzione alla semantica

Dare la semantica ad un programma serve a definire con esattezza cosa fa questo programma, e quindi anche a sviluppare tecniche di ottimizzazione, di debugging, oppure fare verifiche di correttezza, fare traduzione fra formalismi diversi, e molto altro, che sono parte di quello che, nella figura precedente, sono argomenti che non verranno trattati.

La definizione semi-formale della semantica fa parte dello standard di definizione di un linguaggio di programmazione a partire dagli anni '80, ma la descrizione di come devono funzionare i costrutti ha bisogno di un approccio formale. Ci sono vari metodi per definire la semantica di un linguaggio di programmazione, che qui elenchiamo, cercando di evidenziarne le caratteristiche salienti, senza alcuna pretesa di completezza.

- **Algebrica:** Nasce agli inizi anni degli '70, si parte dai tipi di dato che formano un'algebra (spesso detta multi-sorted). La sintassi è data dalla cosiddetta *term algebra*, le operazioni nelle algebre semantiche sono definite tramite equazioni, e la semantica è l'omeomorfismo più generale dalla term algebra alle algebre semantiche. Concetti come tipo di dato astratto (moduli, classi, etc) sono ben descritti da questa semantica.
- **Assiomatica:** Tra le prime ad essere formalizzata, caratterizza gli stati (proprietà), e per ogni costrutto si cerca di capire come è la trasformazione indotta nello stato formalizzata come proprietà dello stato e si basa su una descrizione (più o meno formale) di cosa fa un costrutto. Tale semantica è molto indicata per la verifica di correttezza di un programma, e quindi per lo sviluppo di programmi corretti.
- **Operazionale:** Ottenuta definendo un interprete del linguaggio su di una macchina ospite le cui componenti sono descritte in modo matematico. Lo stato descrive complessivamente il sistema, e la transizione specifica un passo di computazione che modifica lo stato e/o il programma. Una computazione è una sequenza di transizioni. È utile soprattutto perché fornisce direttamente un modello di implementazione.
- **Denotazionale:** Formalizza quale è la trasformazione sugli stati indotta dal programma, sostanzialmente una funzione dagli input agli output di un programma, ed è focalizzata su quale è la trasformazione e non su come tale trasformazione avviene, e poggia su basi matematiche sostanzialmente sviluppate negli anni '70. (denoto = esprimo con chiarezza, indico)



Prima di discutere più formalmente la semantica operativa di un linguaggio di programmazione, diamo la nozione di paradigma.

### 3.1.1 Paradigma di programmazione

Un paradigma di programmazione è definito come segue:

**Definizione 3.1.1 — Paradigma di programmazione.** Un paradigma di programmazione è un insieme di strumenti concettuali forniti da un linguaggio di programmazione per la stesura del codice sorgente di un programma. Esso definisce il modo in cui il programmatore concepisce e percepisce il programma stesso, e quindi anche la sua esecuzione.

I diversi paradigmi si differenziano per i concetti e le astrazioni usate per rappresentare gli elementi di un programma, come ad esempio le funzioni, gli oggetti, le variabili o i vincoli, e anche per i procedimenti usati per l'esecuzione delle procedure di elaborazione dei dati, come l'assegnamento, l'iterazione, la ricorsione, il data flow, l'applicazione di funzioni.

Facciamo un elenco di paradigmi di programmazione, senza la pretesa di esaurirli tutti.

**Definizione 3.1.2 — Paradigma imperativo.** Nel paradigma imperativo è il programmatore a stabilire quale sarà l'istruzione successiva. Quindi è la scrittura del programma stesso a dire quale sarà la sua esecuzione. Il modello di base è quello dei primi linguaggi di programmazione, come il Fortran, quindi spesso ci si concentra sull'algoritmo visto come sequenza di passi. Un passo di computazione in generale modifica lo stato (concetto che vedremo meglio dopo).

**Definizione 3.1.3 — Paradigma funzionale.** In questo paradigma il programmatore definisce quali sono le funzioni e come queste vengono invocate ed usate. Il focus non è tanto sull'algoritmo come sequenza di passi ma su quali sono le funzioni per risolvere il problema e quali risultati producono. Il modello è il  $\lambda$ -calcolo. Un passo di computazione in generale restituisce un valore.

**Definizione 3.1.4 — Paradigma procedurale.** In questo paradigma si prova a coniugare la parte imperativa con la definizione di funzioni, che in questo paradigma vengono chiamate procedure. Un passo di computazioni in generale ha un effetto sullo stato ma anche, potenzialmente, restituisce un valore. I linguaggi come Pascal o Algol possono essere presi come linguaggi propri di questo paradigma.

**Definizione 3.1.5 — Paradigma object oriented.** In questo paradigma le varie parti di programma interagiscono scambiandosi messaggi. Sono quindi questi messaggi, scambiato tra parti di programma che sono visti come oggetti medesimi. Il linguaggio di programmazione paradigmatico è lo SmallTalk, e la sua precisa semantica è quella algebrica.

**Definizione 3.1.6 — Paradigma logico.** la logica, in particolare le clausole di Horn, sono alla base di questo paradigma di programmazione. Il linguaggio proprio di questo paradigma è il Prolog

## 3.2 Semantica operativa

Possiamo definirla in modo formale, guidati dalla sintassi di un linguaggio di programmazione, o meglio dalla sintassi dei vari costrutti. Seguiamo il cosiddetto approccio **SOS** (Structured Operational Semantics). Per prima cosa dobbiamo definire cosa è lo stato rispetto al quale valutiamo (eseguiamo) un costrutto.

La semantica operativa è per noi definita rispetto a una nozione di stato. Dato un programma  $p$ , con **Name**( $p$ ) indichiamo l'insieme dei *nomi* che compaiono nel programma. Saranno quindi tutti gli *identificatori* come i nomi di variabili o simili. Non ne diamo una definizione formale. I valori che possono essere associati ai vari nomi li indichiamo con **Value** e non dipendono dal programma medesimo, e anche in questo caso non ne diamo una definizione formale.

**Definizione 3.2.1 — Stato.** Dato un programma  $p$ , lo stato è un'associazione tra **Name**( $p$ ) e **Value**.

La definizione è generica, dato che non si specifica precisamente come è fatta l'associazione tra nomi e valori. Quindi, per noi (e per ora), lo stato può essere una sequenza finita di coppie (in cui il primo elemento, cioè il nome della variabile, compare una sola volta). Ogni coppia  $(X, n)$  ha come primo elemento il nome (in questo caso  $X$ ) e come secondo elemento il valore (in questo caso  $n$ ). Possiamo più semplicemente vedere lo stato come una funzione dai nomi (**Name**( $p$ )) ai valori associabili ai nomi (**Value**).

La nozione di stato dipende anche molto dal paradigma che consideriamo. Ad esempio, nel paradigma imperativo e in quello procedurale, lo stato ha una parte che chiameremo ambiente (una funzione da nomi ad oggetti che chiameremo denotabili), una memoria, che sarà una funzione da locazioni (che assumeremo denotabili) a oggetti memorizzabili, uno heap, che sarà una struttura dati per la memorizzazione delle strutture dati dinamiche, mentre nel paradigma funzionale avremo solo l'ambiente, cioè l'associazione tra nomi e oggetti denotabili.

Prima di introdurre un qualche linguaggio e di definirne la semantica, bisogna stabilire cosa fa una semantica operativa. Ovviamente questo dipende dal linguaggio e dal paradigma a cui tale linguaggio appartiene. Nel caso di linguaggi imperativi la semantica operativa definisce un cambiamento di stato, mentre nei linguaggi funzionali la semantica operativa restituisce il risultato della valutazione, quindi un valore.

Una computazione è in generale una catena di cambiamenti, di stato nel caso imperativo e di valori nel caso funzionale. Questi cambiamenti vengono descritti da opportune regole, che hanno la forma:

$$\frac{\text{premessa}}{\text{conseguenza}}$$

ed in generale le regole sono guidate dalla sintassi. In generale le transizioni per un costrutto composto  $A \text{ op } B$  sono date in termini di quelli dei componenti  $A$  e  $B$ .



## 4. Espressioni aritmetiche non tipizzate

### 4.1 Introduzione

Per parlare rigorosamente riguardo ai sistemi di tipi e delle loro proprietà, dobbiamo iniziare affrontando formalmente alcuni aspetti più basilari dei linguaggi di programmazione.

In particolare, abbiamo bisogno di strumenti chiari, precisi e matematicamente trattabili per esprimere e ragionare sulla sintassi e la semantica dei programmi.

In questo capitolo svilupperemo gli strumenti necessari per generare un piccolo linguaggio di numeri e booleani. Questo linguaggio è così banale da essere quasi al di sotto della considerazione, ma funge da veicolo diretto per l'introduzione di diversi concetti fondamentali: **sintassi astratta**, **definizioni** e **dimostrazioni induttive**, valutazione e modellazione degli **errori di runtime**.

Il linguaggio usato in questo capitolo contiene solo una manciata di forme sintattiche: le costanti booleane `true` and `false`, espressioni condizionali, costanti numeriche, l'operatore aritmetico `succ(successore)` e `pred(predecessore)` e un operatore di test `iszero` che restituisce `true` se il suo argomento è zero, `false` altrimenti. Queste forme sintattiche possono essere riassunte in modo compatto dalla seguente grammatica

|                                   |                  |
|-----------------------------------|------------------|
| <code>t ::=</code>                | (termini)        |
| <code>true</code>                 | (costante true)  |
| <code>  false</code>              | (costante false) |
| <code>  if t then t else t</code> | (condizionale)   |
| <code>  0</code>                  | (costante zero)  |
| <code>  succ t</code>             | (successore)     |
| <code>  pred t</code>             | (predecessore)   |
| <code>  iszero t</code>           | (zero test)      |

Le convenzioni usate in questa grammatica (e in tutto il libro) sono simili a quelle del BNF standard. La prima riga (`t ::=`) dichiara che stiamo definendo l'insieme di termini e che utilizzeremo la lettera `t` per spaziare tra i termini. Ogni riga che segue fornisce una forma sintattica alternativa per i termini. In ogni punto in cui compare il simbolo `t`, possiamo sostituire qualsiasi termine. Le frasi in corsivo a destra sono solo commenti.

Il simbolo  $t$  nella parte destra delle regole di questa grammatica viene chiamato una **metavariabile**. È una variabile nel senso che è un segnaposto per qualche termine particolare, e "meta" nel senso che non è una variabile del linguaggio oggetto (il semplice linguaggio di programmazione di cui stiamo attualmente descrivendo la sintassi) ma piuttosto del metalinguaggio: la notazione in cui viene data la descrizione. Il prefisso "meta" deriva dalla meta-matematica, il sottocampo della logica il cui oggetto sono le proprietà matematiche dei sistemi per la matematica e ragionamento logico (che include linguaggi di programmazione). Questo campo ci dà anche il termine meta-teoria, che significa la raccolta di affermazioni vere che possiamo fare su un particolare sistema logico (o linguaggio di programmazione) e, per estensione, lo studio di tali affermazioni. Pertanto, frasi come "metateoria della sottotipizzazione" in questo libro possono essere intese come "lo studio formale delle proprietà dei sistemi con sottotipizzazione".

Per il momento, le parole termine ed espressione sono usate in modo intercambiabile. Quando inizieremo a discutere i calcoli con ulteriori categorie sintattiche come i tipi, useremo espressione per tutti i tipi di frasi sintattiche (incluse term expressions, type expressions, kind expressions, etc.), riservando il termine al senso più specializzato di frasi che rappresentano calcoli (cioè, frasi che possono essere sostituite alla metavariable  $t$ )

Un programma nella lingua attuale è solo un termine costruito dalle forme date dalla grammatica sopra.

■ **Esempio 4.1** Ecco alcuni esempi di programmi, insieme ai risultati della loro valutazione:

```
if true then 0 else (succ 0) → 0
if false then 0 else (succ 0) → succ 0
if (iszero 0) then 0 else (succ 0) → 0
if (iszero (succ 0)) then 0 else (succ 0) → succ 0
iszero (pred (succ 0)) → true
```

Durante la composizione, gli esempi vengono elaborati automaticamente dall'implementazione corrispondente al sistema formale in discussione (arith); le risposte visualizzate sono l'output effettivo dell'implementazione.

Negli esempi, gli argomenti composti per succ, pred e iszero sono racchiusi tra parentesi per la leggibilità. Le parentesi non sono menzionate nella grammatica dei termini, che ne definisce solo la sintassi astratta. Naturalmente, la presenza o l'assenza di parentesi fa poca differenza nell'estremamente semplice lingua con cui abbiamo a che fare in questo momento: le parentesi sono solitamente usate per risolvere le ambiguità nella grammatica, ma questa grammatica non ha alcuna ambiguità: ogni sequenza di token può essere parsata come un termine in un modo univoco.

Per brevità negli esempi, utilizziamo numeri arabi standard per i numeri, che sono rappresentati formalmente come applicazioni nidificate di succ su 0. Ad esempio, succ(succ(succ(0))) è scritto come 3.

I risultati della valutazione sono termini di una forma particolarmente semplice: saranno sempre o costanti booleane o numeri (applicazioni nidificate di zero o più istanze di succ su 0). Tali termini sono chiamati **valori** e svolgeranno un ruolo speciale nella nostra formalizzazione dell'ordine di valutazione dei termini.

Si noti che la sintassi dei termini consente la formazione di alcuni termini dall'aspetto dubbio come succ true e if 0 then 0 else 0. Avremo altro da dire su tali termini in seguito, anzi, in un certo senso sono proprio ciò che rende interessante questo minuscolo linguaggio per i nostri scopi, poiché sono esempi del tipo di programmi senza senso che vorremmo escludere da un sistema di tipi.

## 4.2 Sintassi

Esistono diversi modi equivalenti per definire la sintassi del nostro linguaggio. Ne abbiamo già visto uno nella grammatica a pagina 31. Questa grammatica è in realtà solo una notazione compatta per la seguente definizione induttiva:

**Definizione 4.2.1 — Termini, Induzione.** L'insieme dei *termini* è il più piccolo insieme  $\mathcal{T}$  tale che:

1.  $\{\text{true}, \text{false}, 0\} \subseteq \mathcal{T}$ ;
2. if  $t_1 \in \mathcal{T}$  then  $\{\text{succ } t_1, \text{pred } t_1, \text{iszero } t_1\} \in \mathcal{T}$ ;
3. if  $t_1 \in \mathcal{T}, t_2 \in \mathcal{T}$ , and  $t_3 \in \mathcal{T}$  then if  $t_1$  then  $t_2$  else  $t_3 \in \mathcal{T}$ .

Poiché le definizioni induttive sono onnipresenti nello studio dei linguaggi di programmazione, vale la pena soffermarsi un momento a esaminarla in dettaglio. La prima clausola ci dice tre semplici espressioni che sono in  $\mathcal{T}$ . La seconda e la terza clausola ci danno regole con cui possiamo giudicare che certe espressioni composte sono in  $\mathcal{T}$ . Infine, la parola "più piccolo" ci dice che  $\mathcal{T}$  non ha elementi oltre a quelli richiesti da queste tre clausole.

Come la grammatica a pagina 31, questa definizione non dice nulla sull'uso delle parentesi per contrassegnare i sottotermini composti. Formalmente, ciò che sta realmente accadendo è che stiamo definendo  $\mathcal{T}$  come un insieme di alberi, non come un insieme di stringhe. L'uso delle parentesi negli esempi è solo un modo per chiarire la relazione tra la forma linearizzata dei termini che scriviamo sulla pagina e la forma reale dell'albero sottostante.

Un'abbreviazione diversa per la stessa definizione induttiva dei termini utilizza il formato della regola di inferenza bidimensionale comunemente usato nelle presentazioni di sistemi logici in "stile di deduzione naturale":

**Definizione 4.2.2 — Termini, Regole di inferenza.** L'insieme dei termini è definito dalle seguenti regole di inferenza:

$$\begin{array}{ccc}
 \text{true} \in \mathcal{T} & \text{false} \in \mathcal{T} & 0 \in \mathcal{T} \\
 \\
 \frac{t_1 \in \mathcal{T}}{\text{succ } t_1 \in \mathcal{T}} & \frac{t_1 \in \mathcal{T}}{\text{pred } t_1 \in \mathcal{T}} & \frac{t_1 \in \mathcal{T}}{\text{iszero } t_1 \in \mathcal{T}} \\
 \\
 \frac{t_1 \in \mathcal{T} \quad t_2 \in \mathcal{T} \quad t_3 \in \mathcal{T}}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \in \mathcal{T}}
 \end{array}$$

Le prime tre regole sono assiomi, mentre le successive 4 sono regole di inferenza.

Ogni regola viene letta: "Se abbiamo stabilito le affermazioni nelle premesse elencate sopra la riga, allora possiamo trarre la conclusione sotto la riga". Il fatto che  $\mathcal{T}$  sia l'insieme più piccolo che soddisfa queste regole è spesso (come qui) non dichiarato esplicitamente.

Due punti della terminologia meritano di essere menzionati. In primo luogo, le regole senza premesse (come le prime tre sopra) sono spesso chiamate assiomi. In questo libro, il termine regola di inferenza è usato genericamente per includere sia gli assiomi che le "regole proprie" con una o più premesse. Gli assiomi sono generalmente scritti senza barra, poiché non c'è nulla da superare. In secondo luogo, per essere completamente pedanti, quelle che chiamiamo "regole di inferenza" sono in realtà schemi di regole, poiché le loro premesse e conclusioni possono includere metavariable. Formalmente, ogni schema rappresenta l'insieme infinito di regole concrete che possono essere ottenute sostituendo ogni metavariable in modo coerente con tutte le frasi della categoria sintattica appropriata, cioè, nelle regole sopra, sostituendo ogni  $t$  con ogni possibile termine.



### 4.3 Stili di semantica

Dopo aver formulato rigorosamente la sintassi del nostro linguaggio, abbiamo bisogno di una definizione altrettanto precisa di come vengono valutati i termini, cioè la semantica della lingua. Esistono tre approcci di base per formalizzare la semantica:

1. **Semantica operativa:** specifica il comportamento di un linguaggio di programmazione definendo per esso una semplice macchina astratta. Questa macchina è "astratta" nel senso che usa i termini del linguaggio come codice macchina, piuttosto che un set di istruzioni di microprocessore di basso livello. Per i linguaggi semplici, uno stato della macchina è solo un termine e il comportamento della macchina è definito da una funzione di transizione che, per ogni stato, fornisce lo stato successivo eseguendo un passaggio di semplificazione sul termine o dichiara che la macchina ha terminato. Il significato di un termine  $t$  può essere considerato lo stato finale che la macchina raggiunge quando viene avviata con  $t$  come stato iniziale. (A rigor di termini, ciò che stiamo descrivendo qui è il cosiddetto stile di **small-steps semantic**, a volte chiamato semantica operativa strutturale).

A volte è utile fornire due o più semantiche operative diverse per un singolo linguaggio: alcune più astratte, con stati macchina che sembrano simili ai termini scritti dal programmatore, altre più vicine alle strutture manipolate da un vero interprete o compilatore per il linguaggio. Dimostrare che i comportamenti di queste diverse macchine corrispondono in un certo senso appropriato quando si esegue lo stesso programma equivale a dimostrare la correttezza di un'implementazione del linguaggio.

2. **Semantica denotazionale:** assume una visione più astratta del significato: invece di solo una sequenza di stati della macchina, il significato di un termine è considerato un oggetto matematico, come un numero o una funzione. Dare la semantica denotazionale per un linguaggio consiste nel trovare una raccolta di domini semantici e quindi definire una funzione di interpretazione che mappa i termini negli elementi di questi domini. La ricerca di domini semantici appropriati per modellare le varie caratteristiche del linguaggio ha dato origine a un'area di ricerca ricca ed elegante nota come *teoria dei domini*.

Uno dei principali vantaggi della semantica denotazionale è che astrae dai dettagli grintosi della valutazione ed evidenzia i concetti essenziali del linguaggio. Inoltre, le proprietà della raccolta scelta di domini semantici possono essere utilizzate per derivare leggi potenti per ragionare sui comportamenti dei programmi, leggi per dimostrare che due programmi hanno esattamente lo stesso comportamento, ad esempio, o che il comportamento di un programma soddisfa alcune specifiche. Infine, dalle proprietà della raccolta di domini semantici pre-scelta, è spesso immediatamente evidente che varie cose (desiderabili o indesiderabili) sono impossibili in un linguaggio.

3. **Semantica assiomatica:** adotta un approccio più diretto a queste leggi: invece di definire prima i comportamenti dei programmi (dando alcune semantiche operative o denotazionali) e poi derivare leggi da questa definizione, i metodi assiomatici prendono le leggi stesse come definizione del linguaggio. Il significato di un termine è proprio ciò che può essere dimostrato al riguardo. La bellezza dei metodi assiomatici è che focalizzano l'attenzione sul processo di ragionamento sui programmi. È questa linea di pensiero che ha dato all'informatica idee così potenti come le invariants.

Durante gli anni '60 e '70, la semantica operativa era generalmente considerata inferiore agli altri due stili, utile per definizioni rapide e sporche delle caratteristiche del linguaggio, ma inelegante e matematicamente debole. Ma negli anni '80 i metodi più astratti cominciarono a incontrare problemi tecnici sempre più spinosi, e la semplicità e la flessibilità dei metodi operativi divennero al confronto sempre più attraenti, soprattutto alla luce dei nuovi sviluppi nell'area da parte di numerosi ricercatori, a cominciare da *Structural Operational Semantics* di Plotkin (1981), *Natural Semantics*



di Kahn (1987) e il lavoro di Milner su CCS (1980; 1989; 1999), che hanno introdotto formalismi più eleganti e hanno mostrato quante delle potenti tecniche matematiche sviluppate nel contesto della semantica denotazionale potrebbe essere trasferita in un contesto operativo. La semantica operativa è diventata un'area di ricerca energetica a sé stante ed è spesso il metodo preferito per definire i linguaggi di programmazione e studiarne le proprietà. È usato esclusivamente in questo libro.

## 4.4 Evaluation

Lasciando da parte i numeri per il momento, iniziamo con la semantica operativa delle sole espressioni booleane. La Figura 4.1 riassume la definizione. Esaminiamo ora le sue parti in dettaglio. La colonna di sinistra della Figura 4.1 è una grammatica che definisce due insiemi di espressioni. Il primo è solo una ripetizione (per comodità) della sintassi dei termini. Il secondo definisce un sottoinsieme di termini, detti valori, che sono possibili risultati finali della valutazione. Qui, i valori sono solo le costanti true e false. La metavariable  $v$  viene utilizzata in tutto il libro per indicare i valori.

| <i>Sintassi:</i>                        | <i>Evaluation:</i> <span style="border: 1px solid black; padding: 2px;"><math>t \rightarrow t'</math></span> |
|-----------------------------------------|--------------------------------------------------------------------------------------------------------------|
| $t ::=$ (termini)                       |                                                                                                              |
| true (costante true)                    | if true then $t_2$ else $t_3 \rightarrow t_2$ (E-IFTRUE)                                                     |
| false (costante false)                  |                                                                                                              |
| if $t$ then $t$ else $t$ (condizionale) | if false then $t_2$ else $t_3 \rightarrow t_3$ (E-IFFALSE)                                                   |
| $v ::=$ (valori)                        |                                                                                                              |
| true (valore true)                      | $\frac{t_1 \rightarrow t'_1}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3}$                            |
| false (valore false)                    | $\rightarrow \text{if } t'_1 \text{ then } t_2 \text{ else } t_3$ (E-IF)                                     |

Figura 4.1: Booleani definiti con la small-step semantic

La colonna di destra definisce una *relazione di valutazione* (Alcuni esperti preferiscono usare il termine riduzione per questa relazione, riservando "evaluation" alla variante "big-step" che mappa i termini direttamente ai loro valori finali.) sui termini, scritta  $t \rightarrow t'$  e pronunciata " $t$  restituisce  $t'$  in un passaggio". L'intuizione è che, se  $t$  è lo stato della macchina astratta in un dato momento, allora la macchina può fare un passo di calcolo e cambiare il suo stato in  $t'$ . Questa relazione è definita da tre regole di inferenza (o, se preferisci, da due assiomi e una regola, poiché le prime due non hanno premesse).

La prima regola, E-IFTRUE, dice che, se il termine da valutare è un condizionale la cui condizione è letteralmente la costante true, allora la macchina può eliminare l'espressione condizionale e lasciare la parte del then,  $t_2$ , come nuovo stato della macchina (vale a dire, il prossimo termine da valutare). Allo stesso modo, E-IFFALSE dice che un condizionale la cui condizione è letteralmente false valuta in un passaggio il suo ramo else,  $t_3$ . La E- nei nomi di queste regole ricorda che fanno parte della relazione di valutazione; le regole per le altre relazioni avranno prefissi diversi.

La terza regola di valutazione, E-IF, è più interessante. Dice che, se la condizione  $t_1$  valuta  $t'_1$ , allora il condizionale intero if  $t_1$  then  $t_2$  else  $t_3$  restituisce if  $t'_1$  then  $t_2$  else  $t_3$ . In termini di macchine astratte, una macchina nello stato if  $t_1$  then  $t_2$  else  $t_3$  può fare un passo per dichiarare

se  $\text{if } t'_1 \text{ then } t_2 \text{ else } t_3$  se un'altra macchina il cui stato è solo  $t_1$  può fare un passo per dichiarare  $t'_1$ .

Ciò che queste regole *non* dicono è importante quanto ciò che dicono. Le costanti *true* e *false* non valutano nulla, poiché non appaiono come il lato sinistro di nessuna delle regole. Inoltre, non esiste una regola che consenta di valutare una sottoespressione *then*- o *else*- di un *if* prima di valutare l'*if* stesso:

■ **Esempio 4.2** Ad esempio, il termine

$$\text{if true then (if false then false else false) else true} \quad (4.1)$$

non valuta *if true then false else true*. La nostra unica scelta è valutare prima il condizionale esterno, usando E-IF. Questa interazione tra le regole determina una particolare strategia di valutazione per i condizionali, corrispondente all'ordine di valutazione familiare nei linguaggi di programmazione comuni: per valutare un condizionale, dobbiamo prima valutarne la condizione; se la condizione è essa stessa un condizionale, dobbiamo prima valutare la sua condizione; e così via. Le regole E-IFTRUE ed E-IFFALSE ci dicono cosa fare quando arriviamo alla fine di questo processo e ci troviamo con un condizionale la cui condizione è già completamente valutata.

In un certo senso, E-IFTRUE e E-IFFALSE svolgono il vero lavoro di valutazione, mentre E-IF aiuta a determinare dove deve essere svolto il lavoro. Il diverso carattere delle regole è talvolta enfatizzato facendo riferimento a E-IFTRUE e E-IFFALSE come *regole di calcolo* e E-IF come *regola di congruenza*.

Per essere un po' più precisi su queste intuizioni, possiamo definire formalmente la relazione di valutazione come segue.

**Definizione 4.4.1 — Istanza di una regola di inferenza.** Un'*istanza* di una regola di inferenza si ottiene sostituendo costantemente ogni metavariable con lo stesso termine nella conclusione della regola e tutte le sue premesse (se presenti).

■ **Esempio 4.3** Per esempio

$$\text{if true then true else (if false then false else false)} \rightarrow \text{true}$$

è un'istanza di E-IFTRUE, dove entrambe le occorrenze di  $t_2$  sono state sostituite da *true* e  $t_3$  è stato sostituito da *if false then false else false*.

**Definizione 4.4.2 — Soddisfazione delle regole di inferenza.** Una regola è *soddisfatta* da una relazione se, **per ogni istanza della regola**, la conclusione è nella relazione oppure una delle premesse non lo è.

**Definizione 4.4.3 — One-step evaluation.** La relazione di valutazione *one-step*  $\rightarrow$  è la più piccola relazione binaria sui termini che soddisfano le tre regole della Figura 4.1. Quando la coppia  $(t, t')$  è nella relazione di valutazione, diciamo che "lo *statement* di valutazione (o giudizio)  $t \rightarrow t'$  è derivabile".

La forza della parola "più piccolo" qui è che un'affermazione  $t \rightarrow t'$  è derivabile se e solo se è verificata dalle regole: o è un'istanza di uno degli assiomi E-IFTRUE e E-IFFALSE, oppure è la conclusione di un'istanza della regola E-IF la cui premessa è derivabile. La derivabilità di una data affermazione può essere verificata esibendo un albero di derivazione le cui foglie sono etichettate con istanze di E-IFTRUE o E-IFFALSE e i cui nodi interni sono etichettati con istanze di E-IF.

■ **Esempio 4.4** Ad esempio, se si abbrevia

$$\begin{aligned} s &\stackrel{def}{=} \text{if true then false else false} \\ t &\stackrel{def}{=} \text{if } s \text{ then false else false} \\ u &\stackrel{def}{=} \text{if false then true else true} \end{aligned}$$

per evitare di uscire dal bordo della pagina, allora la derivabilità dell'affermazione

$$\text{if } t \text{ then false else false} \rightarrow \text{if } u \text{ then false else false} \quad (4.2)$$

è attestata dal seguente albero di derivazione:

$$\frac{\frac{\frac{}{s \rightarrow \text{false}} \quad [E\text{-IFTRUE}]}{t \rightarrow u} \quad [E\text{-IF}]}{\text{if } t \text{ then false else false} \rightarrow \text{if } u \text{ then false else false}} \quad [E\text{-IF}]$$

oppure con una riscrivendo le abbreviazioni:

$$\frac{\frac{\frac{}{\text{if true then false else false} \rightarrow \text{false}} \quad [E\text{-IFTRUE}]}{\text{if (if true then false else false) then true else true} \rightarrow \text{if false then true else true}} \quad [E\text{-IF}]}{\text{if (if (if true then false else false) then true else true) then false else false} \rightarrow \text{if (if false then true else true) then false else false}} \quad [E\text{-IF}]$$

Chiamare questa struttura un albero può sembrare un po' strano, poiché non contiene rami. In effetti, gli alberi di derivazione che testimoniano le dichiarazioni di valutazione avranno sempre questa forma snella: poiché nessuna regola di valutazione ha più di una premessa, non c'è modo di costruire un albero di derivazione ramificato. La terminologia avrà più senso quando consideriamo le derivazioni per altre relazioni definite induttivamente, come la tipizzazione, dove alcune regole hanno premesse multiple.

Il fatto che un'affermazione di valutazione  $t \rightarrow t'$  sia derivabile se esiste un albero di derivazione con  $t \rightarrow t'$  come etichetta alla sua radice è spesso utile quando si ragiona sulle proprietà della relazione di valutazione. In particolare, porta direttamente a una tecnica di dimostrazione chiamata induzione su derivazioni. La dimostrazione del seguente teorema illustra questa tecnica.

**Teorema 4.4.1 — Determinazione della valutazione One-step.** Se  $t \rightarrow t'$  e  $t \rightarrow t''$  allora  $t' = t''$ .

#### Dimostrazione 4.4.1

Questo è dimostrabile per induzione su una derivazione di  $t \rightarrow t'$ . Ad ogni passo dell'induzione, assumiamo il risultato desiderato per tutte le derivazioni minori e procediamo con un'analisi del caso della regola di valutazione utilizzata alla radice della derivazione. (Si noti che l'induzione qui non è sulla lunghezza di una sequenza di valutazione: stiamo osservando solo un singolo passaggio di valutazione. Potremmo anche dire che stiamo eseguendo l'induzione sulla struttura di  $t$ , poiché la struttura di una "derivazione della valutazione" segue direttamente la struttura del termine che viene ridotto. In alternativa, potremmo anche eseguire l'induzione sulla derivazione di  $t \rightarrow t''$ .)

Se l'ultima regola usata nella derivazione di  $t \rightarrow t'$  è E-IFTRUE, allora sappiamo che  $t$  ha la forma  $\text{if } t_1 \text{ then } t_2 \text{ else } t$ , dove  $t_1 = \text{true}$ . Ma ora è ovvio che l'ultima regola nella

derivazione di  $t \rightarrow t''$  non può essere E-IFFALSE, poiché non possiamo avere sia  $t_1 = \text{true}$  che  $t_1 = \text{false}$ . Inoltre, l'ultima regola nella seconda derivazione non può essere nemmeno E-IF, poiché la premessa di questa regola richiede che  $t_1 \rightarrow t'_1$  per qualche  $t'_1$ , ma abbiamo già osservato che quando  $\text{true}$  viene valutato non restituisce nulla. Quindi l'ultima regola nella seconda derivazione può essere solo E-IFTRUE, e ne segue immediatamente che  $t' = t''$ .

Allo stesso modo, se l'ultima regola usata nella derivazione di  $t \rightarrow t'$  è E-IFFALSE, allora l'ultima regola nella derivazione di  $t \rightarrow t''$  deve essere la stessa e il risultato è immediato.

Infine, se l'ultima regola usata nella derivazione di  $t \rightarrow t'$  è E-IF, allora la forma di questa regola ci dice che  $t$  ha la forma  $\text{if } t_1 \text{ then } t_2 \text{ else } t$ , dove  $t_1 \rightarrow t'_1$  per qualche  $t'_1$ . Per lo stesso ragionamento di cui sopra, l'ultima regola nella derivazione di  $t \rightarrow t'$  può essere solo E-IF, il che ci dice che  $t$  ha la forma  $\text{if } t_1 \text{ then } t_2 \text{ else } t_3$  (che già conosciamo) e che  $t_1 \rightarrow t'_1$  per qualche  $t'_1$ . Ma ora si applica l'ipotesi di induzione (poiché le derivazioni di  $t_1 \rightarrow t'_1$  e  $t_1 \rightarrow t''_1$  sono sottoderivazioni delle derivazioni originarie di  $t \rightarrow t'$  e  $t \rightarrow t''$ ), ottenendo  $t'_1 = t''_1$ . Questo ci dice che  $t' = \text{if } t'_1 \text{ then } t_2 \text{ else } t_3 = \text{if } t''_1 \text{ then } t_2 \text{ else } t_3 = t''$ , come richiesto.

La nostra relazione di valutazione in un passaggio mostra come una macchina astratta si sposta da uno stato all'altro durante la valutazione di un dato termine. Ma come programmatori siamo altrettanto interessati ai risultati finali della valutazione, cioè agli stati da cui la macchina non può fare un passo.

**Definizione 4.4.4 — Forma normale.** Un termine  $t$  è in *forma normale* se non vi si applica alcuna regola di valutazione. Ad esempio Se non ci sono  $t'$  tali che  $t \rightarrow t'$  (A volte diciamo " $t$  è una forma normale" come abbreviazione di " $t$  è un termine in forma normale").

Abbiamo già osservato che  $\text{true}$  e  $\text{false}$  sono forme normali nel sistema attuale (poiché tutte le regole di valutazione hanno il lato sinistro il cui costruttore più esterno è un  $\text{if}$ , ovviamente non c'è modo di istanziare nessuna delle regole in modo che il suo lato sinistro diventi  $\text{true}$  o  $\text{false}$ ). Possiamo riformulare questa osservazione in termini più generali come un fatto sui valori:

**Teorema 4.4.2** Ogni valore è in forma normale.

Quando arricchiamo il sistema con espressioni aritmetiche (e, nei capitoli successivi, altri costrutti), faremo sempre in modo che il Teorema 4.4.2 rimanga valido: essere in forma normale fa parte di ciò che significa essere un valore (cioè, un risultato completamente valutato) e qualsiasi definizione di linguaggio in cui non è così viene semplicemente infranta.

Nel sistema attuale vale anche il viceversa del Teorema 4.4.2: ogni forma normale è un valore. Questo non sarà il caso in generale; infatti, le forme normali che non sono valori giocano un ruolo critico nella nostra analisi degli errori di runtime, come vedremo quando arriveremo alle espressioni aritmetiche più avanti in questa sezione.

**Teorema 4.4.3** Se  $t$  è in forma normale, allora  $t$  è un valore.

#### Dimostrazione 4.4.2

Supponiamo che  $t$  non sia un valore. È facile mostrare, per induzione strutturale su  $t$ , che non è una forma normale.

Poiché  $t$  non è un valore, deve avere la forma  $\text{if } t_1 \text{ then } t_2 \text{ else } t_3$  per alcuni  $t_1, t_2$  e  $t_3$ .

Consideriamo le possibili forme di  $t_1$ . Se  $t_1 = \text{true}$ , allora chiaramente  $t$  non è una forma normale, poiché corrisponde al lato sinistro di E-IFTRUE. Allo stesso modo se  $t_1 = \text{false}$ .

Se  $t_1$  non è né true né false, allora non è un valore. Si applica quindi l'ipotesi di induzione, che ci dice che  $t_1$  non è una forma normale, cioè che esiste qualche  $t'_1$  tale che  $t_1 \rightarrow t'_1$ . Ma questo significa che possiamo usare E-IF per derivare  $t \rightarrow \text{if } t'_1 \text{ then } t_2 \text{ else } t$ , quindi neanche  $t$  è una forma normale.

A volte è conveniente poter vedere molti passaggi della valutazione come un'unica grande transizione di stato. Lo facciamo definendo una relazione di valutazione a più fasi che mette in relazione un termine con tutti i termini che possono essere derivati da esso mediante zero o più singoli passaggi di valutazione.

**Definizione 4.4.5 — Multi-step evaluation.** La relazione di valutazione multi-step  $\rightarrow^*$  è la chiusura riflessiva e transitiva della valutazione one-step. Essa è la più piccola relazione tale che

1. se  $t \rightarrow t'$  allora  $t \rightarrow^* t'$ ; (chiusura rispetto a  $\rightarrow$ )
2.  $t \rightarrow^* t$  per ogni  $t$  (chiusura riflessiva);
3. se  $t \rightarrow^* t'$  e  $t' \rightarrow^* t''$  allora  $t \rightarrow^* t''$  (chiusura transitiva).

In altre parole, la relazione di valutazione multi-step tiene conto di più passi di valutazione, invece di solo un passo alla volta. La definizione specifica indica che se un'espressione  $t$  può essere valutata in un'altra espressione  $t'$  in un solo passo ( $t \rightarrow t'$ ), allora  $t$  può essere valutata in  $t'$  in più passi ( $t \rightarrow^* t'$ ). Inoltre, se un'espressione  $t$  può essere valutata in un'altra espressione  $t'$  in più passi e  $t'$  può essere valutata in un'altra espressione  $t''$  in più passi, allora  $t$  può essere valutata in  $t''$  in più passi ( $t \rightarrow^* t'$  e  $t' \rightarrow^* t'' \Rightarrow t \rightarrow^* t''$ ).

Avere una notazione esplicita per la valutazione in più fasi rende facile affermare fatti come i seguenti:

**Teorema 4.4.4 — Unicità della forma normale.** Se  $t \rightarrow^* u$  e  $t \rightarrow^* u'$  dove  $u$  e  $u'$  sono entrambe forme normali, allora  $u = u'$

### Dimostrazione 4.4.3

Corollario della determinazione della valutazione One-step (4.4.1)

L'ultima proprietà di valutazione che consideriamo prima di rivolgere la nostra attenzione alle espressioni aritmetiche è il fatto che *ogni* termine può essere valutato fino a ottenere un valore. Chiaramente, questa è un'altra proprietà che non è necessaria in linguaggi più ricchi con caratteristiche come definizioni di funzioni ricorsive. Anche nelle situazioni in cui la proprietà regge, la sua dimostrazione è generalmente molto più sottile di quella che stiamo per vedere.

La maggior parte delle dimostrazioni di terminazione in informatica hanno la stessa forma di base: Per prima cosa, scegliamo un *insieme ben fondato*  $S$  e diamo una funzione  $f$  che mappa "stati macchina" (here, termini) in  $S$ . Successivamente, mostriamo che, ogni volta che uno stato macchina  $t$  può fare un passo verso un altro stato  $t'$ , abbiamo  $f(t') < f(t)$ . Osserviamo ora che una sequenza infinita di passi di valutazione a partire da  $t$  può essere mappata, tramite  $f$ , in una catena infinita decrescente di elementi di  $S$ . Poiché  $S$  è ben fondato, non può esistere tale catena infinita decrescente, e quindi non esisterà nessuna sequenza di valutazione infinita. La funzione  $f$  è spesso chiamata *misura di terminazione* (termination measure) per la relazione di valutazione. (detto in parole semplici è come se facessimo un ciclo for sulla funzione di evaluation small-step sino ad ottenere un valore, ossia il risultato).

**Teorema 4.4.5 — Fine della valutazione.** Per ogni termine  $t$  esiste una qualche forma normale  $t'$  tale che  $t \rightarrow^* t'$

#### Dimostrazione 4.4.4

Basta osservare che ogni passo di valutazione riduce la dimensione del termine e che la dimensione è una misura di terminazione perché il solito ordine sui numeri naturali è ben fondato

Il nostro prossimo compito è estendere la definizione di valutazione alle espressioni aritmetiche. La Figura 4.2 riassume le nuove parti della definizione. (La notazione nell'angolo in alto a destra di 4.2 ci ricorda di considerare questa figura come un'estensione di 4.1, non un linguaggio indipendente a sé stante.)

| <i>Sintassi:</i> |                     | <i>Evaluation:</i>                                                                | $t \rightarrow t'$ |
|------------------|---------------------|-----------------------------------------------------------------------------------|--------------------|
| $t ::= \dots$    | (termini)           |                                                                                   |                    |
| 0                | (costante zero)     | $\frac{t_1 \rightarrow t'_1}{\text{succ } t_1 \rightarrow \text{succ } t'_1}$     | (E-SUCC)           |
| succ $t$         | (successore)        | pred 0 $\rightarrow$ 0                                                            | (E-PREDZERO)       |
| pred $t$         | (predecessore)      | pred (succ $nv_1$ ) $\rightarrow nv_1$                                            | (E-PREDSUCC)       |
| iszero $t$       | (zero test)         | $\frac{t_1 \rightarrow t'_1}{\text{pred } t_1 \rightarrow \text{pred } t'_1}$     | (E-PRED)           |
| $v ::= \dots$    | (valori)            | iszero 0 $\rightarrow$ true                                                       | (E-ISZEROZERO)     |
| $nv$             | (valore numerico)   | iszero (succ $nv_1$ ) $\rightarrow$ false                                         | (E-ISZEROSUCC)     |
| $nv ::=$         | (valori numerici)   | $\frac{t_1 \rightarrow t'_1}{\text{iszero } t_1 \rightarrow \text{iszero } t'_1}$ | (E-ISZERO)         |
| 0                | (valore zero)       |                                                                                   |                    |
| succ $v$         | (valore successore) |                                                                                   |                    |

Figura 4.2: Espressioni aritmetiche definite con la small-step semantic

Di nuovo, la definizione dei termini è solo una ripetizione della sintassi che abbiamo visto nel 4.1. La definizione di valori è un po' più interessante, poiché richiede l'introduzione di una nuova categoria sintattica di *valori numerici*. L'intuizione è che il risultato finale della valutazione di un'espressione aritmetica può essere un numero, dove un numero è o 0 o il successore di un numero (ma non il successore di un valore arbitrario: vorremo dire che succ(true) è un errore, non un valore).

Le regole di valutazione nella colonna di destra della Figura 4.2 seguono lo stesso schema visto nella Figura 4.1. Esistono quattro *regole di calcolo* (E-PREDZERO, E-PREDSUCC, E-ISZEROZERO e E-ISZEROSUCC) che mostrano come si comportano gli operatori pred e iszero quando applicati ai numeri, e tre regole di congruenza (E-SUCC, E-PRED e E-ISZERO) che indirizzano la valutazione nel "primo" sottoterminale di un termine composto. A rigor di termini, dovremmo ora ripetere la Definizione 4.4.3 ("la relazione di valutazione a un passo sulle espressioni aritmetiche è la relazione più piccola che soddisfa tutte le istanze delle regole nelle figure 4.1 e 4.2..."). Per

evitare di sprecare spazio su questo tipo di testo standard, è pratica comune considerare le regole di inferenza come costituenti la definizione della relazione da sole, lasciando "la relazione più piccola contenente tutte le istanze..." come inteso.

La categoria sintattica dei *valori numerici* ( $nv$ ) gioca un ruolo importante in queste regole. In E-PREDSUCC, per esempio, il fatto che il lato sinistro sia  $\text{pred}(\text{succ } nv_1)$  (piuttosto che  $\text{pred}(\text{succ } t_1)$ , per esempio) significa che questa regola non può essere usata per valutare  $\text{pred}(\text{succ}(\text{pred } 0))$  a  $\text{pred } 0$ , poiché ciò richiederebbe l'istanziamento della metavariable  $nv_1$  con  $\text{pred } 0$ , che non è un valore numerico. Invece, l'unico passo successivo nella valutazione del termine  $\text{pred}(\text{succ}(\text{pred } 0))$  ha il seguente albero di derivazione:

$$\frac{\frac{\frac{}{\text{pred } 0 \rightarrow 0} [\text{E-PREDZERO}]}{\text{succ}(\text{pred } 0) \rightarrow \text{succ } 0} [\text{E-SUCC}]}{\text{pred}(\text{succ}(\text{pred } 0)) \rightarrow \text{pred}(\text{succ } 0)} [\text{E-PRED}]$$

Formalizzare la semantica operativa di un linguaggio ci costringe a specificare il comportamento di tutti i termini, inclusi, nel caso in questione, termini come  $\text{pred } 0$  e  $\text{succ false}$ . Secondo le regole della Figura 4.2, il predecessore di 0 è definito come 0. Il successore di false, d'altra parte, non è definito per valutare nulla (cioè è una forma normale). Chiamiamo tali termini bloccati.

**Definizione 4.4.6 — Termine bloccato.** Un termine chiuso  $t$  è bloccato se non esiste alcun termine  $t'$  tale che  $t \rightarrow^* t'$ . Ossia, un termine chiuso è bloccato se è in forma normale ma non è un valore. Un termine è bloccato se non può essere valutato. Un termine bloccato è detto *stallo* (*stuck*).

Gli stalli ci danno una semplice nozione di *errore a runtime* per la nostra semplice macchina. Intuitivamente, caratterizza le situazioni in cui la semantica operativa non sa cosa fare perché il programma ha raggiunto uno "stato senza significato". In un'implementazione più concreta del linguaggio, questi stati potrebbero corrispondere a guasti della macchina di vario tipo: errori di segmentazione, esecuzione di istruzioni illegali, ecc. Qui, riduciamo tutti questi tipi di cattivo comportamento nel singolo concetto di "stato bloccato".

#### 4.4.1 Big-step semantic

Esistono due stili di semantica operativa di uso comune. Quello usato in questo libro è chiamato lo stile small-step, perché la definizione della relazione di valutazione mostra come i singoli passi del calcolo vengono usati per riscrivere un termine, poco a poco, fino a quando non diventa un valore. Oltre a ciò, definiamo una relazione di valutazione a più passaggi che ci consente di parlare di termini che valutano (in molti passaggi) i valori. Uno stile alternativo, chiamato big-step semantic (o talvolta semantica naturale), formula direttamente la nozione di "valuta questo termine fino al valore finale", scritto  $t \Downarrow v$ . Le regole di valutazione big-step per il nostro linguaggio delle espressioni booleane e aritmetiche sono così:

$$v \Downarrow v \quad (\text{B-VALUE})$$

$$\frac{t_1 \Downarrow \text{true} \quad t_2 \Downarrow v_2}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \Downarrow v_2} \quad (\text{B-IFTRUE})$$

$$\frac{t_1 \Downarrow \text{false} \quad t_3 \Downarrow v_3}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \Downarrow v_3} \quad (\text{B-IFFALSE})$$

$$\frac{t_1 \Downarrow nv_1}{\text{succ } t_1 \Downarrow \text{succ } nv_1} \quad (\text{B-SUCC})$$



$$\frac{t_1 \Downarrow 0}{\text{pred } t_1 \Downarrow 0} \quad (\text{B-PREDZERO})$$

$$\frac{t_1 \Downarrow \text{succ } nv_1}{\text{pred } t_1 \Downarrow nv_1} \quad (\text{B-PREDSUCC})$$

$$\frac{t_1 \Downarrow 0}{\text{iszero } t_1 \Downarrow \text{true}} \quad (\text{B-ISZEROZERO})$$

$$\frac{t_1 \Downarrow \text{succ } nv_1}{\text{iszero } t_1 \Downarrow \text{false}} \quad (\text{B-ISZEROSUCC})$$



## 5. Lambda Calcolo non tipizzato

### 5.1 Introduzione

Questo capitolo rivede la definizione e alcune proprietà di base del lambda-calcolo non tipizzato o puro, il substrato “computazionale” per la maggior parte dei sistemi di tipi descritti nel resto del libro.

Nella metà degli anni sessanta, Peter Landin osservò che una linguaggio di programmazione complesso può essere compreso formulandolo come un piccolo calcolo principale che cattura i meccanismi essenziali del linguaggio, insieme ad una collezione di *forme derivate* convenienti il cui comportamento è compreso traducendole nel principale (Landin 1964, 1965, 1966; vedi anche Tennent 1981). Il linguaggio principale usato da Landin era il *lambda-calcolo*, un sistema formale inventato negli anni '20 da Alonzo Church (1936, 1941), in cui tutte le computazioni sono ridotte alle operazioni di base di definizione e applicazione di funzioni. Seguendo l'ispirazione di Landin, così come il lavoro pionieristico di *John McCarthy* su Lisp (1959, 1981), il lambda-calcolo ha visto un ampio utilizzo nella specifica delle caratteristiche dei linguaggi di programmazione, nel design e nell'implementazione di linguaggi, e nello studio dei sistemi di tipi. La sua importanza deriva dal fatto che può essere visto contemporaneamente come un linguaggio di programmazione semplice in cui le computazioni possono essere descritte e come un oggetto matematico su cui possono essere fatte affermazioni rigorose.

Il lambda-calcolo è solo uno dei molti calcoli principali che sono stati usati per scopi simili. Il *pi-calcolo* di Milner, Parrow, e Walker (1992, 1991) è diventato un linguaggio principale popolare per definire la semantica dei linguaggi di programmazione basati su messaggi e concorrenti, mentre *Abadi e Cardelli's object calculus* (1996) distilla le caratteristiche essenziali dei linguaggi orientati agli oggetti. La maggior parte dei concetti e delle tecniche che svilupperemo per il lambda-calcolo possono essere trasferite direttamente a questi altri calcoli.

Il lambda-calcolo può essere arricchito in molti modi. In primo luogo, è spesso conveniente aggiungere una sintassi concreta speciale per le caratteristiche come i numeri, le tuple, i record, ecc., il cui comportamento può già essere simulato nel linguaggio principale. Più interessante, possiamo aggiungere caratteristiche più complesse come celle di riferimento mutabili o gestione eccezioni non locali, che possono essere modellati nel linguaggio principale solo con traduzioni piuttosto pesanti. Queste estensioni portano alla fine a linguaggi come ML (Gordon, Milner, e Wadsworth, 1979; Milner, Tofte, e Harper, 1990; Weis, Aponte, Laville, Mauny, e Suárez, 1989;

Milner, Tofte, Harper, e MacQueen, 1997), Haskell (Hudak et al., 1992), o Scheme (Sussman e Steele, 1975; Kelsey, Clinger, e Rees, 1998). Come vedremo nei capitoli successivi, le estensioni al linguaggio principale spesso coinvolgono estensioni anche al sistema di tipi.

## 5.2 Basi sul lambda-calcolo

L'astrazione procedurale (o funzionale) è una *caratteristica chiave* di essenzialmente tutti i linguaggi di programmazione. Invece di scrivere lo stesso calcolo più e più volte, scriviamo una procedura o una funzione che esegue il calcolo in modo generico, in termini di uno o più parametri nominati, e quindi istanziamo questa funzione quando necessario, fornendo valori per i parametri in ogni caso. Per esempio, è naturale per un programmatore prendere una lunga e ripetitiva espressione come

$$(5*4*3*2*1) + (7*6*5*4*3*2*1) - (3*2*1)$$

e riscriverla come `factorial(5) + factorial(7) - factorial(3)` dove:

$$\text{factorial}(n) = \text{if } n = 0 \text{ then } 1 \text{ else } n * \text{factorial}(n-1)$$

Per ogni numero non negativo  $n$ , istanziando la funzione fattoriale con l'argomento  $n$  si ottiene come risultato il fattoriale di  $n$ . Se scriviamo " $\lambda n. \dots$ " come scorciatoia per "la funzione che, per ogni  $n$ , produce  $\dots$ ", possiamo riformulare la definizione di fattoriale come:

$$\text{factorial} = \lambda n. \text{if } n = 0 \text{ then } 1 \text{ else } n * \text{factorial}(n-1)$$

Allora `factorial(0)` significa "la funzione ( $\lambda n. \text{if } n = 0 \text{ then } 1 \text{ else } \dots$ ) applicata all'argomento 0", cioè "il valore che risulta quando la variabile argomento  $n$  nel corpo della funzione ( $\lambda n. \text{if } n = 0 \text{ then } 1 \text{ else } \dots$ ) è sostituito da 0", cioè "if 0 = 0 then 1 else  $\dots$ ", cioè 1.

Il lambda-calcolo (o  $\lambda$ -calcolo) incarna questo tipo di definizione e applicazione di funzioni nella forma più pura possibile. Nel lambda-calcolo *tutto* è una funzione: gli argomenti accettati dalle funzioni sono essi stessi funzioni e il risultato restituito da una funzione è un'altra funzione.

La sintassi del lambda-calcolo comprende solo tre tipi di termini.<sup>1</sup> Una variabile  $x$  di per sé è un termine; l'astrazione di una variabile  $x$  da un termine  $t_1$ , scritto  $\lambda x. t_1$ , è un termine; e l'applicazione di un termine  $t_1$  ad un altro termine  $t_2$ , scritto  $t_1 t_2$ , è un termine. Questi modi di formare i termini sono riassunti nella seguente grammatica.

|                |                     |
|----------------|---------------------|
| $t ::= \dots$  | (termini)           |
| $x$            | (variabile)         |
| $\lambda x. t$ | (lambda-astrazione) |
| $t_1 t_2$      | (applicazione)      |

Le sottosezioni che seguono esplorano alcuni punti delicati di questa definizione.

<sup>1</sup>La locuzione lambda-termine è usata per riferirsi a termini arbitrari nel lambda-calcolo. I termini lambda che iniziano con  $\lambda$  sono spesso chiamati lambda-astrazioni.

### 5.2.1 Sintassi concreta e astratta

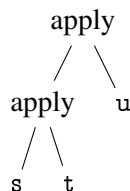
Quando si discute la sintassi dei linguaggi di programmazione, è utile distinguere due livelli <sup>2</sup> di struttura. La *sintassi concreta* (o *sintassi di superficie*) del linguaggio si riferisce alle stringhe di caratteri che i programmatori leggono e scrivono direttamente. La *sintassi astratta* è una rappresentazione interna molto più semplice dei programmi come alberi etichettati (chiamati *alberi di sintassi astratta* o *AST*). La rappresentazione ad albero rende immediatamente ovvia la struttura dei termini, rendendola un adattamento naturale per le complesse manipolazioni coinvolte sia nelle definizioni del linguaggio rigorose (e nelle dimostrazioni su di esse) sia all'interno di compilatori e interpreti.

La trasformazione dalla sintassi concreta a quella astratta avviene in due fasi. Innanzitutto, un analizzatore lessicale (o *lexer*) converte la stringa di caratteri scritta dal programmatore in una sequenza di *token*: identificatori, parole chiave, costanti, punteggiatura, etc. Il lexer rimuove i commenti e si occupa di questioni come gli spazi bianchi e le convenzioni sulle maiuscole e i formati per le costanti numeriche e di stringa. Successivamente, un *parser* trasforma questa sequenza di token in un albero di sintassi astratto. Durante l'analisi (parsing), varie convenzioni come la *precedenza* degli operatori e l'*associatività* riducono la necessità di ingombrare i programmi end-user con parentesi per indicare esplicitamente la struttura delle espressioni composte. Ad esempio, \* si lega più strettamente di +, quindi il parser interpreta l'espressione senza parentesi  $1+2*3$  come l'albero della sintassi astratta a sinistra in basso anziché quello a destra:



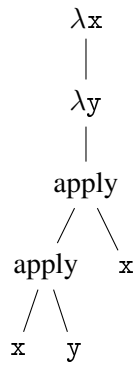
Il centro dell'attenzione in questo libro è sulla sintassi astratta, non concreta. Grammatiche come quella per i lambda-termini sopra dovrebbero essere intese come descrizioni di strutture ad albero lecite, non stringhe di token o caratteri. Naturalmente, quando scriviamo termini in esempi, definizioni, teoremi e dimostrazioni, avremo bisogno di esprimerli in una notazione concreta e lineare, ma abbiamo sempre in mente i loro alberi di sintassi astratti sottostanti.

Per evitare di scrivere troppe parentesi, adottiamo due convenzioni quando scriviamo lambda-termini in forma lineare. Innanzitutto, l'applicazione si associa a sinistra, ovvero  $s\ t\ u$  sta per lo stesso albero di  $(s\ t)\ u$ :



In secondo luogo, si assume che i corpi delle astrazioni si estendano il più a destra possibile, così che, per esempio,  $\lambda x. y. x\ y\ x$  sta per lo stesso albero di  $\lambda x. (\lambda y. ((x\ y)\ x))$ :

<sup>2</sup>Le definizioni di linguaggi completi a volte usano anche più livelli. Ad esempio, seguendo Landin, è spesso utile definire i comportamenti di alcuni costrutti del linguaggio come forme derivate, traducendoli in combinazioni di altre caratteristiche più basilari. Il sottolinguaggio ristretto che contiene solo queste caratteristiche fondamentali viene quindi chiamata *linguaggio interno* (o *IL*), mentre il linguaggio completo che include tutte le forme derivate è chiamato *linguaggio esterno* (*EL*). La trasformazione da EL a IL è (almeno concettualmente) eseguita in un passaggio separato, dopo il parsing.



### 5.2.2 Variabili e Metavariabili

Un'altra sottigliezza nella definizione della sintassi di cui sopra riguarda l'uso di metavariabili. Continueremo a utilizzare la metavariable  $t$  (così come  $s$ , e  $u$ , con o senza pedici) per indicare un termine arbitrario.<sup>3</sup> Allo stesso modo,  $x$  (così come  $y$  e  $z$ ) sta per una variabile arbitraria. Nota, qua, che  $x$  è una metavariable che il cui dominio va oltre quello delle variabili! A peggiorare le cose, l'insieme di nomi brevi è limitato e vorremmo anche usare  $x$ ,  $y$ , ecc. come variabili del linguaggio oggetto. In tali casi, tuttavia, il contesto chiarirà sempre cosa sia l'oggetto. Ad esempio, in una frase come "Il termine  $\lambda x. y. x y$  ha la forma  $\lambda z. s$ , dove  $z = x$  e  $s = \lambda y. x y$ ", i nomi  $z$  e  $s$  sono metavariabili, mentre  $x$  e  $y$  sono variabili del linguaggio oggetto.

### 5.2.3 Scope (o ambiente)

Un ultimo punto che dobbiamo affrontare riguardo alla sintassi del lambda-calcolo è l'*ambiente* (*scope*) delle variabili. Un'occorrenza della variabile  $x$  si dice *vincolata* (bound) quando è presente nel corpo  $t$  di un'astrazione  $\lambda x. t$ . (Più precisamente, è vincolato da questa astrazione. Equivalentemente, possiamo dire che  $\lambda x$  è un *legante* (binder) il cui ambiente è  $t$ .)

Un'occorrenza di  $x$  è *libera* se appare in una posizione in cui non è vincolata da un'astrazione che la racchiude su  $x$ . Ad esempio, le occorrenze di  $x$  in  $x y$  e  $\lambda y. x y$  sono libere, mentre quelle in  $\lambda x. x$  e  $\lambda z. \lambda x. \lambda y. x (y z)$  sono vincolate. In  $(\lambda x. x) x$ , la prima occorrenza di  $x$  è vincolata e la seconda è libera.

Un termine senza variabili libere si dice *chiuso*; i termini chiusi sono anche chiamati *combinatori*. Il combinatore più semplice, chiamato *funzione identità*

$$\text{id} = \lambda x. x$$

non fa nulla ma restituisce il suo argomento.

### 5.2.4 Semantica operativa

Nella sua forma pura, il lambda-calcolo non ha costanti incorporate o operatori primitivi: niente numeri, operazioni aritmetiche, condizionali, record, loop, sequenziamento, I/O, ecc. L'unico mezzo con cui i termini "calcolano" è l'applicazione di funzioni agli argomenti (che sono essi stessi funzioni). Ogni passaggio del calcolo consiste nel riscrivere un'applicazione il cui componente di sinistra è un'astrazione, sostituendo il componente di destra con la variabile associata nel corpo dell'astrazione. Graficamente, scriviamo

<sup>3</sup>Naturalmente, in questo capitolo,  $t$  spazia su lambda-termini, non su espressioni aritmetiche. In tutto il libro,  $t$  spazierà sempre i termini del calcolo in discussione al momento.

$$(\lambda x. t_{12}) t_2 \rightarrow [x \mapsto t_2] t_{12}$$

dove  $[x \mapsto t_2]t_{12}$  significa "il termine ottenuto sostituendo tutte le occorrenze libere di  $x$  in  $t_{12}$  con  $t_2$ ".

Ad esempio, il termine  $(\lambda x. x)$   $y$  restituisce  $y$  e il termine  $(\lambda x. x (\lambda x. x)) (u r)$  restituisce  $u r$   $(\lambda x. x)$ . Seguendo Church, un termine della forma  $(\lambda x. t_{12}) t_2$  è chiamato **redex** ("espressione riducibile"), e l'operazione di riscrittura di un redex secondo la regola di cui sopra è chiamata **beta-riduzione**.

Parecchie strategie di valutazione diverse per il lambda-calcolo sono state studiate nel corso degli anni da progettisti e teorici del linguaggio di programmazione. Ogni strategia definisce quale o quali redex in un termine possono attivarsi nella fase successiva della valutazione.<sup>4</sup>

- Con la **beta-riduzione piena**, qualsiasi redex può essere ridotto in qualsiasi momento. Ad ogni passaggio scegliamo un redex, ovunque all'interno del termine che stiamo valutando, e lo riduciamo.

■ **Esempio 5.1** Si consideri ad esempio il termine

$$(\lambda x. x) ((\lambda x. x) (\lambda z. (\lambda x. x) z))$$

che possiamo scrivere in modo più leggibile come `id (id (λz. id z))`. Questo termine contiene tre redex:

1. `id (id (λz. id z))`
2. `id ((id (λz. id z)))`
3. `id (id (λz. id z))`

Con la **beta-riduzione piena**, potremmo scegliere, ad esempio, di iniziare con il redex più interno, poi fare quello nel mezzo, e poi il più esterno:

$$\begin{aligned} & \text{id (id (λz. id z))} \\ \rightarrow & \text{id (id (λz.z))} \\ \rightarrow & \text{id (λz.z)} \\ \rightarrow & \lambda z. z \\ \rightarrow & \end{aligned}$$

- Nella strategia dell'**ordine normale**, il redex più a sinistra e più esterno viene sempre ridotto per primo.

■ **Esempio 5.2** Secondo questa strategia, il termine dell'esempio precedente sarebbe ridotto come segue:

$$\begin{aligned} & \text{id (id (λz. id z))} \\ \rightarrow & \text{id (λz. id z)} \\ \rightarrow & \lambda z. \text{id z} \\ \rightarrow & \lambda z. z \\ \rightarrow & \end{aligned}$$

Utilizzando questa strategia (e quelle sottostanti), la relazione di valutazione è in realtà una funzione parziale: ogni termine  $t$  valuta in un passo al massimo un termine  $t'$ .

<sup>4</sup>Alcune persone usano i termini "riduzione" e "valutazione" come sinonimi. Altri usano "valutazione" solo per strategie che implicano qualche nozione di "valore" e "riduzione" altrimenti.



- La strategia della **chiamata per nome** (*call by name*) è ancora più restrittiva, non consentendo riduzioni all'interno delle astrazioni.

■ **Esempio 5.3** Partendo dallo stesso termine di prima, eseguiremmo le prime due riduzioni come in ordine normale, ma poi ci fermeremmo prima dell'ultima e considereremmo  $\lambda z. \text{id } z$  come forma normale:

$$\begin{aligned} & \text{id } (\text{id } (\lambda z. \text{id } z)) \\ \rightarrow & \text{id } (\lambda z. \text{id } z) \\ \rightarrow & \lambda z. \text{id } z \\ \nrightarrow & \end{aligned}$$

Varianti di chiamata per nome sono state utilizzate in alcuni noti linguaggi di programmazione, in particolare Algol-60 (Naur, 1963) e Haskell (Hudak, 1992). Haskell utilizza effettivamente una versione ottimizzata nota come *chiamata per necessità* (*call by need*) (Wadsworth, 1971; Ariola, 1995) che, invece di rivalutare un argomento ogni volta che viene utilizzato, sovrascrive tutte le occorrenze dell'argomento con il suo valore la prima volta viene valutato, evitando la necessità di una successiva rivalutazione. Questa strategia richiede di mantenere una certa condivisione nella rappresentazione dei termini in fase di esecuzione: in effetti, si tratta di una relazione di riduzione su *grafi* di sintassi astratta, piuttosto che su alberi di sintassi.

- La maggior parte dei linguaggi utilizza una strategia di **chiamata per valore** (*call by value*), in cui vengono ridotti solo i redex più esterni *e* in cui un redex viene ridotto solo quando il suo lato destro è già stato ridotto a un valore, un termine che ha terminato il calcolo e non può essere ulteriormente ridotto.<sup>5</sup>

■ **Esempio 5.4** Utilizzando questa strategia, il nostro termine di esempio si riduce come segue:

$$\begin{aligned} & \text{id } (\text{id } (\lambda z. \text{id } z)) \\ \rightarrow & \text{id } (\lambda z. \text{id } z) \\ \rightarrow & \lambda z. \text{id } z \\ \nrightarrow & \end{aligned}$$

La strategia *call-by-value* è **rigorosa**, nel senso che gli argomenti delle funzioni vengono *sempre* valutati, indipendentemente dal fatto che siano utilizzati o meno dal corpo della funzione. Al contrario, strategie **non rigorose** (o *lazy*) come *call-by-name* e *call-by-need* valutano solo gli argomenti effettivamente utilizzati.

La scelta della strategia di valutazione in realtà fa poca differenza quando si discute di sistemi di tipo. I problemi che motivano le varie feature della tipizzazione e le tecniche utilizzate per affrontarli sono più o meno gli stessi per tutte le strategie. In questo libro usiamo la chiamata per valore, sia perché si trova nei linguaggi più conosciuti, sia perché è la più facile da arricchire con feature come eccezioni e riferimenti.

<sup>5</sup>Nell'attuale calcolo semplificato, gli unici valori sono le lambda-astrazioni. I calcoli più ricchi includeranno altri valori: costanti numeriche e booleane, stringhe, tuple di valori, record di valori, liste di valori, ecc.



## 5.3 Programmazione con il lambda-calcolo

Il lambda-calcolo è molto più potente di quanto potrebbe suggerire la sua minuscola definizione. In questa sezione, sviluppiamo una serie di esempi standard di programmazione nel lambda-calcolo. Questi esempi non hanno lo scopo di suggerire che il lambda-calcolo debba essere preso come un vero e proprio linguaggio di programmazione a sé stante (tutti i linguaggi di alto livello ampiamente usati forniscono modi più chiari ed efficienti per svolgere gli stessi compiti) ma piuttosto sono intesi come esercizi di riscaldamento per prendere confidenza con il sistema.

### 5.3.1 Argomenti multipli

Per iniziare, osserva che il lambda-calcolo non fornisce alcun supporto integrato per le funzioni multi-argomento. Naturalmente, questo non sarebbe difficile da aggiungere, ma è ancora più facile ottenere lo stesso effetto utilizzando *funzioni di ordine superiore* che producono funzioni come risultati. Supponiamo che  $s$  sia un termine che coinvolge due variabili libere  $x$  e  $y$  e che vogliamo scrivere una funzione  $f$  che, per ogni coppia  $(v, w)$  di argomenti, fornisca il risultato della sostituzione di  $v$  con  $x$  e  $w$  con  $y$  in  $s$ . Invece di scrivere  $f = \lambda(x, y).s$ , come potremmo fare in un linguaggio di programmazione più ricco, scriviamo  $f = \lambda x. \lambda y. s$ . Cioè,  $f$  è una funzione che, dato un valore  $v$  per  $x$ , produce una funzione che, dato un valore  $w$  per  $y$ , produce il risultato desiderato. Applichiamo quindi  $f$  ai suoi argomenti uno alla volta, scrivendo  $f\ v\ w$  (es:  $(f\ v)\ w$ ), che si riduce a  $((\lambda y. [x \mapsto v]s)\ w)$  e quindi a  $[y \mapsto w][x \mapsto v]s$ . Questa trasformazione di funzioni multi-argomento in funzioni di ordine superiore è chiamata **currying** in onore di Haskell Curry, un contemporaneo di Church.

### 5.3.2 Booleani di Church

Un'altra caratteristica del linguaggio che può essere facilmente codificata nel lambda-calcolo sono i valori booleani e i condizionali. Definire i termini `tru` e `fls`<sup>6</sup> come segue:

```
tru = λt.λf.t
fls = λt.λf.f
```

I termini `tru` e `fls` possono essere visti come rappresentanti dei valori booleani "vero" e "falso", nel senso che possiamo usare questi termini per eseguire l'operazione di verifica della verità di un valore booleano. In particolare, possiamo usare l'applicazione per definire un combinatore `test` con la proprietà che il `test b v w` si riduce a  $v$  quando  $b$  è `tru` e si riduce a  $w$  quando  $b$  è `fls`.

```
test = λl.λm.λn. l m n
```

Il combinatore `test` in realtà non fa molto: `test b v w` si riduce semplicemente a  $b\ v\ w$ . In effetti, il booleano  $b$  stesso è il condizionale: prende due argomenti e sceglie il primo (se è `tru`) o il secondo (se è `fls`).

<sup>6</sup>La notazione abbreviata di questi nomi ha lo scopo di evitare confusione con le costanti booleane primitive `true` e `false` del Capitolo 4.

■ **Esempio 5.5** Ad esempio, il termine  $\text{test } \text{tru } v \ w$  si riduce come segue:

|                                                                      |                                     |
|----------------------------------------------------------------------|-------------------------------------|
| $\text{test } \text{tru } v \ w$                                     |                                     |
| $= (\lambda l. \lambda m. \lambda n. l \ m \ n) \ \text{tru } v \ w$ | (per definizione di $\text{test}$ ) |
| $\rightarrow (\lambda m. \lambda n. \text{tru } m \ n) \ v \ w$      | (riduzione redex sottolineato)      |
| $\rightarrow (\lambda n. \text{tru } v \ n) \ w$                     | (riduzione redex sottolineato)      |
| $\rightarrow \text{tru } v \ w$                                      | (riduzione redex sottolineato)      |
| $= (\lambda t. \lambda f. t) \ v \ w$                                | (per definizione di $\text{tru}$ )  |
| $\rightarrow (\lambda f. v) \ w$                                     | (riduzione redex sottolineato)      |
| $\rightarrow v$                                                      | (riduzione redex sottolineato)      |

Possiamo anche definire operatori booleani come la congiunzione logica come funzioni:

$\text{and} = \lambda b. \lambda c. b \ c \ \text{fls}$

Cioè,  $\text{and}$  è una funzione che, dati due valori booleani  $b$  e  $c$ , restituisce  $c$  se  $b$  è  $\text{tru}$  e  $\text{fls}$  se  $b$  è  $\text{fls}$ ; quindi  $\text{and } b \ c$  restituisce  $\text{tru}$  se sia  $b$  che  $c$  sono  $\text{tru}$  e  $\text{fls}$  se  $b$  o  $c$  è  $\text{fls}$ .

■ **Esempio 5.6** Calcola  $\text{and } \text{tru } \text{tru}$ :

|                                                                         |                                    |
|-------------------------------------------------------------------------|------------------------------------|
| $\text{and } \text{tru } \text{tru}$                                    |                                    |
| $= (\lambda b. \lambda c. b \ c \ \text{fls}) \ \text{tru } \text{tru}$ | (Per definizione di $\text{and}$ ) |
| $\rightarrow (\lambda c. \text{tru } c \ \text{fls}) \ \text{tru}$      | (riduzione redex sottolineato)     |
| $\rightarrow \text{tru } \text{tru } \text{fls}$                        | (riduzione redex sottolineato)     |
| $= (\lambda t. \lambda f. t) \ \text{tru } \text{fls}$                  | (per definizione di $\text{tru}$ ) |
| $\rightarrow (\lambda f. \text{tru}) \ \text{fls}$                      | (riduzione redex sottolineato)     |
| $\rightarrow \text{tru}$                                                | (riduzione redex sottolineato)     |

**Esercizio 5.1** Calcola  $\text{and } \text{tru } \text{fls}$

**Soluzione:**

|                                                                         |                                    |
|-------------------------------------------------------------------------|------------------------------------|
| $\text{and } \text{tru } \text{fls}$                                    |                                    |
| $= (\lambda b. \lambda c. b \ c \ \text{fls}) \ \text{tru } \text{fls}$ | (Per definizione di $\text{and}$ ) |
| $\rightarrow (\lambda c. \text{tru } c \ \text{fls}) \ \text{fls}$      | (riduzione redex sottolineato)     |
| $\rightarrow \text{tru } \text{fls } \text{fls}$                        | (riduzione redex sottolineato)     |
| $= (\lambda t. \lambda f. t) \ \text{fls } \text{fls}$                  | (per definizione di $\text{tru}$ ) |
| $\rightarrow (\lambda f. \text{fls}) \ \text{fls}$                      | (riduzione redex sottolineato)     |
| $\rightarrow \text{fls}$                                                | (riduzione redex sottolineato)     |

L'operatore di  $\text{or}$  e  $\text{not}$  sono definiti come segue:

$\text{or} = \lambda b. \lambda c. b \ \text{tru } c$

$\text{not} = \lambda b. b \ \text{fls } \text{tru}$

### 5.3.3 Coppie(Pairs)

Usando i booleani, possiamo codificare coppie di valori come termini.

```
pair = λf.λs.λb. b f s
fst  = λp. p tru
snd  = λp. p fls
```

Cioè,  $\text{pair } v \ w$  è una funzione che, quando applicata a un valore booleano  $b$ , applica  $b$  a  $v$  e  $w$ . Per definizione di booleani, questa applicazione restituisce  $v$  se  $b$  è  $\text{tru}$  e  $w$  se  $b$  è  $\text{fls}$ , quindi la prima e la seconda funzione di proiezione  $\text{fst}$  e  $\text{snd}$  possono essere implementate semplicemente fornendo il valore booleano appropriato.

■ **Esempio 5.7** Per verificare che  $\text{fst } (\text{pair } v \ w) \rightarrow^* v$ , calcolare come segue:

|                                                                        |                                |
|------------------------------------------------------------------------|--------------------------------|
| $\text{fst } (\text{pair } v \ w)$                                     |                                |
| $= \text{fst } ((\lambda f. \lambda s. \lambda b. b \ f \ s) \ v \ w)$ | (per definizione di pair)      |
| $\rightarrow \text{fst } ((\lambda s. \lambda b. b \ v \ s) \ w)$      | (riduzione redex sottolineato) |
| $\rightarrow \text{fst } ((\lambda b. b \ v \ w))$                     | (riduzione redex sottolineato) |
| $= (\lambda p. p \ \text{tru}) (\lambda b. b \ v \ w)$                 | (per definizione di fst)       |
| $\rightarrow (\lambda b. b \ v \ w) \ \text{tru}$                      | (riduzione redex sottolineato) |
| $\rightarrow \text{tru } v \ w$                                        | (riduzione redex sottolineato) |
| $= v$                                                                  |                                |

### 5.3.4 Numerali di Church

Rappresentare i numeri con termini lambda è solo leggermente più complicato di quanto abbiamo appena visto. I **numerali di Church**  $c_0, c_1, c_2$ , ecc., sono definiti come segue:

```
c0 = λs.λz. z
c1 = λs.λz. s z
c2 = λs.λz. s (s z)
c3 = λs.λz. s (s (s z))
```

Cioè, ogni numero  $n$  è rappresentato da un combinatore  $c_n$  che prende due argomenti,  $s$  e  $z$  (per “successore” e “zero”), e applica  $s$ ,  $n$  volte, a  $z$ . Come con i valori booleani e le coppie, questa codifica trasforma i numeri in entità attive: il numero  $n$  è rappresentato da una funzione che esegue qualcosa  $n$  volte, una specie di numerale unario attivo.

7

Possiamo definire la funzione *successore* con i numerali di Church come segue:

```
succ = λn.λs.λz. s (n s z)
```

Il termine  $\text{succ}$  è un combinatore che prende un numerale di Church  $n$  e restituisce un altro numerale di Church, cioè produce una funzione che prende argomenti  $s$  e  $z$  e applica ripetutamente  $s$  a  $z$ .

<sup>7</sup>Il lettore potrebbe aver già osservato che  $c_0$  e  $\text{fls}$  sono effettivamente lo stesso termine. Simili “giochi di parole” sono comuni nei linguaggi assembly, in cui lo stesso modello di bit può rappresentare molti valori diversi (un int, un float, un indirizzo, quattro caratteri, ecc.) a seconda di come viene interpretato e nei linguaggi di basso livello come C, che identifica anche 0 e *false*.

Otteniamo il giusto numero di applicazioni da  $s$  a  $z$  passando prima  $s$  e  $z$  come argomenti a  $n$ , e poi applicando esplicitamente  $s$  ancora una volta al risultato.

■ **Esempio 5.8** Supponiamo di voler applicare  $\text{succ}$  a  $c_1$ :

$$\text{succ } c_1 = (\lambda n. \lambda s. \lambda z. s \ (n \ s \ z)) \ c_1$$

Per applicare  $\text{succ}$  a  $c_1$ , si segue questa procedura:

- Sostituire  $c_1$  nella formula di  $\text{succ}$  :

$$\text{succ } c_1 = \lambda s. \lambda z. s \ (c_1 \ s \ z)$$

- Rinominare le variabili di  $c_1$  nella formula di  $\text{succ}$  :

$$\text{succ } c_1 = \lambda s. \lambda z. s \ (([s/s_1][z/z_1]c_1) \ s \ z)$$

- Sostituire le variabili di  $c_1$  nella formula di  $\text{succ}$  :

$$\text{succ } c_1 = \lambda s. \lambda z. s \ ((\lambda s_1. \lambda z_1. s_1 \ z_1) \ s \ z)$$

- Sostituire ancora le variabili:

$$\text{succ } c_1 = \lambda s. \lambda z. s \ (s \ z)$$

Quindi  $\text{succ } c_1 = c_2$ .

Allo stesso modo, l'**addizione di numerali di Church** può essere eseguita da un termine più che prende due numerali di Church,  $m$  e  $n$ , come argomenti, e produce un altro numerale di Church, cioè una funzione, che accetta gli argomenti  $s$  e  $z$ , applica  $s$  iterato  $n$  volte a  $z$  (passando  $s$  e  $z$  come argomenti a  $n$ ), quindi applica  $s$  iterato  $m$  più volte al risultato:

$$\text{plus} = \lambda m. \lambda n. \lambda s. \lambda z. m \ s \ (n \ s \ z)$$

L'implementazione della **moltiplicazione** usa un altro trucco: poiché  $\text{plus}$  prende i suoi argomenti uno alla volta, applicandolo a un solo argomento  $n$  si ottiene la funzione che aggiunge  $n$  a qualunque argomento gli venga dato. Passare questa funzione come primo argomento a  $m$  e  $c_0$  come secondo argomento significa "applicare la funzione che aggiunge  $n$  al suo argomento, ripetuto  $m$  volte, a zero", cioè "sommare  $m$  copie di  $n$ ".

$$\text{mult} = \lambda m. \lambda n. m \ (\text{plus } n) \ c_0$$

Per testare se un numerale di Church è zero, dobbiamo trovare una coppia di argomenti appropriati che ci restituiscano questa informazione, specificamente dobbiamo applicare il nostro numero a una coppia di termini  $zz$  e  $ss$  in modo tale che applicando  $ss$  a  $zz$  una o più volte si ottenga  $\text{fls}$ , mentre non applicandolo affatto produce  $\text{tru}$ . Chiaramente, dovremmo prendere  $zz$  come  $\text{tru}$ . Per  $ss$ , usiamo una funzione che elimina il suo argomento e restituisce sempre  $\text{fls}$ :

$$\text{iszero} = \lambda m. m \ (\lambda x. \text{fls}) \ \text{tru}$$

Sorprendentemente, la sottrazione usando i numerali di Church è un po' più difficile dell'addizione. Può essere fatto usando la seguente "funzione predecessore" piuttosto complicata che, dato  $c_0$  come argomento, restituisce  $c_0$  e, dato  $c_{i+1}$ , restituisce  $c_i$ .

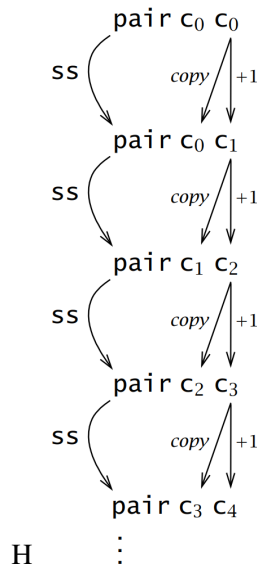


Figura 5.1: Il predecessore di un numero di Church

```

zz = pair c0 c0;
ss = λp. pair (snd p) (plus c1 (snd p))
prd = λm. (fst (m ss zz))

```

Questa definizione funziona utilizzando  $m$  come funzione per applicare  $m$  copie della funzione  $ss$  al valore iniziale  $zz$ . Ogni copia di  $ss$  prende come argomento una coppia di numerali  $\text{pair } c_i c_j$  e restituisce come risultato la coppia  $\text{pair } c_j c_{j+1}$  (vedi Figura 5.1). Quindi applicando  $ss$ ,  $m$  volte, alla coppia  $c_0 c_0$  si ottiene la coppia  $c_0 c_0$  quando  $m = 0$  e la coppia  $c_{m-1} c_m$  quando  $m$  è positivo. In entrambi i casi, il predecessore di  $m$  si trova nel primo componente.

## 5.4 Formalità

Per il resto del capitolo, consideriamo più dettagliatamente la sintassi e la semantica operativa del lambda-calcolo. La maggior parte della struttura di cui abbiamo bisogno è strettamente analoga a quella che abbiamo visto nel Capitolo 4 (per evitare di ripetere quella struttura alla lettera, ci rivolgiamo qui solo al puro lambda-calcolo, privo di ornamenti booleani o numeri). Tuttavia, l'operazione di sostituzione di un termine per una variabile comporta alcune sottigliezze sorprendenti.

### 5.4.1 Sintassi

Come nel capitolo 4, i termini che definiscono la grammatica astratta dovrebbero essere letti come una scorciatoia per un insieme definito induttivamente di alberi di sintassi astratti.

È più interessante che possiamo dare una semplice definizione induttiva dell'insieme di variabili che appaiono libere in un termine lambda.

**Definizione 5.4.1 — Insieme delle variabili libere.** L'insieme delle variabili libere in un termine  $t$ , scritto  $FV(t)$ , è l'insieme delle variabili che appaiono in  $t$  ma non sono legate da una

quantificazione. Esso è definito come segue:

$$\begin{aligned} FV(x) &= \{x\} \\ FV(\lambda x. t_1) &= FV(t_1) \setminus \{x\} \\ FV(t_1 t_2) &= FV(t_1) \cup FV(t_2) \end{aligned}$$

### 5.4.2 Sostituzione

L'operazione di sostituzione risulta piuttosto delicata, se esaminata in dettaglio. La operazione di sostituzione introdotta in questa sezione, è compatto e intuitivo e funziona bene per esempi e definizioni e dimostrazioni matematiche.

È istruttivo arrivare a una definizione di sostituzione attraverso un paio di tentativi sbagliati. Innanzitutto, proviamo la definizione ricorsiva più ingenua possibile. (Formalmente, stiamo definendo una funzione  $[x \mapsto s]$  per induzione sul suo argomento  $t$ .)

$$\begin{aligned} [x \mapsto s]x &= s \\ [x \mapsto s]y &= y && \text{se } y \neq x \\ [x \mapsto s](\lambda y. t) &= \lambda y. [x \mapsto s]t \\ [x \mapsto s](t_1 t_2) &= ([x \mapsto s]t_1) ([x \mapsto s]t_2) \end{aligned}$$

Questa definizione funziona bene per la maggior parte degli esempi. Ad esempio:

$$[x \mapsto (\lambda z. z w)](\lambda y. x) = \lambda y. \lambda z. z w$$

che corrisponde alle nostre intuizioni su come dovrebbe comportarsi la sostituzione. Tuttavia, se siamo sfortunati con la nostra scelta dei nomi delle variabili associate, la definizione fallisce. Per esempio:

$$[x \mapsto y](\lambda x. x) = \lambda x. y$$

Ciò è in conflitto con l'intuizione di base sulle astrazioni funzionali secondo cui *i nomi delle variabili vincolate non contano*: la funzione identità è esattamente la stessa sia che la scriviamo  $\lambda x. x$  o  $\lambda y. y$  o  $\lambda mhanz. mhanz$ . Se questi non si comportano esattamente allo stesso modo sotto sostituzione, allora non si comporteranno allo stesso modo nemmeno sotto riduzione, il che sembra sbagliato.

Chiaramente, il primo errore che abbiamo commesso nella definizione ingenua di sostituzione è che non abbiamo distinto tra *occorrenze libere* di una variabile  $x$  in un termine  $t$  (che dovrebbe essere sostituito durante la sostituzione) e *occorrenze legate* (bound), che non dovrebbero. Quando raggiungiamo un'astrazione che lega il nome  $x$  all'interno di  $t$ , l'operazione di sostituzione dovrebbe interrompersi. Questo porta al prossimo tentativo:

$$\begin{aligned} [x \mapsto s]x &= s \\ [x \mapsto s]y &= y && \text{se } y \neq x \\ [x \mapsto s](\lambda y. t) &= \begin{cases} \lambda y. t & \text{se } y = x \\ \lambda y. [x \mapsto s]t & \text{se } y \neq x \end{cases} \\ [x \mapsto s](t_1 t_2) &= ([x \mapsto s]t_1) ([x \mapsto s]t_2) \end{aligned}$$

Questo è meglio, ma ancora non del tutto corretto. Ad esempio, considera cosa succede quando sostituiamo il termine  $z$  per la variabile  $x$  nel termine  $\lambda z. x$ :

$$[x \mapsto z](\lambda z. x) = \lambda z. z$$

Questa volta abbiamo commesso essenzialmente l'errore opposto: abbiamo trasformato la funzione costante  $\lambda z. x$  nella funzione identità! Di nuovo, questo si è verificato solo perché ci è capitato di scegliere  $z$  come nome della variabile vincolata nella funzione costante, quindi chiaramente c'è ancora qualcosa che non va. Questo fenomeno delle variabili libere in un termine  $s$  che diventano vincolate quando  $s$  viene ingenuamente sostituito in un termine  $t$  è chiamato **variable capture**. Per evitarlo, dobbiamo assicurarci che i nomi delle variabili vincolate di  $t$  siano tenuti distinti dai nomi delle variabili libere di  $s$ . Un'operazione di sostituzione che lo fa correttamente è chiamata sostituzione **capture-avoiding**. (Questo è quasi sempre ciò che si intende con il termine non qualificato "sostituzione".) Possiamo ottenere l'effetto desiderato aggiungendo un'altra condizione secondaria alla seconda clausola del caso di astrazione:

$$\begin{aligned}
 [x \mapsto s]x &= s \\
 [x \mapsto s]y &= y && \text{se } y \neq x \\
 [x \mapsto s](\lambda y. t) &= \begin{cases} \lambda y. t & \text{se } y = x \\ \lambda y. [x \mapsto s]t & \text{se } y \neq x \text{ and } y \notin FV(s) \end{cases} \\
 [x \mapsto s](t_1 t_2) &= ([x \mapsto s]t_1) ([x \mapsto s]t_2)
 \end{aligned}$$

Ora ci siamo quasi: questa definizione di sostituzione fa la cosa giusta quando non fa nulla. Il problema ora è che la nostra ultima correzione ha trasformato la sostituzione in un'operazione parziale. Ad esempio, la nuova definizione non fornisce alcun risultato per  $[x \mapsto y z](\lambda y. x y)$ : la variabile vincolata  $y$  del termine sostituito dentro non è uguale a  $x$ , ma appare libera in  $(y z)$ , quindi nessuna delle clausole della definizione si applica. Una soluzione comune per quest'ultimo problema nei sistemi di tipi e nella letteratura lambda-calcolo è lavorare con i termini "fino alla ridenominazione delle variabili associate"<sup>8</sup>.

Per convenzione si dice che: *I termini che differiscono solo nei nomi delle variabili legate sono intercambiabili in tutti i contesti.*

Ciò significa in pratica che il nome di qualsiasi variabile legata a  $\lambda$  può essere cambiato in un altro nome (facendo costantemente lo stesso cambiamento nel corpo di  $\lambda$ ), in qualsiasi punto in cui ciò sia conveniente.

■ **Esempio 5.9** Ad esempio, se vogliamo calcolare  $[x \mapsto y z](\lambda y. x y)$ , riscriviamo prima  $(\lambda y. x y)$  come, diciamo,  $(\lambda w. x w)$ . Calcoliamo quindi  $[x \mapsto y z](\lambda w. x w)$ , ottenendo  $(\lambda w. y z w)$

Questa convenzione rende l'operazione di sostituzione "buona" siccome totale, poiché ogni volta che ci troviamo ad applicarla ad argomenti per i quali è indefinita, possiamo rinominarla come necessario, in modo che le condizioni secondarie siano soddisfatte. In effetti, adottando questa convenzione, possiamo formulare la definizione di sostituzione in modo un po' più conciso. La prima clausola per le astrazioni può essere eliminata, poiché possiamo sempre assumere (rinominando se necessario) che la variabile vincolata  $y$  sia diversa sia da  $x$  che dalle variabili libere di  $s$ . Questo produce la forma finale della definizione.

#### Definizione 5.4.2 — Sostituzione.

$$\begin{aligned}
 [x \mapsto s]x &= s \\
 [x \mapsto s]y &= y && \text{se } y \neq x \\
 [x \mapsto s](\lambda y. t) &= \lambda y. [x \mapsto s]t && \text{se } y \neq x \text{ and } y \notin FV(s) \\
 [x \mapsto s](t_1 t_2) &= ([x \mapsto s]t_1) ([x \mapsto s]t_2)
 \end{aligned}$$

<sup>8</sup>Church ha usato il termine conversione alfa per l'operazione di ridenominazione coerente di una variabile vincolata in un termine. Questa terminologia è ancora comune: potremmo anche dire che stiamo lavorando con termini "fino alla conversione alfa".



## 5.4.3 Semantica operativa

| <i>Sintassi:</i> |                          | <i>Evaluation:</i>                                           | $t \rightarrow t'$    |
|------------------|--------------------------|--------------------------------------------------------------|-----------------------|
| $t ::= \dots$    | (termini)                | $t_1 \rightarrow t'_1$                                       |                       |
| $x$              | (variabile)              | $t_1 t_2 \rightarrow t'_1 t_2$                               | (E-APP <sub>1</sub> ) |
| $\lambda x. t$   | (astrazione)             | $t_2 \rightarrow t'_2$                                       | (E-APP <sub>2</sub> ) |
| $t t$            | (applicazione)           | $v_1 t_2 \rightarrow v_1 t'_2$                               |                       |
|                  |                          | $(\lambda x. t_{12}) v_2 \rightarrow [x \mapsto v_2] t_{12}$ | (E-APPABS)            |
| $v ::= \dots$    | (valori)                 |                                                              |                       |
| $\lambda x. t$   | (valore dell'astrazione) |                                                              |                       |

Figura 5.2: Lambda-calcolo non tipato definite con la small-step semantic

La semantica operativa dei lambda-termini è riassunta nella Figura 5.2. L'insieme di valori qui è più interessante di quanto visto nel caso delle espressioni aritmetiche. Poiché la valutazione (call-by-value) si interrompe quando raggiunge un lambda, i valori possono essere termini lambda arbitrari.

La relazione di valutazione appare nella colonna di destra della figura. Come nella valutazione delle espressioni aritmetiche, ci sono due tipi di regole: la regola di calcolo E-APPABS e le regole di congruenza E-APP<sub>1</sub> e E-APP<sub>2</sub>.

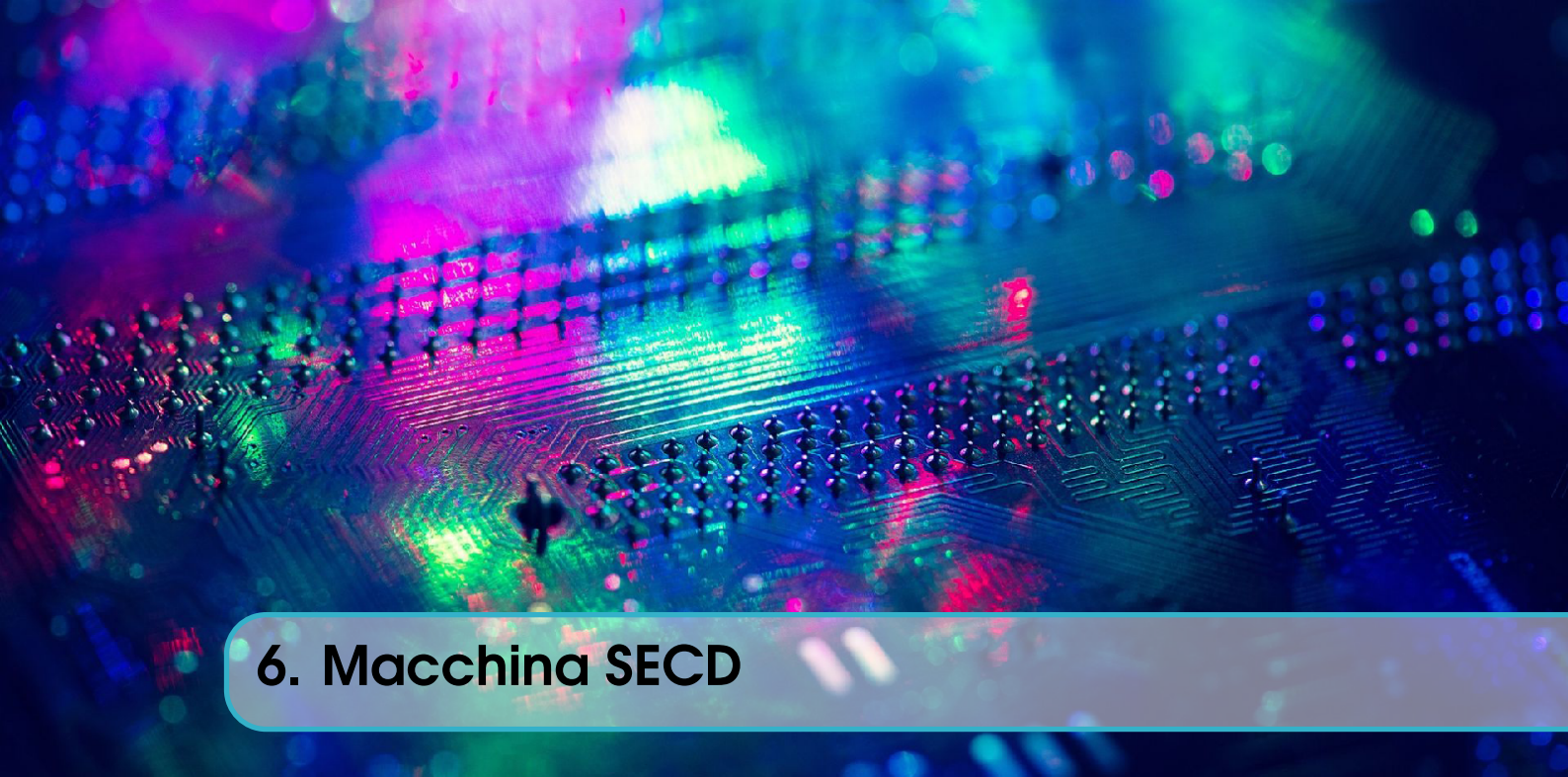
Si noti come la scelta delle metavariable in queste regole aiuti a controllare l'ordine di valutazione. Poiché  $v_2$  varia solo sui valori, il lato sinistro della regola E-APPABS può corrispondere a qualsiasi applicazione il cui lato destro è un valore. Allo stesso modo, la regola E-APP<sub>1</sub> si applica a qualsiasi applicazione il cui lato sinistro non è un valore, poiché  $t_1$  può corrispondere a qualsiasi termine, ma la premessa richiede inoltre che  $t_1$  possa fare un passo. E-APP<sub>2</sub>, d'altra parte, non può attivarsi fino a quando il lato sinistro non è un valore in modo che possa essere associato alla metavariable valore  $v$ . Prese insieme, queste regole determinano completamente l'ordine di valutazione per un'applicazione  $t_1 t_2$ : usiamo prima E-APP<sub>1</sub> per ridurre  $t_1$  a un valore, quindi usiamo E-APP<sub>2</sub> per ridurre  $t_2$  a un valore e infine usiamo E-APPABS per eseguire l'applicazione stessa.

Si noti che, nel puro lambda-calcolo, le lambda-astrazioni sono gli unici valori possibili, quindi se raggiungiamo uno stato in cui E-APP<sub>1</sub> è riuscito a ridurre  $t_1$  a un valore, allora questo valore deve essere una lambda-astrazione. Questa osservazione fallisce, ovviamente, quando aggiungiamo al linguaggio altri costrutti come booleani primitivi, poiché questi introducono forme di valori diverse dalle astrazioni.

# Parte 2

|           |                                                                          |            |
|-----------|--------------------------------------------------------------------------|------------|
| <b>6</b>  | <b>Macchina SECD</b> .....                                               | <b>59</b>  |
| 6.1       | Introduzione                                                             |            |
| 6.2       | Descrizione della macchina                                               |            |
| 6.3       | L'interprete                                                             |            |
| <b>7</b>  | <b>Espressioni aritmetiche tipizzate</b> .....                           | <b>63</b>  |
| 7.1       | Tipi                                                                     |            |
| 7.2       | La relazione di tipizzazione                                             |            |
| 7.3       | Sicurezza = Progresso + Preservazione                                    |            |
| <b>8</b>  | <b>Lambda Calcolo tipizzato semplice</b> ..                              | <b>67</b>  |
| 8.1       | Tipi di funzione                                                         |            |
| 8.2       | La relazione di tipizzazione                                             |            |
| 8.3       | Vaneggi del prof                                                         |            |
| <b>9</b>  | <b>Linguaggio imperativo</b> .....                                       | <b>73</b>  |
| 9.1       | Lo stato                                                                 |            |
| 9.2       | Il cuore di un linguaggio imperativo: IMP                                |            |
| <b>10</b> | <b>Ambiente e Memoria</b> .....                                          | <b>79</b>  |
| 10.1      | Ambiente                                                                 |            |
| 10.2      | Scope                                                                    |            |
| 10.3      | Ambiente locale                                                          |            |
| 10.4      | Gestione della memoria                                                   |            |
| 10.5      | Memoria e pila degli RdA                                                 |            |
| 10.6      | Ambiente e pila degli RdA                                                |            |
| <b>11</b> | <b>Il linguaggio IMP con i blocchi e le dichiarazioni (locali)</b> ..... | <b>93</b>  |
| 11.1      | Il linguaggio IMP con i blocchi e le dichiarazioni (locali)              |            |
| <b>12</b> | <b>Parametri, astrazione e higher order</b> ..                           | <b>97</b>  |
| 12.1      | Programmi strutturati                                                    |            |
| 12.2      | Astrazione                                                               |            |
| 12.3      | Higher order                                                             |            |
| <b>13</b> | <b>Linguaggio Funzionale</b> .....                                       | <b>107</b> |
| <b>14</b> | <b>Esercizi</b> .....                                                    | <b>109</b> |
|           | <b>Index</b> .....                                                       | <b>118</b> |





## 6. Macchina SECD

### 6.1 Introduzione

La macchina SECD è una macchina virtuale creata da Peter Landin molto, molto tempo fa. La macchina si rivolge a linguaggi funzionali e ha un set di istruzioni piccolo e (relativamente) semplice. Le lettere stanno per **S**tack, **E**nvironment, **C**ontrol, **D**ump, che sono i registri interni della macchina. I registri **Stack**, **Control** e **Dump** puntano a degli **stack**, mentre **Environment** punta a un **array associativo**.

La macchina SECD è stata la prima ad essere specificamente progettata per valutare le espressioni di lambda-calcolo. È stato originariamente descritto da Peter J. Landin in "*The Mechanical Evaluation of Expressions*" nel 1964. La descrizione pubblicata da Landin era piuttosto astratta e lasciava aperte molte scelte di implementazione (come una semantica operativa).

### 6.2 Descrizione della macchina

Il linguaggio è il seguente:

$$t ::= c_i \mid x \mid op_i(t_1, \dots, t_{a_i}) \mid \text{if}(t_1, t_2, t_3) \mid \text{fun}(f, x, t) \mid \text{apply}(t_1, t_2) \quad (6.1)$$

Dove:

- $c_i$  sono le costanti del linguaggio
- $x$  sono gli identificatori (che devono avere un nome nell'ambiente)
- $op_i$  sono un numero di operazioni primitive, ognuna con la propria arietà (numero di argomenti)
- $\text{if}(t_1, t_2, t_3)$  è un costrutto condizionale
- $\text{fun}(f, x, t)$  è una funzione anonima
- $\text{apply}(t_1, t_2)$  è l'applicazione di una funzione

La macchina è una pila dove ogni elemento della pila (**Stack**) è una tripla  $(S, E, C)$  dove **S** è uno stack di risultati, **E** è un'ambiente (funzione da nomi a valori denotabili), e **C** è una stack di elementi da valutare. La macchina SECD ha anche un registro **Dump** che contiene una stack di coppie  $(S, E)$ . Il registro **Dump** viene usato per salvare lo stato della macchina quando si entra in una funzione.

Dato che abbiamo i valori denotabili, dobbiamo sostanzialmente dire quali sono: interi, booleani e funzioni (**fun**( $f, x, t$ )).

1.  $S$  è una pila di valori denotabili (ogni valore lo indichiamo con  $v$ )
2.  $E$  è una funzione che associa ad ogni identificatore  $x$  la sua denotazione
3.  $C$  è una pila di elementi di controllo, dove un elemento di controllo obbedisce alla seguente grammatica:

$$ce ::= t \mid \text{Prim}(op_i) \mid \text{App} \mid \text{if} \quad (6.2)$$

Le  $\text{Prim}(op_i)$  sono la controparte implementativa delle varie  $op_i$  sintattiche e  $\tilde{op}_i$  sono la controparte *semantica*

4. lo **stato iniziale** della macchina è  $([], \perp, t :: []) :: []$  dove  $\perp$  è la funzione ovunque indefinita (o se volete quella che ad ogni variabile  $x$  associa il valore *undefined*)
5. Lo **stato finale** della macchina è  $(v :: [], \perp, []) :: []$ .

### 6.3 L'interprete

Di seguito vengono presentate le regole per il funzionamento della macchina:

$$(S, E, op_i(t_1, \dots, t_{a_i}) :: C) :: D \mapsto (S, E, t_1 :: \dots :: t_{a_i} :: \text{Prim}(op_i) :: C) :: D \quad (\text{implementazione operatore})$$

$$(S, E, \text{if}(t_1, t_2, t_3) :: C) :: D \mapsto (S, E, t_1 :: \text{If} :: t_2 :: t_3 :: C) :: D \quad (\text{implementazione if})$$

$$(S, E, \text{apply}(t_1, t_2) :: C) :: D \mapsto (S, E, t_1 :: t_2 :: \text{App} :: C) :: D \quad (\text{implementazione apply})$$

$$(S, E, \mathbf{n} :: C) :: D \mapsto (n :: S, E, C) :: D, \quad \text{dove } \mathbf{n} \text{ è un numero intero} \quad (\text{valutazione costante intera})$$

$$(S, E, \mathbf{b} :: C) :: D \mapsto (b :: S, E, C) :: D, \quad \text{dove } \mathbf{b} \text{ è un booleano} \quad (\text{valutazione costante booleana})$$

$$(S, E, \text{var}(x) :: C) :: D \mapsto (E(x) :: S, E, C) :: D, \quad \text{dove } \text{var}(\mathbf{x}) \text{ è una variabile} \quad (\text{valutazione variabile})$$

$$(S, E, \text{fun}(f, x, t) :: C) :: D \mapsto (\text{fun}(f, x, t) :: S, E, C) :: D \quad (\text{valutazione funzione})$$

$$(v_2 :: v_1 :: S, E, \text{App} :: C) :: D \mapsto ([], E'[v_1/f, v_2/x], t :: []) :: (S, E, C) :: D, \quad \text{dove } v_1 = \text{fun}(f, x, t) \quad (\text{valutazione applicazione funzione})$$

$$(v :: S, E, []) :: (S', E', C') :: D \mapsto (v :: S', E', C') :: D \quad (\text{valutazione terminale})$$

$$(v_{a_i} :: \dots :: v_1 :: S, E, \text{Prim}(op_i) :: C) :: D \mapsto (\tilde{op}_i(v_1, \dots, v_{a_i}) :: S, E, C) :: D \quad (\text{valutazione operatore})$$

$$(v :: S, E, \text{If} :: t_2 :: t_3 :: C) :: D \mapsto (S, E, t_2 :: C) :: D, \quad \text{se } v = \text{true} \quad (\text{valutazione if true})$$

$$(v :: S, E, \text{If} :: t_2 :: t_3 :: C) :: D \mapsto (S, E, t_3 :: C) :: D, \quad \text{se } v = \text{false} \quad (\text{valutazione if false})$$

Questo insieme di regole può sembrare complesso, per questo verranno presentati alcuni esempi di valutazione.

■ **Esempio 6.1** Supponiamo di volere effettuare la valutazione dell'espressione  $1 + 2$  utilizzando la SE machine. Inizialmente la macchina si trova nello stato iniziale, quindi:

$$(S, E, t :: C) :: D \mapsto ([], \perp, t :: []) :: [] \quad (6.3)$$

dove:

- $S = []$
- $E = \perp$
- $t = 1 + 2$
- $C = []$
- $D = []$

$1 + 2$  è un'operazione primitiva e si rappresenta con  $\text{più}(1, 2)$ . Viene poi applicata la regola di implementazione dell'operatore:

$$([], \perp, \text{più}(1, 2) :: []) :: [] \mapsto ([], \perp, 1 :: 2 :: \text{Prim}(\text{più}) :: []) :: []$$

dove  $\text{Prim}(\text{più})$  è la controparte implementativa dell'operatore primitivo  $\text{più}$ . Ora nel control stack sono presenti 2 costanti numeriche, viene quindi applicata la regola di valutazione delle costanti numeriche 2 volte:

$$([], \perp, 1 :: 2 :: \text{Prim}(\text{più}) :: []) :: [] \mapsto (1 :: [], \perp, 2 :: \text{Prim}(\text{più}) :: []) :: []$$

$$(1 :: [], \perp, 2 :: \text{Prim}(\text{più}) :: []) :: [] \mapsto (2 :: 1 :: [], \perp, \text{Prim}(\text{più}) :: []) :: []$$

Ora nel control stack è presente l'implementazione dell'operatore primitivo  $\text{più}$ , viene quindi applicata la regola di valutazione dell'operatore:

$$(2 :: 1 :: [], \perp, \text{Prim}(\text{più}) :: []) :: [] \mapsto (3 :: [], \perp, []) :: []$$

dove 3 è il risultato dell'operazione  $1 + 2$ . Ora il control stack è vuoto, viene quindi applicata la regola di valutazione terminale:

$$(3 :: [], \perp, []) :: [] \mapsto (3 :: [], \perp, []) :: []$$

■ **Esempio 6.2** Supponiamo di voler calcolare il valore di  $\text{id}(x)$  dove  $\text{id}(x) = x$  e  $x = 3$ .  $\text{id}(x)$  è una funzione e si rappresenta con  $\text{fun}(\text{id}, x, x)$ . Per applicare la funzione  $\text{id}$  a 3 si utilizza  $\text{apply}(\text{fun}(\text{id}, x, x), 3)$  Inizialmente la macchina si trova nello stato iniziale, quindi:

$$(S, E, t :: C) :: D \mapsto ([], \perp, t :: []) :: [] \quad (6.4)$$

dove:

- $S = []$
- $E = \perp$
- $t = \text{apply}(\text{fun}(\text{id}, x, x), 3)$
- $C = []$
- $D = []$

Nel control stack è presente l'applicazione della funzione  $\text{id}$  a 3, viene quindi applicata la regola di valutazione dell'implementazione dell'apply:

$$([], \perp, \text{apply}(\text{fun}(\text{id}, x, x), 3) :: []) :: [] \mapsto ([], \perp, \text{fun}(\text{id}, x, x) :: 3 :: \text{app} :: []) :: []$$

Adesso nel control stack è presente la funzione, quindi si applica la regola di valutazione della funzione:

$$([], \perp, \text{fun}(\text{id}, x, x) :: 3 :: \text{app} :: []) :: [] \mapsto (\text{fun}(\text{id}, x, x) :: [], \perp, 3 :: \text{app} :: []) :: []$$

Adesso nel control stack è presente una costante numerica, quindi si applica la regola di valutazione delle costanti numeriche:

$$(\text{fun}(id, x, x) :: [], \perp, 3 :: \text{app} :: []) :: [] \mapsto (3 :: \text{fun}(id, x, x) :: [], \perp, \text{app} :: []) :: []$$

Nel control stack è presente l'implementazione dell'apply, quindi si applica la regola di valutazione dell'applicazione di funzione:

$$(3 :: \text{fun}(id, x, x) :: [], \perp, \text{app} :: []) :: [] \mapsto ([], \perp, [\text{fun}(id, x, x)/id, 3/x], x :: []) :: ([], \perp, []) :: []$$

dove  $[\text{fun}(id, x, x)/id, 3/x]$  è l'ambiente in cui è stata applicata la funzione  $id$  a 3. Ora nel control stack è presente una variabile, quindi si applica la regola di valutazione delle variabili:

$$([], \perp, [\text{fun}(id, x, x)/id, 3/x], x :: []) :: ([], \perp, []) :: [] \mapsto (3 :: [], \perp, [\text{fun}(id, x, x)/id, 3/x], []) :: ([], \perp, []) :: []$$

Ora il control stack è vuoto, viene quindi applicata la regola di valutazione terminale:

$$(3 :: [], \perp, [\text{fun}(id, x, x)/id, 3/x], []) :: ([], \perp, []) :: [] \mapsto (3 :: [], \perp, []) :: []$$

dove  $S', E', C', D' = ([], \perp, []) :: []$ . Il risultato è 3.

#### Esercizio 6.1 — Esercizio d'esame. Durante l'esecuzione della seguente SECD:

$$([], \perp, \text{apply}(\text{fun}(f, x, \text{apply}(x, \text{apply}(\text{fun}(g, y, \text{times}(y, 2)), 1))), \text{fun}(h, z, \text{diff}(8, z))) :: []) :: []$$

non ci sono le seguenti configurazioni:

1.  $(\text{fun}(f, x, \text{apply}(x, \text{apply}(\text{fun}(g, y, \text{times}(y, 2)), 1))) :: [], \perp, \text{fun}(h, z, \text{diff}(8, z)) :: \text{App} :: []) :: []$
2.  $(\text{fun}(f, x, \text{apply}(x, \text{apply}(\text{fun}(g, y, \text{times}(y, 2)), 1))) :: \text{fun}(h, z, \text{diff}(8, z)) :: [], \perp, \text{App} :: []) :: []$
3.  $(\text{fun}(f, x, \text{apply}(x, \text{apply}(\text{fun}(g, y, \text{times}(y, 2)), 1))) :: \text{fun}(h, z, \text{diff}(8, z)) :: [], \perp, \text{App} :: []) :: []$
4.  $(10 :: [], \perp, []) :: []$

**Soluzione:** Per la soluzione guardare il powerpoint "*Esercizio SECD*". La soluzione dovrebbe essere che non sono presenti le configurazioni 2 e 4, mentre sono presenti le configurazioni 1 e 3. L'esercizio si può risolvere in maniera più semplice di quella mostrata nel powerpoint, siccome osservando l'esercizio si può notare eseguendo solo i primi passaggi che la configurazione 2 non può essere presente (mentre la 1 e la 3 sì) in quanto non è possibile che la funzione  $h$  sia presente nella prima della funzione  $f$  nel control stack (in quanto la funzione  $h$  viene passata come parametro alla funzione  $f$ ). Infine per la configurazione 4, non è necessario calcolare tutto con la SECD machine, infatti se si capisce come è strutturata la funzione finale si può calcolare a mano che il risultato sarà 6 e non 10, quindi la configurazione 4 non può essere presente.



## 7. Espressioni aritmetiche tipizzate

Nei precedenti capitoli, abbiamo utilizzato un semplice linguaggio di espressioni booleane e aritmetiche per introdurre strumenti di base per la descrizione precisa della sintassi e della valutazione. Torniamo ora a questo semplice linguaggio e lo incrementiamo con tipi statici. Ancora una volta, il sistema di tipi in sé è quasi banale, ma fornisce un ambiente in cui introdurre concetti che ricorreranno in tutto il libro

### 7.1 Tipi

Ricordiamo la sintassi per le espressioni aritmetiche

|                    |                           |
|--------------------|---------------------------|
| $t ::=$            | (termini)                 |
| true               | (costante true)           |
| false              | (costante false)          |
| if t then t else t | (condizionale)            |
| 0                  | (costante 0)              |
| succ t             | (successore)              |
| pred t             | (predecessore)            |
| iszero t           | (test di uguaglianza a 0) |

Abbiamo anche definito i valori di questo linguaggio come segue:

|          |                                    |
|----------|------------------------------------|
| $v ::=$  | (valori)                           |
| true     | (valore true)                      |
| false    | (valore false)                     |
| nv       | (valori numerici)                  |
| $nv ::=$ | (valori numerici)                  |
| 0        | (valore 0)                         |
| succ nv  | (successore di un valore numerico) |

Valutando un termine si possono ottenere i valori definiti sopra, oppure si può ottenere un errore di valutazione. Per esempio, valutando il termine `pred false` si ottiene un errore di valutazione. Questo perché il linguaggio non è tipizzato, quindi non è possibile distinguere tra valori numerici e valori booleani. Per esempio, il termine `0` può essere sia un valore numerico che un valore booleano. Per questo motivo, il linguaggio non è in grado di distinguere tra i due casi.

I termini bloccati corrispondono a programmi privi di significato o errati. Vorremmo quindi poter dire, senza valutare effettivamente un termine, che la sua valutazione *non si bloccherà* sicuramente. Per fare ciò, dobbiamo essere in grado di distinguere tra termini il cui risultato sarà un valore numerico (poiché questi sono gli unici che dovrebbero apparire come argomenti di `pred`, `succ` e `iszero`) e termini il cui risultato sarà un valore booleano (poiché solo questi dovrebbero apparire come la condizione di un condizionale). Introduciamo due tipi, `Nat` e `Bool`, per classificare i termini in questo modo. Le metavariable `S`, `T`, `U`, ecc. verranno utilizzate in tutto il libro per variare i tipi.

Dire che “un termine `t` ha tipo `T`” (o “`t` appartiene a `T`,” o “`t` è un elemento di `T`”) significa che `t` “ovviamente” valuta un valore della forma appropriata dove per “ovviamente” intendiamo che possiamo vederlo staticamente, senza fare alcuna valutazione di `t`. Ad esempio, il termine `if true then false else true` ha il tipo `Bool`, mentre `pred (succ (pred (succ 0)))` ha il tipo `Nat`. Tuttavia, la nostra analisi dei tipi di termini sarà conservativa, facendo uso solo di informazioni statiche. Ciò significa che non saremo in grado di concludere che termini come `if (iszero 0) then 0 else false` o `if true then 0 else false` abbiano alcun tipo, anche se la loro valutazione, di fatto, non si blocca.

## 7.2 La relazione di tipizzazione

La relazione di tipizzazione per le espressioni aritmetiche, scritte <sup>1</sup> “`t : T`”, è definita da un insieme di regole di inferenza che assegnano i tipi ai termini, riassunte nelle figure 7.1 e 7.2. Come nel capitolo 4 diamo le regole per i booleani e quelle per i numeri in due figure diverse, poiché in seguito vorremo talvolta farvi riferimento separatamente.

Le regole `T-TRUE` e `T-FALSE` nella Figura 7.1 assegnano il tipo `Bool` a costanti booleane `true` e `false`. La regola `T-IF` assegna un tipo a un’espressione condizionale in base ai tipi delle sue sottoespressioni: la condizione `t1` deve restituire un valore booleano, mentre `t2` e `t3` devono entrambi restituire valori dello stesso tipo. I due usi della singola metavariable `T` esprimono il vincolo che il risultato dell’`if` sia il tipo dei rami `then`- e `else`-, e che questo possa essere di qualsiasi tipo (o `Nat` o `Bool` o, quando arriviamo a calcoli con più insiemi interessanti di tipi, qualsiasi altro tipo). Le regole per i numeri nella Figura 7.2 hanno una forma simile. `T-ZERO` assegna il tipo `Nat` alla costante `0`. `T-SUCC` assegna a un termine della forma `succ t` il tipo `Nat`, purché `t` abbia il tipo `Nat`. Allo stesso modo, `T-PRED` e `T-ISZERO` affermano che `pred` produce un `Nat` quando il suo argomento ha il tipo `Nat` e `iszero` produce un `Bool` quando il suo argomento ha il tipo `Nat`.

<sup>1</sup> Spesso è utilizzato il simbolo  $\in$  anziché :

| Nuova forma sintattica: |                     | Nuove regole di tipizzazione                                                            |           | $t : T$ |
|-------------------------|---------------------|-----------------------------------------------------------------------------------------|-----------|---------|
| $T ::=$                 | (tipi)              | $true : Bool$                                                                           | (T-TRUE)  |         |
| $Bool$                  | (tipo dei booleani) | $false : Bool$                                                                          | (T-FALSE) |         |
|                         |                     | $\frac{t_1 : Bool \quad t_2 : T \quad t_3 : T}{if \ t_1 \ then \ t_2 \ else \ t_3 : T}$ | (T-IF)    |         |

Figura 7.1: Regole di tipizzazione per booleani (B) [Estende B (fig 4.1)]

| Nuova forma sintattica: |                            | Nuove regole di tipizzazione        |            | $t : T$ |
|-------------------------|----------------------------|-------------------------------------|------------|---------|
| $T ::= \dots$           | (tipi)                     | $\frac{t : Nat}{succ \ t : Nat}$    | (T-SUCC)   |         |
| $Nat$                   | (tipo dei numeri naturali) | $\frac{t : Nat}{pred \ t : Nat}$    | (T-PRED)   |         |
|                         |                            | $\frac{t : Nat}{iszero \ t : Bool}$ | (T-ISZERO) |         |
|                         |                            | $0 : Nat$                           | (T-ZERO)   |         |

Figura 7.2: Regole di tipizzazione per numeri (NB) [Estende NB(fig 4.2) e B (fig 7.1)]

**Definizione 7.2.1 — Relazione di tipizzazione.** Formalmente, la **relazione di tipizzazione** per le espressioni aritmetiche è la più piccola relazione binaria tra termini e tipi che soddisfa tutte le istanze delle regole nelle Figure 7.1 e 7.2. Un termine  $t$  è tipizzabile (o ben tipizzato, "well typed") se esiste qualche  $T$  tale che  $t : T$

Quando ragioniamo sulla relazione di tipizzazione, faremo spesso affermazioni come "Se un termine della forma  $succ \ t$  ha un qualsiasi tipo, allora ha il tipo  $Nat$ ". Il lemma seguente ci fornisce un compendio di affermazioni di base di questa forma, ciascuna immediatamente successiva alla forma della corrispondente regola di tipizzazione.

|                                                                  |                                                                                            |
|------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <b>Lemma 7.2.1 — Inversione della relazione di tipizzazione.</b> | 1. Se $true : R$ allora $R = Bool$                                                         |
|                                                                  | 2. Se $false : R$ allora $R = Bool$                                                        |
|                                                                  | 3. Se $if \ t_1 \ then \ t_2 \ else \ t_3 : R$ allora $t_1 : Bool$ e $t_2 : R$ e $t_3 : R$ |
|                                                                  | 4. Se $0 : R$ allora $R = Nat$                                                             |
|                                                                  | 5. Se $succ \ t : R$ allora $t : Nat$ e $R = Nat$                                          |
|                                                                  | 6. Se $pred \ t : R$ allora $t : Nat$ e $R = Nat$                                          |
|                                                                  | 7. Se $iszero \ t : R$ allora $t : Nat$ e $R = Bool$                                       |

Il lemma di inversione è talvolta chiamato *lemma di generazione* per la relazione di tipizzazione, poiché, data un'affermazione di tipizzazione valida, mostra come potrebbe essere stata generata una dimostrazione di questa affermazione. Il lemma di inversione conduce direttamente a un algoritmo ricorsivo per il calcolo dei tipi di termini, poiché ci dice, per un termine di ogni forma sintattica, come calcolare il suo tipo (se ne ha uno) dai tipi dei suoi sottotermini. Torneremo in dettaglio su questo punto nel prossimo capitolo.

Nel capitolo 4 abbiamo introdotto il concetto di derivazioni valutative. Allo stesso modo, una

derivazione di tipizzazione è un albero di istanze delle regole di tipizzazione. Ogni coppia  $(t, T)$  nella relazione di tipizzazione è giustificata da una derivazione di tipizzazione con conclusione  $t : T$ .

■ **Esempio 7.1** Ad esempio, ecco l'albero di derivazione per l'istruzione di tipizzazione "if iszero 0 then 0 else pred 0 : Nat":

$$\frac{\frac{\frac{}{0 : \text{Nat}} \text{T-ZERO}}{\text{iszero } 0 : \text{Bool}} \text{T-ISZERO} \quad \frac{\frac{}{0 : \text{Nat}} \text{T-ZERO} \quad \frac{\frac{}{\text{pred } 0 : \text{Nat}} \text{T-ZERO}}{\text{pred } 0 : \text{Nat}} \text{T-PRED}}{\text{if iszero 0 then 0 else pred 0 : Nat}} \text{T-IF} \quad (7.1)$$

In altre parole, le istruzioni sono asserzioni formali sulla tipizzazione dei programmi, le regole di tipizzazione sono implicazioni tra istruzioni e le derivazioni sono deduzioni basate sulle regole di tipizzazione.

**Teorema 7.2.2 — Unicità dei tipi.** Ogni termine  $t$  ha al massimo un tipo. Cioè, se  $t$  è tipizzabile, allora il suo tipo è unico. Inoltre, c'è solo una derivazione di questa tipizzazione costruita dalle regole di inferenza nelle figure 7.1 e 7.2.

Nel sistema di tipi semplice che stiamo trattando in questo capitolo, ogni termine ha un singolo tipo (se ne ha uno), e c'è sempre un solo albero di derivazione che lo dimostra. In realtà con la sottotipizzazione (che non vedremo) un singolo termine può avere molti tipi, e in generale ci possono essere molti modi per derivare la dichiarazione che un dato termine ha un dato tipo.

### 7.3 Sicurezza = Progresso + Preservazione

La più semplice delle proprietà di questo sistema di tipizzazione o di qualsiasi altro è la sicurezza (anche detta *correttezza*): i termini ben tipizzati non vanno "male". Abbiamo già scelto come formalizzare cosa significa che un termine vada "male": significa raggiungere uno "stato bloccato" (Definizione 4.4.6) che non è designato come valore finale ma dove le regole di valutazione non dicono cosa fare dopo. Quindi, quello che vogliamo sapere è che i termini ben tipizzati non si bloccano. Lo mostreremo in due passaggi, comunemente noti come i teoremi *progress* e *preservation*.

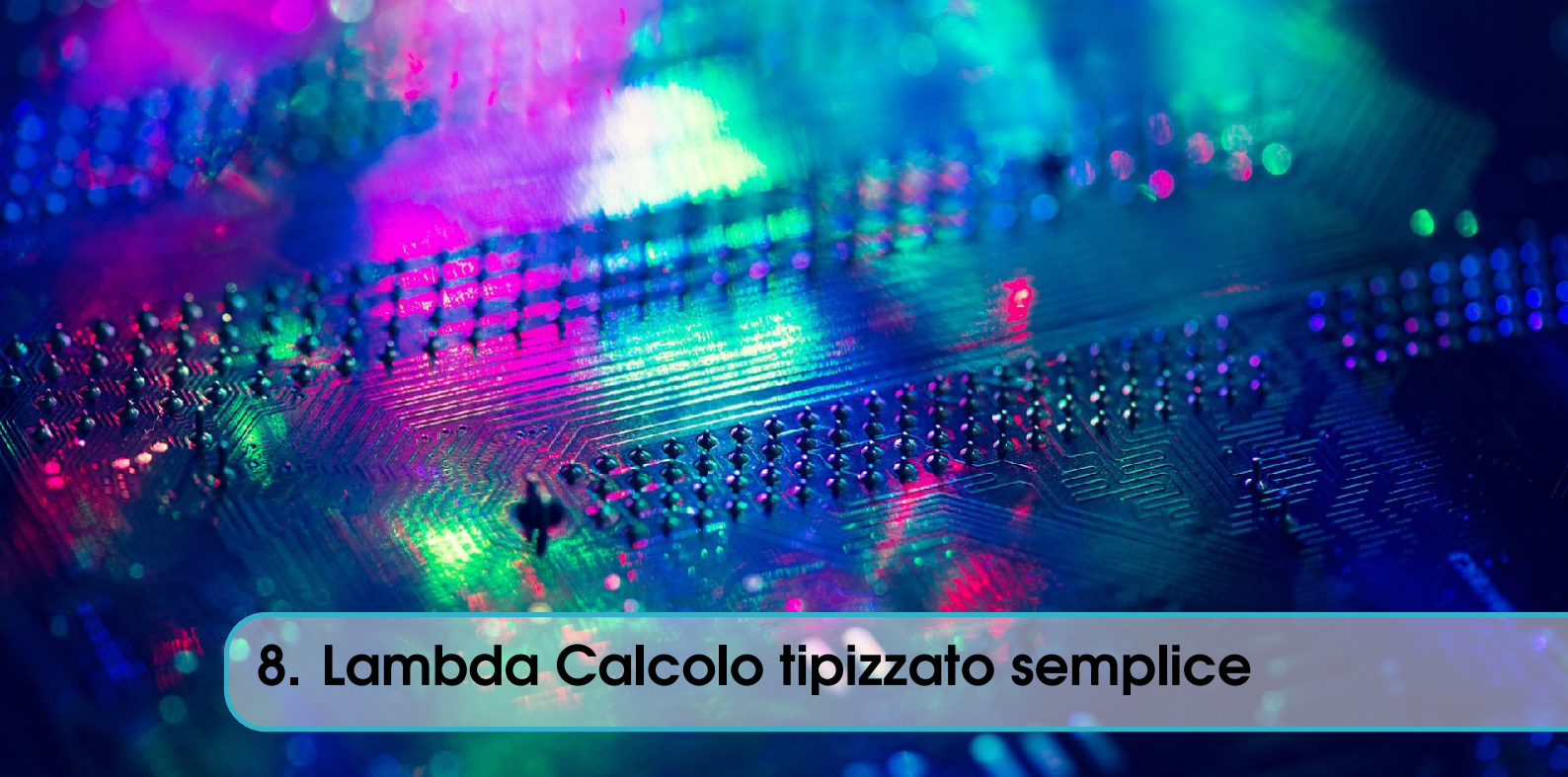
- **Progress:** Un termine ben tipizzato non può essere bloccato (o è un valore o si può effettuare ancora almeno un passo secondo le regole di valutazione).
- **Preservation:** Se viene effettuato un passo di valutazione su un termine ben tipizzato, allora il termine risultante è anche esso ben tipizzato.

Queste proprietà insieme ci dicono che un termine ben tipizzato non può mai raggiungere uno stato bloccato durante la valutazione.

**Teorema 7.3.1 — Progress.** Supponiamo che  $t$  sia un termine ben tipizzato (che è  $t : T$  per qualche  $T$ ). Allora  $t$  è un valore o altrimenti esiste un termine  $t'$  tale che  $t \rightarrow t'$ .

**Teorema 7.3.2 — Preservation.** Se  $t : T$  e  $t \rightarrow t'$ , then  $t' : T$ .





## 8. Lambda Calcolo tipizzato semplice

### 8.1 Tipi di funzione

Nel Capitolo 7 abbiamo introdotto un semplice sistema di tipi statici per le espressioni aritmetiche con due tipi: `Bool`, che classifica i termini la cui valutazione produce un valore booleano, e `Nat`, che classifica i termini la cui valutazione produce un numero. I termini “mal tipizzati” che non appartengono a nessuno di questi tipi includono tutti i termini che raggiungono stati bloccati (stuck) durante la valutazione (ad esempio, `if 0 then 1 else 2`) così come alcuni termini che effettivamente si comportano bene durante la valutazione, ma per i quali la classificazione statica è troppo prudente (come `if true then 0 else false`). Supponiamo di voler costruire un sistema di tipi simile per un linguaggio che combini booleani (per brevità, ignoreremo i numeri in questo capitolo) con le primitive del puro lambda-calcolo. Cioè, vogliamo introdurre regole di tipizzazione per **variabili, astrazioni e applicazioni** che

- mantengano la sicurezza del tipo, cioè soddisfino i teoremi di conservazione e progresso del tipo, 7.3.1 e 7.3.2, e
- non siano troppo conservativi. Cioè, dovrebbero assegnare tipi alla maggior parte dei programmi che ci interessa scrivere.

Naturalmente, poiché il puro lambda-calcolo è Turing completo, non c'è speranza di fornire un'analisi di tipo esatto per queste primitive. Ad esempio, non c'è modo di determinare in modo affidabile se un programma come

```
if <calcolo lungo e complicato> then true else ( $\lambda x.x$ )
```

restituisce un valore booleano o una funzione senza effettivamente eseguire il lungo e complicato calcolo e vedere se restituisce `true` o `false`. Ma, in generale, il calcolo lungo e complicato potrebbe anche **divergere**, e anche qualsiasi typechecker che tenti di prevedere con precisione il suo risultato divergerà.

Per estendere il sistema di tipi per i booleani in modo da includere funzioni, abbiamo chiaramente bisogno di aggiungere un tipo che classifichi i termini la cui valutazione risulta in una funzione. In prima approssimazione, chiamiamo questo tipo  $\rightarrow$ . Se aggiungiamo una regola di tipo

$$\lambda x. t : \rightarrow$$

assegnando ad ogni  $\lambda$ -astrazione il tipo  $\rightarrow$ , possiamo classificare sia termini semplici come  $\lambda x.x$

che termini composti come `if true then ( $\lambda x. \text{true}$ ) else ( $\lambda x. \lambda y. y$ )` come funzioni che restituiscono un risultato.

Ma questa analisi approssimativa è chiaramente troppo conservativa: funzioni come  $\lambda x. \text{true}$  e  $\lambda x. \lambda y. y$  sono raggruppate insieme nello stesso tipo  $\rightarrow$ , ignorando il fatto che applicando la prima a `true` si ottiene un valore booleano, mentre applicando la seconda a `true` si ottiene un'altra funzione. In generale, per dare un tipo utile al risultato di un'applicazione, abbiamo bisogno di saperne di più sul lato sinistro che non solo sul fatto che si tratta di una funzione: **abbiamo bisogno di sapere che tipo restituisce la funzione**. Inoltre, per essere sicuri che la funzione si comporti correttamente quando viene chiamata, dobbiamo tenere traccia del tipo di argomenti che si aspetta. Per tenere traccia di queste informazioni, sostituiamo il semplice tipo  $\rightarrow$  con una famiglia infinita di tipi della forma  $T_1 \rightarrow T_2$ , ciascuna delle quali classifica funzioni che si aspettano argomenti di tipo  $T_1$  e restituiscono risultati di tipo  $T_2$ .

**Definizione 8.1.1** L'insieme di tipi semplici sul tipo `Bool` è generato da seguente grammatica:

|                   |                       |
|-------------------|-----------------------|
| $T ::=$           | (tipi)                |
| <code>Bool</code> | (tipo dei booleani)   |
| $T \rightarrow T$ | (tipo delle funzioni) |

Il costruttore di tipo  $\rightarrow$  è associativo a destra, ovvero l'espressione  $T_1 \rightarrow T_2 \rightarrow T_3$  sta per  $T_1 \rightarrow (T_2 \rightarrow T_3)$ . (Come in Ocaml)

Ad esempio  $\text{Bool} \rightarrow \text{Bool}$  è il tipo di funzioni che associano argomenti booleani a risultati booleani.  $(\text{Bool} \rightarrow \text{Bool}) \rightarrow (\text{Bool} \rightarrow \text{Bool})$  o, equivalentemente,  $(\text{Bool} \rightarrow \text{Bool}) \rightarrow \text{Bool} \rightarrow \text{Bool}$ , è il tipo di funzioni che accettano funzioni da booleano a booleano come argomenti e le restituiscono come risultati.

## 8.2 La relazione di tipizzazione

Per assegnare un tipo a un'astrazione come  $\lambda x. t$ , dobbiamo calcolare cosa accadrà quando l'astrazione viene applicata a qualche argomento. La domanda successiva che si pone è: come facciamo a sapere che tipo di argomenti aspettarci? Ci sono due possibili risposte:

- possiamo semplicemente annotare la  $\lambda$ -astrazione con il tipo previsto dei suoi argomenti
- possiamo analizzare il corpo dell'astrazione per vedere come viene usato l'argomento e provare a dedurre, da questo, che tipo dovrebbe avere.

Per ora scegliamo la prima alternativa. Invece di solo  $\lambda x. t$ , scriveremo  $\lambda x: T_1. t_2$ , dove l'annotazione sulla variabile bound ci dice di assumere che l'argomento sarà di tipo  $T_1$ .

In generale, i linguaggi in cui le annotazioni di tipo nei termini vengono utilizzate per aiutare a guidare il correttore di caratteri sono chiamati **esplicitamente tipate** (*explicitly typed*). I linguaggi in cui chiediamo al typechecker di *inferire* o *ricostruire* queste informazioni sono chiamati **implicitamente tipati** (*implicitly typed*).<sup>1</sup> La maggior parte di questo libro si concentrerà sui linguaggi tipizzati in modo esplicito.

Una volta che conosciamo il tipo dell'argomento dell'astrazione, è chiaro che il tipo del risultato della funzione sarà proprio il tipo del corpo  $t_2$ , dove si presume che le occorrenze di  $x$  in  $t_2$  denotino termini di tipo  $T_1$ . Questa intuizione è catturata dalla seguente regola di tipizzazione:

$$\frac{x : T_1 \vdash t_2 : T_2}{\vdash \lambda x: T_1. t_2 : T_1 \rightarrow T_2} \quad (\text{T-ABS})$$

<sup>1</sup>Nella letteratura sul  $\lambda$ -calculus, viene usato anche il termine *sistemi di assegnazione del tipo*.

Poiché i termini possono contenere  $\lambda$ -astrazioni nidificate, avremo bisogno, in generale, di parlare di molte di queste assunzioni. Ciò modifica la relazione di tipizzazione da una relazione a due posizioni,  $t : T$ , a una relazione a tre posizioni,  $\Gamma \vdash t : T$ , dove  $\Gamma$  (Gamma) è un insieme di ipotesi sui tipi delle variabili libere in  $t$ .

**Definizione 8.2.1 — Contesto di tipizzazione.** Formalmente, un **contesto di tipizzazione** (chiamato anche **ambiente di tipi**)  $\Gamma$  è una sequenza di variabili e dei loro tipi, e l'operatore "virgola" estende  $\Gamma$  aggiungendo un nuovo legame a destra. Il contesto vuoto a volte è scritto  $\emptyset$ , ma di solito lo omettiamo semplicemente, scrivendo  $\vdash t : T$  per intendere "Il termine chiuso  $t$  ha il tipo  $T$  sotto l'insieme vuoto di ipotesi".



Il simbolo *turnstile*  $\vdash$  ha diverse interpretazioni. Nel lambda-calcolo tipato, il turnstile è usato per separare le assunzioni di tipo dai giudizi di tipo (typing judgment).

Per evitare confusione tra il nuovo legame e qualsiasi legame che potrebbe già apparire in  $\Gamma$ , richiediamo che il nome  $x$  sia scelto in modo che sia distinto dalle variabili legate da  $\Gamma$ . Poiché la nostra convenzione è che le variabili vincolate da  $\lambda$ -astrazioni possono essere rinominate ogni volta che è conveniente, questa condizione può sempre essere soddisfatta rinominando la variabile vincolata se necessario.  $\Gamma$  può quindi essere pensato come una funzione finita dalle variabili ai loro tipi. Seguendo questa intuizione, definiamo  $\text{dom}(\Gamma)$  l'insieme delle variabili vincolate da  $\Gamma$ . La regola per tipizzare le astrazioni ha la forma generale

$$\frac{\Gamma, x : T_1 \vdash t_2 : T_2}{\Gamma \vdash \lambda x:T_1. t_2:T_1 \rightarrow T_2} \quad (\text{T-ABS})$$

dove la premessa aggiunge un'altra ipotesi a quelle della conclusione.

Anche la regola di tipizzazione per le variabili segue immediatamente da questa discussione: *una variabile ha qualunque tipo stiamo attualmente supponendo che abbia*.

$$\frac{x \in \Gamma}{\Gamma \vdash x : T} \quad (\text{T-VAR})$$

in cui la premessa  $x \in \Gamma$  si legge "Il tipo assunto per  $x$  in  $\Gamma$  è  $T$ ."

Infine, abbiamo bisogno di una regola di tipizzazione per le applicazioni.

$$\frac{\Gamma \vdash t_1 : T_{11} \rightarrow T_{12} \quad \Gamma \vdash t_2 : T_{11}}{\Gamma \vdash t_1 t_2 : T_{12}} \quad (\text{T-APP})$$

Se  $t_1$  restituisce una funzione mappando gli argomenti in  $T_{11}$  ai risultati in  $T_{12}$  (assumendo che i valori rappresentati dalle sue variabili libere abbiano i tipi assunti per essi in  $\Gamma$ ), e se  $t_2$  restituisce un risultato in  $T_{11}$ , allora il risultato dell'applicazione di  $t_1$  a  $t_2$  sarà un valore di tipo  $T_{12}$ . Le regole di tipizzazione per le costanti booleane e le espressioni condizionali sono le stesse di prima (Figura 7.1, dobbiamo aggiungere un contesto  $\Gamma$  per ogni tipo nella figura).

Si noti, tuttavia, che la metavariable  $T$  nella regola per i condizionali

$$\frac{\Gamma \vdash t_1 : \text{Bool} \quad \Gamma \vdash t_2 : T \quad \Gamma \vdash t_3 : T}{\Gamma \vdash \text{if } t_1 \text{ then } t_2 \text{ else } t_3 : T} \quad (\text{T-IF})$$

ora può essere istanziata a qualsiasi tipo di funzione, permettendoci di tipizzare condizionali i cui rami sono funzioni:



Queste regole di digitazione sono riassunte nella Figura 8.1 (insieme alla sintassi e alle regole di valutazione, per completezza). Le regioni evidenziate nella figura indicano materiale nuovo rispetto al lambda-calcolo non tipizzato, sia nuove regole che nuovi bit aggiunti a vecchie regole.

Come abbiamo fatto con booleani e numeri, abbiamo diviso la definizione del calcolo completo in due parti: il puro lambda-calcolo tipizzato semplice senza alcun tipo di base, mostrato in questa figura, e un insieme separato di regole per booleani, che abbiamo già visto nella Figura 7.1 (dobbiamo aggiungere un contesto  $\Gamma$  a ogni istruzione di tipizzazione in quella figura, ovviamente). Usiamo spesso il simbolo  $\lambda_{\rightarrow}$  per riferirci al lambda-calcolo semplicemente tipizzato (usiamo lo stesso simbolo per sistemi con diversi insiemi di tipi di base).

Figura 8.1: Lambda-calcolo tipizzato semplice ( $\lambda_{\rightarrow}$ )

Le istanze delle regole di tipizzazione per  $\lambda_{\rightarrow}$  possono essere combinate in alberi di derivazione, proprio come abbiamo fatto per le espressioni aritmetiche tipizzate.

■ **Esempio 8.1** Ad esempio, ecco una derivazione che dimostra che il termine  $(\lambda x.\text{Bool}.x)$  true ha il tipo `Bool` nel contesto vuoto.

$$\frac{\frac{\frac{x:\text{Bool} \in x:\text{Bool}}{x:\text{Bool} \vdash x:\text{Bool}} \text{ T-VAR} \quad \frac{}{\emptyset \vdash \lambda x:\text{Bool}.x : \text{Bool} \rightarrow \text{Bool}} \text{ T-ABS} \quad \frac{}{\emptyset \vdash \text{true} : \text{Bool}} \text{ T-TRUE}}{\emptyset \vdash (\lambda x:\text{Bool}.x)\text{true} : \text{Bool}} \text{ T-APP}$$

### 8.3 Vaneggi del prof

In questa parte userò le formule del prof e proverò a interpretarle.

Viene definito un insieme di termini

$$t ::= \text{True} \mid \text{False} \mid \text{if } t_0 \text{ then } t_1 \text{ else } t_2 \mid t_1 \text{ and } t_2 \mid t_1 \text{ or } t_2 \mid \text{not } t \mid \\ \text{Zero} \mid \text{Succ } t \mid \text{Pred } t \mid \text{IsZero } t \mid \text{fun } x:T. t \mid \text{apply}(t_1, t_2) \mid X$$

In cui  $X$  è un identificatore arbitrario,  $X \in Var$ . Viene definito un insieme di tipi

$$T ::= \text{Bool} \mid \text{Nat} \mid T \rightarrow T$$

Viene poi definito cosa è un contesto, ossia un insieme di associazioni tra identificatori e tipi

$$\Gamma : Var \rightarrow T \quad (8.1)$$

dove  $\Gamma$  è  $\Gamma \subseteq Var \times T$ . In cui  $\Gamma$  è un contesto,  $\Gamma(x)$  è il tipo associato all'identificatore  $x$  e  $\Gamma \vdash t : T$  è la tipizzazione di  $t$  nel contesto  $\Gamma$  con tipo  $T$ .

Vengono poi definite le regole di tipizzazione

$$\begin{array}{c} \frac{}{\Gamma \vdash \text{True} : \text{Bool}} \text{T-TRUE} \quad \frac{}{\Gamma \vdash \text{False} : \text{Bool}} \text{T-FALSE} \\ \frac{\Gamma \vdash t_0 : \text{Bool} \quad \Gamma \vdash t_1 : T \quad \Gamma \vdash t_2 : T}{\Gamma \vdash \text{if } t_0 \text{ then } t_1 \text{ else } t_2 : T} \text{T-IF} \quad \frac{\Gamma \vdash t_1 : \text{Bool} \quad \Gamma \vdash t_2 : \text{Bool}}{\Gamma \vdash t_1 \text{ and } t_2 : \text{Bool}} \text{T-AND} \\ \frac{\Gamma \vdash t_1 : \text{Bool} \quad \Gamma \vdash t_2 : \text{Bool}}{\Gamma \vdash t_1 \text{ or } t_2 : \text{Bool}} \text{T-OR} \quad \frac{\Gamma \vdash t : \text{Bool}}{\Gamma \vdash \text{not } t : \text{Bool}} \text{T-NOT} \\ \frac{}{\Gamma \vdash \text{Zero} : \text{Nat}} \text{T-ZERO} \quad \frac{\Gamma \vdash t : \text{Nat}}{\Gamma \vdash \text{Succ } t : \text{Nat}} \text{T-SUCC} \quad \frac{\Gamma \vdash t : \text{Nat}}{\Gamma \vdash \text{Pred } t : \text{Nat}} \text{T-PRED} \\ \frac{\Gamma \vdash t : \text{Nat}}{\Gamma \vdash \text{IsZero } t : \text{Bool}} \text{T-ISZERO} \quad \frac{\Gamma \cup \{(x, T)\} \vdash t : T'}{\Gamma \vdash \text{fun } x:T. t : T \rightarrow T'} \text{T-FUN} \\ \frac{\Gamma \vdash t_1 : T_1 \rightarrow T_2 \quad \Gamma \vdash t_2 : T_1}{\Gamma \vdash \text{apply}(t_1, t_2) : T_2} \text{T-APPLY} \quad \frac{\Gamma(x) = T}{\Gamma \vdash x : T} \text{T-VAR} \quad \frac{(x, T) \in \Gamma}{\Gamma \vdash x : T} \text{T-VAR} \end{array}$$

■ **Esempio 8.2** Ad esempio, il termine  $\text{fun } x:\text{Nat}. \text{Succ } x$  ha tipo  $\text{Nat} \rightarrow \text{Nat}$ , questo è verificabile come segue:

$$\frac{\Gamma \cup \{(x, \text{Nat})\} \vdash \text{Succ } x : \dots}{\Gamma \vdash \text{fun } x:\text{Nat}. \text{Succ } x : \text{Nat} \rightarrow \dots} \text{T-FUN}$$

dove  $\Gamma \cup \{(x, \text{Nat})\}$  è il contesto ottenuto aggiungendo l'associazione  $(x, \text{Nat})$  al contesto  $\Gamma$ . La verifica non è terminata, bisogna verificare il tipo di  $\text{Succ } x$  come segue:

$$\frac{\Gamma \cup \{(x, \text{Nat})\} \vdash x : \dots}{\Gamma \cup \{(x, \text{Nat})\} \vdash \text{Succ } x : \dots} \text{T-SUCC} \\ \frac{\Gamma \cup \{(x, \text{Nat})\} \vdash \text{Succ } x : \text{Nat}}{\Gamma \vdash \text{fun } x:\text{Nat}. \text{Succ } x : \text{Nat} \rightarrow \dots} \text{T-FUN}$$

Infine ci resta da verificare il tipo di  $x$ :

$$\frac{\Gamma \cup \{(x, \text{Nat})\} \vdash x = \text{Nat}}{\Gamma \cup \{(x, \text{Nat})\} \vdash x : \text{Nat}} \text{T-VAR} \\ \frac{\Gamma \cup \{(x, \text{Nat})\} \vdash x : \text{Nat}}{\Gamma \cup \{(x, \text{Nat})\} \vdash \text{Succ } x : \text{Nat}} \text{T-SUCC} \\ \frac{\Gamma \cup \{(x, \text{Nat})\} \vdash \text{Succ } x : \text{Nat}}{\Gamma \vdash \text{fun } x:\text{Nat}. \text{Succ } x : \text{Nat} \rightarrow \text{Nat}} \text{T-FUN}$$

Il contesto  $\Gamma \cup \{(x, \text{Nat})\}$  contiene l'associazione  $(x, \text{Nat})$ , quindi il tipo di  $x$  è  $\text{Nat}$  e la funzione restituisce  $\text{Nat}$ .

■ **Esempio 8.3** Un altro esempio è il termine  $\text{fun } x:\text{Nat}. \text{IsZero}(x)$  che ha tipo  $\text{Nat} \rightarrow \text{Bool}$ , questo è verificabile come segue:

$$\frac{\perp\{\text{Nat}/x\} \vdash \text{IsZero}(x) : \dots}{\perp \vdash \text{fun } x:\text{Nat}. \text{IsZero}(x) : \text{Nat} \rightarrow \dots} \text{T-FUN}$$

dove  $\perp\{\text{Nat}/x\}$  è il contesto ottenuto aggiungendo l'associazione  $\text{Nat}/x$  al contesto  $\perp$ . Questa volta è stata usata una sintassi alternativa, ma equivalente a quella usata nel precedente esempio. La verifica non è terminata, bisogna verificare il tipo di  $\text{IsZero}(x)$  come segue:

$$\frac{\frac{\perp\{\text{Nat}/x\} \vdash x : \dots}{\perp\{\text{Nat}/x\} \vdash \text{IsZero}(x) : \dots} \text{T-ISZERO}}{\perp \vdash \text{fun } x:\text{Nat}. \text{IsZero}(x) : \text{Nat} \rightarrow \dots} \text{T-FUN}$$

Infine ci resta da verificare il tipo di  $x$ :

$$\frac{\frac{\frac{\perp\{\text{Nat}/x\}(x) = \text{Nat}}{\perp\{\text{Nat}/x\} \vdash x : \text{Nat}} \text{T-VAR}}{\perp\{\text{Nat}/x\} \vdash \text{IsZero}(x) : \text{Bool}} \text{T-ISZERO}}{\perp \vdash \text{fun } x:\text{Nat}. \text{IsZero}(x) : \text{Nat} \rightarrow \text{Bool}} \text{T-FUN}$$

Il contesto  $\perp\{\text{Nat}/x\}$  contiene l'associazione  $\text{Nat}/x$ , quindi il tipo di  $x$  è  $\text{Nat}$  e la funzione restituisce  $\text{Bool}$ .

Dalle regole definite prima si possono effettuare alcune osservazioni:

- Se  $\Gamma \vdash t:T$  e  $t \rightarrow v$  allora  $\Gamma \vdash v : T$ . Questo vuol dire che se un termine è di un tipo  $T$  e se il termine può essere valutato in un valore, allora il valore è di tipo  $T$ . Questa proprietà è chiamata *preservazione del tipo*.
- Se  $t \rightarrow v$  e  $\Gamma \vdash v : T$  allora  $\Gamma \vdash t : T$ . Questo vuol dire che se un termine può essere valutato in un valore di tipo  $T$ , allora il termine è di tipo  $T$ . Questa proprietà è chiamata *progresso del tipo*.



## 9. Linguaggio imperativo

### 9.1 Lo stato

Per poter formalizzare la semantica operativa di un linguaggio di programmazione imperativo la prima nozione da introdurre è quella di **stato**. Dato un programma  $p$ , con  $\text{Name}(p)$  indichiamo l'insieme dei nomi (identificatori) che compaiono nel programma. Saranno quindi tutti gli **identificatori** come i nomi di variabili o simili. Non ne diamo una definizione formale. I valori che possono essere associati ai vari nomi li indichiamo con  $\text{Value}$  e non dipendono dal programma medesimo, e anche in questo caso non ne diamo una definizione formale.

**Definizione 9.1.1 — Stato.** Dato un programma  $p$ , lo stato è una funzione (parziale) da  $\text{Name}(p)$  a  $\text{Value}$ .

La nozione di stato dipende anche molto dal paradigma che consideriamo. Ad esempio, nel **paradigma imperativo** e in quello **procedurale**, vedremo che lo stato:

- ha una parte che chiameremo **ambiente** (una funzione da nomi ad oggetti che chiameremo denotabili);
- una **memoria**, che sarà una funzione da locazioni (che assumeremo denotabili) a oggetti memorizzabili;
- uno **heap**, che sarà una struttura dati per la memorizzazione delle strutture dati dinamiche

Mentre nel **paradigma funzionale** avremo **solo l'ambiente**, cioè l'associazione tra nomi e oggetti denotabili.

Prima di introdurre un qualche linguaggio e di definirne la semantica, bisogna stabilire cosa fa una semantica operativa. Ovviamente questo dipende dal linguaggio e dal paradigma a cui tale linguaggio appartiene. Nel caso di linguaggi **imperativi** la semantica operativa definisce un **cambiamento di stato**, mentre nei linguaggi **funzionali** la semantica operativa restituisce il **risultato della valutazione**, quindi un valore.

### 9.2 Il cuore di un linguaggio imperativo: IMP

Vediamo il nucleo di un linguaggio **imperativo**, che chiamiamo IMP. Il linguaggio IMP che viene considerato ha un insieme di identificatori (che prima abbiamo chiamato  $\text{Name}$ ) e che ora

chiamiamo  $Var$  mentre i valori in  $Value$  saranno i numeri interi e i booleani (almeno per ora). Con  $\mathbb{B}$  indichiamo l'insieme dei valori booleani e con  $\mathbb{Z}$  l'insieme dei numeri interi. Lo stato lo vediamo come una funzione da  $Var$  a  $\mathbb{B} \cup \mathbb{Z} \cup \{\perp\}$  dove  $\perp$  indica che ad un certo identificatore non è associato alcun valore. Quindi l'insieme degli stati è

$$\Sigma = \{\sigma \mid \sigma : Var \rightarrow \mathbb{B} \cup \mathbb{Z} \cup \{\perp\}\}$$

Per dire che, dato un nome  $X$  ed uno stato  $\sigma$ , al nome  $X$  è associato un valore, scriviamo  $\sigma(X) \downarrow$ .

### 9.2.1 Espressioni

Diamo prima le regole per la valutazione delle espressioni di un linguaggio imperativo. La sintassi di queste espressioni è

$$e ::= true \mid false \mid n \mid X \mid e_0 + e_1 \mid e_0 \times e_1 \mid e_0 - e_1 \mid e_0 \wedge e_1 \mid e_0 \vee e_1 \mid \neg e \mid e_0 < e_1$$

dove  $X \in Var$  sono le variabili.

Per valutare un'espressione abbiamo bisogno dello stato, che abbiamo detto indichiamo con  $\sigma$ . Le regole sono:

$$\begin{array}{c} \frac{}{\langle n, \sigma \rangle \longrightarrow n} \quad \frac{\sigma(X) \downarrow}{\langle X, \sigma \rangle \longrightarrow \sigma(X)} \quad \frac{\langle e_0, \sigma \rangle \longrightarrow n_0 \quad \langle e_1, \sigma \rangle \longrightarrow n_1}{\langle e_0 + e_1, \sigma \rangle \longrightarrow n = n_0 + n_1} \\ \\ \frac{\langle e_0, \sigma \rangle \longrightarrow n_0 \quad \langle e_1, \sigma \rangle \longrightarrow n_1}{\langle e_0 \times e_1, \sigma \rangle \longrightarrow n = n_0 \times n_1} \quad \frac{\langle e_0, \sigma \rangle \longrightarrow n_0 \quad \langle e_1, \sigma \rangle \longrightarrow n_1}{\langle e_0 - e_1, \sigma \rangle \longrightarrow n = n_0 - n_1} \\ \\ \frac{}{\langle true, \sigma \rangle \longrightarrow true} \quad \frac{}{\langle false, \sigma \rangle \longrightarrow false} \quad \frac{\langle a, \sigma \rangle \longrightarrow t}{\langle \neg a, \sigma \rangle \longrightarrow \neg t} \\ \\ \frac{\langle b_0, \sigma \rangle \longrightarrow t_0 \quad \langle b_1, \sigma \rangle \longrightarrow t_1}{\langle a_0 \vee a_1, \sigma \rangle \longrightarrow t_0 \vee t_1} \quad \frac{\langle a_0, \sigma \rangle \longrightarrow a_0 \quad \langle b_1, \sigma \rangle \longrightarrow t_1}{\langle a_0 \wedge a_1, \sigma \rangle \longrightarrow t_0 \wedge t_1} \\ \\ \frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle a_0 < a_1, \sigma \rangle \longrightarrow n_0 < n_1} \end{array}$$

### 9.2.2 Comandi

Vediamo ora la sintassi dei comandi di un semplice linguaggio imperativo:

$$c ::= \text{skip} \mid X := a \mid c_0; c_1 \mid \text{if } e \text{ then } c_0 \text{ else } c_1 \mid \text{while } e \text{ do } c$$

I comandi usano anche le variabili e le espressioni.

Per poterne dare la semantica dobbiamo prima capire cosa fa un comando in generale. La risposta, come in questi casi capita sempre, è ovvia: un comando modifica lo stato, e nel nostro caso lo stato non sarà altro che un'associazione tra i nomi, che qui sono le variabili, ed i valori associabili alle variabili, che qui sono gli interi e i booleani.

Lo stato lo aggiorniamo quando assegniamo un valore ad una variabile. La notazione è la seguente:  $\sigma\{v/X\}$  che significa che abbiamo assegnato  $v$  alla variabile  $X$ .

$\sigma\{v/X\}$  è la funzione dalle variabili  $Var$  ai valori  $\mathbb{Z} \cup \mathbb{B}$  che

$$\sigma\{v/X\}(Y) = \begin{cases} v & \text{se } X = Y \\ \sigma(Y) & \text{altrimenti} \end{cases}$$

Vediamo ora la *big step* semantics dei comandi. La relazione la indichiamo con  $\Longrightarrow_{bs}$  dove il *bs* indica *big step*.

- **skip.** Questo comando non fa nulla, lascia invariato lo stato. La sua semantica può essere data con la regola

$$\frac{}{\langle \text{skip}, \sigma \rangle \Longrightarrow_{bs} \sigma}$$

- **L'assegnamento:** il comando  $X := e$  modifica lo stato assegnando alla variabile  $X$  il risultato della valutazione dell'espressione  $e$ .

$$\frac{\langle e, \sigma \rangle \longrightarrow v}{\langle X := e, \sigma \rangle \Longrightarrow_{bs} \sigma\{v/X\}}$$

- **il ';',** cioè quello che stabilisce la sequenza di due comandi. La regola è ovvia: devo eseguire il secondo nello stato ottenuto eseguendo il primo

$$\frac{\langle c_0, \sigma \rangle \Longrightarrow_{bs} \sigma'' \quad \langle c_1, \sigma'' \rangle \Longrightarrow_{bs} \sigma'}{\langle c_0; c_1, \sigma \rangle \Longrightarrow_{bs} \sigma'}$$

- **il condizionale (If then else).** In questo caso ho bisogno di due regole, dato che devo capire, in base al valore dell'espressione  $e$ , quale dei due comandi eseguire.

$$\frac{\langle e, \sigma \rangle \longrightarrow \text{true} \quad \langle c_0, \sigma \rangle \Longrightarrow_{bs} \sigma'}{\langle \text{if } e \text{ then } c_0 \text{ else } c_1, \sigma \rangle \Longrightarrow_{bs} \sigma'}$$

e

$$\frac{\langle e, \sigma \rangle \longrightarrow \text{false} \quad \langle c_1, \sigma \rangle \Longrightarrow_{bs} \sigma'}{\langle \text{if } e \text{ then } c_0 \text{ else } c_1, \sigma \rangle \Longrightarrow_{bs} \sigma'}$$

- **l'iterazione (il while).** Anche in questo caso ho due regole. Se la condizione  $e$  è falsa, allora non devo fare nulla, se invece è vera devo prima eseguire il comando  $c$  e, nello stato ottenuto, eseguire il costrutto di iterazione, cioè il while.

$$\frac{\langle b, \sigma \rangle \longrightarrow \text{false}}{\langle \text{while } b \text{ do } c, \sigma \rangle \Longrightarrow_{bs} \sigma}$$

e

$$\frac{\langle b, \sigma \rangle \longrightarrow \text{true} \quad \langle c, \sigma \rangle \Longrightarrow_{bs} \sigma'' \quad \langle \text{while } b \text{ do } c, \sigma'' \rangle \Longrightarrow_{bs} \sigma'}{\langle \text{while } b \text{ do } c, \sigma \rangle \Longrightarrow_{bs} \sigma'}$$

Le regole della semantica operativa ci sono servite per definire due relazioni:

- $\longrightarrow \subseteq \text{Exp} \times \Sigma \times \mathbb{Z}$
- $\Longrightarrow_{bs} \subseteq \text{Com} \times \Sigma \Sigma$

dove *Exp* sono le espressioni e *Com* sono i comandi del nostro linguaggio IMP.

Queste relazioni sono **deterministiche** nel senso che, fissato un elemento in *Exp* o *Com* ed uno stato, ogni relazione contiene una sola tripla che ha esattamente quei due primi elementi, quindi sostanzialmente definiscono delle funzioni.

**Teorema 9.2.1** Sia  $e$  un'espressione e  $\sigma$  uno stato, allora se  $\langle e, \sigma \rangle \longrightarrow v$  e  $\langle e, \sigma \rangle \Longrightarrow_{bs} v'$  abbiamo che  $v = v'$ .

Sia  $c$  un comando e  $\sigma$  uno stato, allora se  $\langle c, \sigma \rangle \Longrightarrow_{bs} \sigma'$  e  $\langle c, \sigma \rangle \Longrightarrow_{bs} \sigma''$  abbiamo che  $\sigma' = \sigma''$

La conseguenza di questo teorema è che le due relazioni definiscono delle funzioni. In particolare la relazione  $\longrightarrow$  definisce la funzione

$$\mathcal{E} : \text{Exp} \times \Sigma \rightarrow \mathbb{Z} \cup \mathbb{B}$$

e la relazione  $\Longrightarrow_{bs}$  definisce la funzione

$$\mathcal{C} : \text{Com} \times \Sigma \rightarrow \Sigma$$

La semantica ci aiuta anche a dire quando due costrutti sono *equivalenti*. Dato che a noi nel linguaggio imperativo interessano i comandi, formalizziamo cosa significa che due comandi sono equivalenti.

**Definizione 9.2.1 — Equivalenza di comandi.** Due comandi  $c, c' \in \text{Com}$  sono equivalenti, scritto come  $c \sim c'$ , se e solo se per ogni stato  $\sigma \in \Sigma$  vale che  $\mathcal{C}(c, \sigma) = \mathcal{C}(c', \sigma)$ .

$$c \sim c' \Leftrightarrow \forall \sigma \in \Sigma. \mathcal{C}(c, \sigma) = \mathcal{C}(c', \sigma)$$

Va osservato che la definizione di equivalenza può essere usata in due modi. Se una parte dalla semantica, la definizione serve a caratterizzare quali sono i costrutti tra di loro equivalenti, se si parte invece dalla *richiesta* che certi costrutti siano equivalenti allora possiamo costruire la semantica in modo che tale equivalenza venga rispettata.

■ **Esempio 9.1** Ad esempio, grazie e a questa definizione abbiamo che

$$\text{while } e \text{ do } c \sim \text{if } e \text{ then } (c; \text{while } e \text{ do } c) \text{ else skip}$$

In questa semantica operativa abbiamo posto l'accento sulla trasformazione di stato, immaginando che tutti i comandi vengano eseguiti *integralmente*, dove integralmente significa che l'esecuzione del comando restituisce uno stato e non una coppia (costrutto, stato), prima di passare allo stato successivo. Potremmo però porre l'attenzione anche su cosa dobbiamo ancora eseguire.

Di seguito le regole modificate tenendo conto di voler sottolineare, nella semantica, proprio questo, diamo quindi la *small step* semantics. Formalmente definiamo due relazioni, una ( $\Longrightarrow_{ss}$ , dove *ss* sta per *small step*) definisce una relazione che *restituisce* uno stato, mentre l'altra ( $\rightsquigarrow$ ) restituisce un comando ed uno stato. Quindi

$$\Longrightarrow_{ss} \subseteq \text{Com} \times \Sigma \times \Sigma$$

e

$$\rightsquigarrow \subseteq \text{Com} \times \Sigma \times \text{Com} \times \Sigma$$

$$\begin{array}{c} \frac{}{\langle \text{skip}, \sigma \rangle \Longrightarrow_{ss} \sigma} \quad \frac{\langle a, \sigma \rangle \rightarrow v}{\langle X := a, \sigma \rangle \Longrightarrow_{ss} \sigma\{v/X\}} \\[10pt] \frac{\langle c_0, \sigma \rangle \Longrightarrow_{ss} \sigma'}{\langle c_0; c_1, \sigma \rangle \rightsquigarrow \langle c_1, \sigma' \rangle} \quad \frac{\langle c_0, \sigma \rangle \rightsquigarrow \langle c'_0, \sigma' \rangle}{\langle c_0; c_1, \sigma \rangle \rightsquigarrow \langle c'_0; c_1, \sigma' \rangle} \\[10pt] \frac{\langle e, \sigma \rangle \rightarrow \text{true}}{\langle \text{if } e \text{ then } c_0 \text{ else } c_1, \sigma \rangle \rightsquigarrow \langle c_0, \sigma \rangle} \quad \frac{\langle e, \sigma \rangle \rightarrow \text{false}}{\langle \text{if } e \text{ then } c_0 \text{ else } c_1, \sigma \rangle \rightsquigarrow \langle c_1, \sigma \rangle} \\[10pt] \frac{\langle e, \sigma \rangle \rightarrow \text{true}}{\langle \text{while } e \text{ do } c, \sigma \rangle \rightsquigarrow \langle c; \text{while } e \text{ do } c, \sigma \rangle} \quad \frac{\langle e, \sigma \rangle \rightarrow \text{false}}{\langle \text{while } e \text{ do } c, \sigma \rangle \Longrightarrow_{ss} \sigma} \end{array}$$

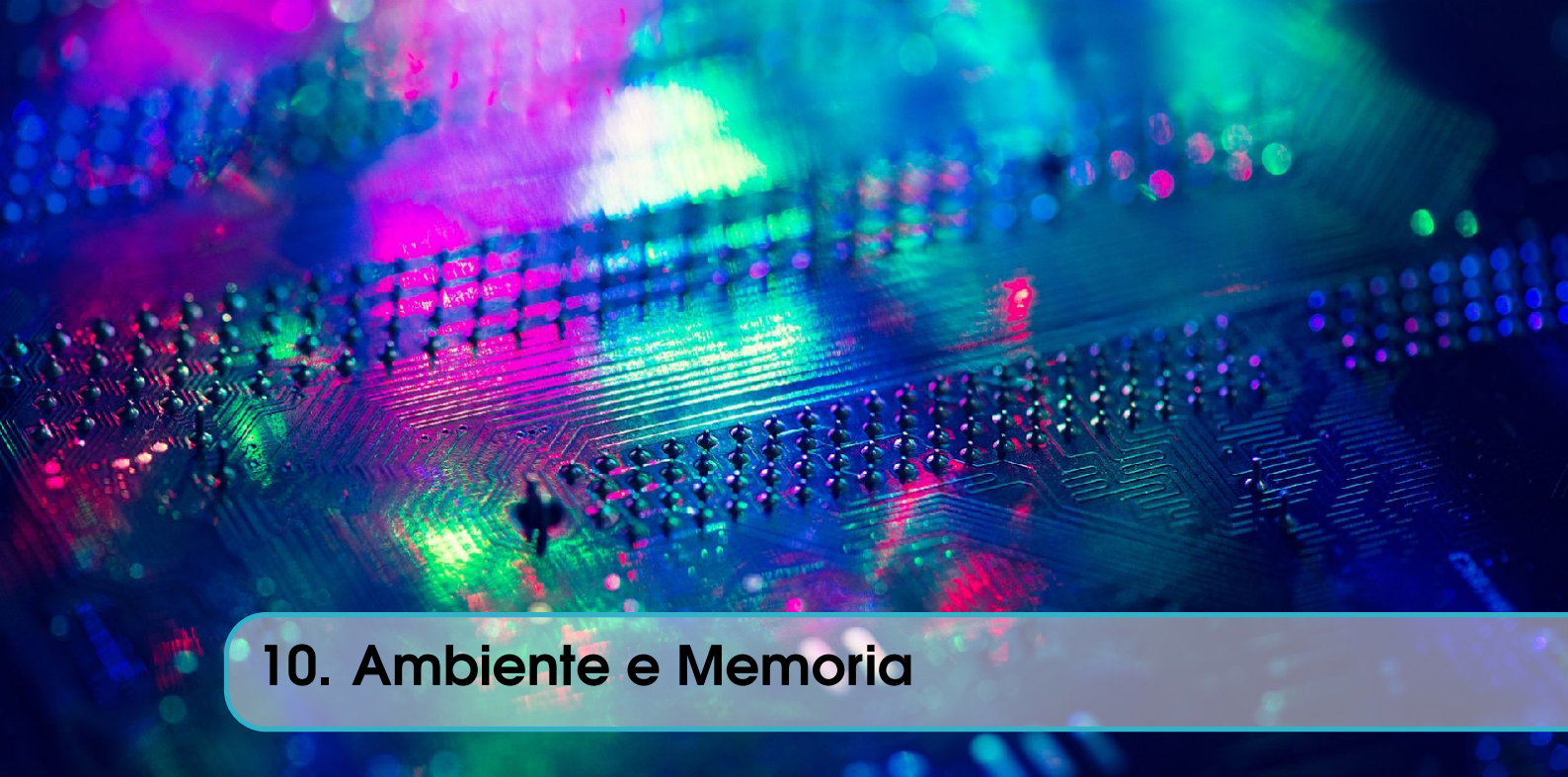


La relazione che a noi interessa è sempre  $\Longrightarrow_{ss} \subseteq Com \times \Sigma \times \Sigma$  dato che siamo sempre interessati a quale è la modifica dello stato determinata dal comando  $c$ , mentre la relazione  $\rightsquigarrow \subseteq Com \times \Sigma \times Com \times \Sigma$  è una relazione *ausiliaria* che serve solo per definire  $\Longrightarrow_{ss}$ .

Le due semantiche, *big step* e *small step*, sono equivalenti.

**Teorema 9.2.2** Per ogni comando  $c$ , per ogni stato  $\sigma$ , abbiamo che  $\langle c, \sigma \rangle \Longrightarrow_{bs} \sigma'$  se e solo se  $\langle c, \sigma \rangle \Longrightarrow_{ss} \sigma'$ .





## 10. Ambiente e Memoria

### 10.1 Ambiente

La nozione di stato che abbiamo usato fino ad ora non è in grado di modellare una vera nozione di astrazione funzionale che è particolarmente rilevante nei linguaggi di programmazione, come vedremo in seguito. Abbiamo quindi bisogno di modellare lo stato in modo diverso.

In un linguaggio di programmazione abbiamo, in generale, dei *nomi* che distinguiamo dalle *parole chiave* (keywords) che sono proprie del linguaggio. Consideriamo il seguente frammento di programma

```
int pippo (x : int) {  
    while (x > 10) {  
        x = x - 2;  
    }  
}
```

In questo frammento di programma ci sono delle *keywords*, ad esempio `int`, `while`, `';`, `'`, `do`, `:=` ma ci sono anche degli *identificatori* come `pippo` e `x` oppure altre sequenze di caratteri come `10` e `2` che indicano oggetti a cui non bisogna dare particolare interpretazione. La grammatica con cui vengono generati i nomi non ci interessa particolarmente, anche se ci possono essere delle regole particolari per capire cosa è associato a quel nome. Un nome lo si usa per identificare un qualche oggetto (valore o altro) che altrimenti non sapremmo identificare, quindi immaginiamo ci siano anche degli oggetti che chiameremo in generale *valori*.

Diamo prima una definizione di insieme dei nomi di un linguaggio.

**Definizione 10.1.1 — Nomi di un linguaggio.** Dato un linguaggio di programmazione  $\mathcal{L}$ , con  $\text{Name}_{\mathcal{L}}$  indichiamo l'insieme dei nomi di quel linguaggio.

Questa definizione ci servirà in seguito.

■ **Esempio 10.1** Facciamo alcuni esempi:

- `var pippo`: la sequenza di caratteri in rosso denota una variabile mentre la sequenza di caratteri in blu è una keyword del linguaggio

- `const pi = 3.14`: la sequenza di caratteri in rosso denota la costante 3.14
- `int fib`: la sequenza di caratteri in rosso denota una funzione

Nei linguaggi i nomi sono spesso *identificatori* (token alfa-numeric), sono nomi anche le *keywords* o simboli come `+`, `:`, `=`, `++`, ecc., che non sono identificatori ma fanno lo stesso parte del linguaggio.

Un nome definito dal programmatore serve ad indicare in generale un oggetto, che è l'oggetto *denotato*, cioè **associato a quel nome**. I nomi sono gli oggetti simbolici **più facili da ricordare** (buona regola di programmazione) e, particolarmente importante, **permettono l'astrazione** (sia sui dati che sul controllo). Ricordiamo che astrarre significa spesso dimenticare dettagli non importanti, che però in qualche forma sono presenti. Il non importanti significa che su quelli non ci concentriamo, mentre ci concentriamo sull'uso che si fa dell'astrazione. Ma astrarre consente anche di definire nuove funzioni o procedure.

Dato che abbiamo detto che a certi nomi (quelli definiti dal programmatore) vogliamo associare un oggetto, che chiamiamo denotabile, cerchiamo di formalizzare cosa è un **oggetto denotabile**. Diamone prima una definizione molto generale.

**Definizione 10.1.2 — Oggetti denotabili.** Un oggetto denotabile è un oggetto a cui possiamo dare un nome.

Osserviamo che non necessariamente diamo nome a tutto, ad esempio, in generale, non diamo nomi alla memoria.

Ci sono i nomi definiti dall'utente come le *variabili*, *parametri formali*, le *procedure* o *funzioni*, i *tipi* definiti dall'utente, le *etichette*, i *moduli*, le *costanti* definite dall'utente e simili, mentre ci sono i nomi definiti dal linguaggio come i *tipi primitivi*, le *operazioni primitive*, le *costanti predefinite* e così via. Va sempre tenuto presente che la formalizzazione di quelli che sono gli oggetti denotabili dipende principalmente dal progettista del linguaggio.

La nozione di oggetto denotabile ci consente di definire cosa è l'**ambiente**.

**Definizione 10.1.3 — Ambiente.** L'ambiente è un insieme di associazioni tra nomi ed oggetti denotabili esistenti a tempo d'esecuzione (run-time) in uno specifico punto del programma ed in un specifico momento dell'esecuzione

L'ambiente è anche chiamato *referencing environment*. Collegate alla nozione di ambiente abbiamo varie nozioni che è opportuno sottolineare:

- La prima è quella ovvia di **binding**, ossia di legame, o anche associazione, che è quella che abbiamo tra nomi ed oggetti denotati;
- la seconda, altrettanto rilevante, è quella di **binding time**, che è il momento in cui avviene il *binding*;
- infine abbiamo la nozione di **scope**, che è la parte di programma in cui una particolare associazione tra nome e oggetto denotato, cioè un binding è valida.

Lo scope ha come sinomini visibilità, campo d'azione, e portata.

**Definizione 10.1.4 — Scope.** Dato un programma ed un dato nome, lo scope di una certa associazione tra il nome e l'oggetto denotato da quel nome, è dato dalla parte di programma in cui quell'associazione è nota a tempo d'esecuzione.

In altre parole lo scope serve per capire come è fatto l'ambiente in un certo punto del programma. Ci domandiamo ora come formare (o inserire) un'associazione tra nome ed oggetto denotabile in un ambiente. In generale lo facciamo esplicitamente con una **dichiarazione**.

**Definizione 10.1.5 — Dichiarazione.** La dichiarazione è un meccanismo, implicito o esplicito, con cui si crea un'associazione nell'ambiente.

Il modo **esplicito** è quello che ci immaginiamo: all'interno del programma troviamo parti come `int x = 3`, `int f()` e simili, e quando si incontrano queste dichiarazioni si crea un'associazione nell'ambiente tra il nome e l'oggetto denotato, che in generale viene creato al momento della dichiarazione.

Il modo **implicito** lo abbiamo principalmente quando abbiamo programmi divisi in moduli, in questo caso l'associazione si crea al momento dell'uso di un nome, che è dichiarato in un'interfaccia.

### 10.1.1 Binding

Il momento in cui avviene il legame, cioè il **binding time** ha particolare rilevanza.

**Definizione 10.1.6 — Binding time statico.** Il binding time può essere **statico**:

- al momento della **progettazione del linguaggio**: tipi primitivi, nomi per operazioni e costanti predefinite,
- al momento della **scrittura del programma**: definizione di alcuni nomi, per esempio variabili, funzioni, il cui legame potrà essere completato dopo (definizioni di interfacce),
- al momento, se c'è, della **compilazione**: il legame di alcuni nomi, ad esempio le variabili globali, può essere determinato in questa fase.

**Definizione 10.1.7 — Binding time dinamico.** Oppure il binding può essere dinamico:

- al momento dell'**esecuzione**: legame definitivo di tutti i nomi non ancora legati, come le variabili locali ai blocchi

Non va confuso con lo scope: sono due fenomeni diversi.

Ci sono dei fenomeni particolari che è utile sottolineare. Lo stesso nome può denotare oggetti distinti o in punti diversi del programma (vedremo che in fondo sono ambienti diversi) o in momenti diversi (ad esempio, procedure ricorsive), oppure nomi diversi possono denotare lo stesso oggetto (aliasing). Il seguente programma (C-like) ne fornisce un esempio:

```
int *X, *Y; // X, Y puntatori ad interi
X = (int *) malloc (sizeof (int));
           // allocata la memoria puntata
*X = 5;    // dereferenziazione (e assegnamento)
Y = X;     // X e Y puntano allo stesso oggetto
*Y = 10;
write(*X); // stampa 10
```

### 10.1.2 Blocchi

Per organizzare l'ambiente dobbiamo introdurre la nozione di blocco, che si riflette nella sintassi del linguaggio.

**Definizione 10.1.8 — Blocco.** Un blocco è regione testuale del programma, identificata da un segnale di inizio ed uno di fine, che può contenere dichiarazioni locali a quella regione.

I blocchi sono o **anonimi** (o in-line) che significa che **alla regione non è associato alcun nome**, oppure possono **non** essere **anonimi** e quindi sono in generale **associati ad una procedura o funzione**. In questo modo il blocco ha un nome, quello della procedura o della funzione. Per identificare i blocchi anonimi abbiamo bisogno di alcuni segnali. In Algol abbiamo `begin ... end`,

in C o Java abbiamo le parentesi graffe {...}, in Lisp abbiamo le parentesi tonde (...) ed in ML invece il `let ... in ...`. Per convenzione le dichiarazioni sono subito dopo il segnale di inizio del blocco, in ML sono tra il `let` e l'`in`.

Un blocco può essere anche visto come un modo per combinare una serie di istruzioni facendole vedere all'esterno come un'unica istruzione (con effetti che sono quelli cumulati). Nel caso della sintassi astratta sarà facile capire cosa sta in un blocco, dato che avremo un costruttore di blocco. Inoltre il blocco consente la gestione locale dei nomi (quelli dichiarati nel blocco sono locali al blocco). Questo aumenta la chiarezza e si possono scegliere i nomi più adatti e significativi. Inoltre i blocchi consentono di migliorare l'occupazione di memoria e permettono la ricorsione. Infine i blocchi possono essere solo annidati (uno dentro l'altro) e mai parzialmente sovrapposti (la fine del primo si trova prima della fine del secondo):

- `begin ... begin ... end ... end`  
in questo caso il blocco rosso e quello blu si sovrappongono parzialmente e questo non è consentito
- `si: begin ... begin ... end ... end`  
in questo caso il blocco blu è totalmente contenuto nel blocco rosso e questo è consentito.

### 10.1.3 Tipi d'ambiente e operazioni

In effetti c'è solo un ambiente, come la definizione di ambiente suggerisce, ma per convenienza introduciamo delle distinzioni:

- ambiente **locale**: sono le associazioni create all'ingresso nel blocco (variabili locali, parametri formali, ecc.),
- ambiente **non locale**: sono le associazioni create fuori dal blocco ma che sono ancora valide all'interno di quel blocco,
- ambiente **globale**: sono le associazioni che sono comuni a tutti i blocchi.

Quindi l'ambiente si suddivide in questi tre tipi. Un ambiente **ha sempre** la parte *locale*, mentre **può avere** la parte *globale* o *non locale*. Osserviamo che l'ambiente globale è un caso particolare di ambiente non locale.

Elenchiamo ora quali sono le operazioni che si compiono sull'ambiente.

- **creazione**: creo un'associazione nome - oggetto denotato (tipicamente in ingresso ad un blocco, quando si incontrano le dichiarazioni,
- **riferimento**: uso in qualche modo l'oggetto denotato attraverso il suo nome,
- **disattivazione**: un'associazione viene disattivata (ad esempio, in un blocco interno uso un nome usato in un blocco più esterno),
- **riattivazione**: un'associazione disattivata ritorna in vita (ad esempio, esco dal blocco interno),
- **distruzione**: l'associazione nome - oggetto denotato viene cancellata definitivamente (esco dal blocco dove l'associazione è stata creata).

Osserviamo che le operazioni sull'ambiente riguardano la creazione, uso e distruzione di un legame, legame tra un nome trovato nel testo del programma ed un oggetto. Fino ad ora abbiamo considerato dell'ambiente solo il legame ed il nome, e tramite il legame ed il nome siamo in grado di accedere o usare l'oggetto denotato. Ci domandiamo quali operazioni si possono compiere su un oggetto.

- la **creazione** dell'oggetto, possibilmente prima o al momento della creazione del legame (per creare un legame devo avere l'oggetto),
- l'**accesso** all'oggetto, facendovi riferimento,
- la **modifica** dell'oggetto, attraverso il nome con cui vi accediamo e con le opportune operazioni che lo modificano l'oggetto ma non il nome ed il legame,
- la **distruzione** dell'oggetto.

Combinando le operazioni su ambiente e oggetti abbiamo il seguente schema:

1. creazione di un oggetto
2. creazione di un legame per l'oggetto (ambiente)
3. accesso e/o modifica dell'oggetto (attraverso il legame)
4. disattivazione di un legame
5. riattivazione di un legame
6. distruzione di un legame
7. distruzione dell'oggetto

La *vita* dell'oggetto va da 1 a 7, mentre la *vita* dell'associazione va da 2 a 6.

La vita di un oggetto in generale non coincide con quella dei legami per quell'oggetto:

- vita **lunga**: la vita dell'oggetto è più lunga di quella del legame (ad esempio, un parametro passato ad una procedura per riferimento, che vedremo è il modo più diffuso)
- vita **corta**: la vita dell'oggetto è più corta di quella del legame.

Mentre la prima è una situazione naturale, la seconda non lo è. Vediamo un esempio:

```
int *X, *Y;
...
X = (int *) malloc (sizeof (int));
Y = X;
...
free(X); // deallocazione memoria (distruzione oggetto)
```

in questo caso il legame tra Y e la memoria esiste sempre anche se non c'è la memoria che è stata deallocata. Il fenomeno è quello dei *dangling reference*<sup>1</sup>.

Una volta creato un nome ed un legame, dove può essere usato? Cioè dove è noto sia il nome che il legame? Abbiamo una regola generale:

**Definizione 10.1.9 — Visibilità delle dichiarazioni locali.** Una dichiarazione locale ad un blocco è visibile in quel blocco e in tutti i blocchi in esso annidati, a meno che non intervenga in tali blocchi una nuova dichiarazione dello stesso nome (che nasconde, o maschera, la precedente).

## 10.2 Scope

Come interpretare la regola in presenza di procedure/funzioni e come interpretarla in presenza di ambiente non locale (e non globale)?

Cerchiamo di precisare il problema. Consideriamo il seguente programma (di cui molte parti sono omesse) scritto, come molti, in stile imperativo.

```
{int x = 10;
...
void uno () {
x++;
}
...
void due () {
int x = 2;
uno();
```

<sup>1</sup>Indica un puntatore che si riferisce ad un'area di memoria non più valida, perché già liberata o perché il puntatore viene utilizzato all'esterno del contesto di esistenza della variabile cui si riferisce.



```

}
...
due();
}

```

Il nome `x` viene definito in due blocchi. Il primo blocco in cui viene dichiarata è un blocco anonimo mentre il secondo blocco è quello con un nome (la procedura `due`).

Nell'esecuzione del programma si effettua una chiamata alla procedura `due`. In questo caso quale `x` viene incrementata?

Si possono adottare due strategie: la prima è far riferimento al nome dichiarato nel **blocco più vicino** che racchiude testualmente il blocco dove il nome viene usato, la seconda è fare riferimento al **legame attivo** al momento della chiamata della procedura/funzione.

- scope *statico*: si fa riferimento al legame creato nel blocco testualmente più vicino
- scope *dinamico*: si fa riferimento al legame attivo

sostanzialmente si deve decidere in quale ambiente risolvere il legame: nel caso dello scope statico nell'ambiente determinato dalla scrittura del programma, nel caso dello scope dinamico nell'ambiente determinato dall'esecuzione del programma.

### Scope statico

Il nome è risolto nel blocco che testualmente lo racchiude:

```

{int x = 10;
  void pippo (int n) {
    x = n+1;
  }
  pippo(3);
  write(x); // stampa 4
  {int x = 2;
    pippo(3); // qui si modifica la x esterna al blocco
    write(x); // stampa 2, la x interna non
  }          // viene modificata dalla pippo
  write(x); // stampa 4
}

```

### Scope dinamico

In questo caso il nome è risolto nel blocco attivato e non ancora disattivato:

```

{int x = 10;
  void pippo (int n) {
    x = n+1;
  }
  pippo(3);
  write(x); // stampa 4
  {int x = 2;
    pippo(3); // qui si modifica la x del blocco anonimo interno
    write(x); // stampa 4, la x interna viene modificata dalla pippo
  }
  write(x); // stampa 4
}

```

### Scope statico o scope dinamico?

Quasi tutti i linguaggi di programmazione hanno lo scope statico, dato che lo scope statico soddisfa i principi di *indipendenza* dalla posizione dell'uso del nome: una procedura può essere chiamata in contesti diversi ma importa solo dove è stata dichiarata, e di indipendenza dai nomi locali: se si modifica un nome locale non si ripercuote sulla procedura. Inoltre lo scope statico consente di specializzare le funzioni/procedure.

Vediamo, con due esempi, i principi di indipendenza dello scope statico.

#### Indipendenza dalla posizione

```
{int x = 10;
  void uno () {
    x++;
  }
  void due () {
    int x = 2;
    uno();
  }
  due();
  uno()
}
```

La procedura uno viene usata in contesti diversi: all'interno della due e nel blocco in cui è dichiarata, ma non importa dove si usa, la x a cui fa riferimento è sempre la stessa, quella dichiarata nel blocco anonimo. Se lo scope fosse dinamico, questo non sarebbe vero.

#### Indipendenza dai nomi locali

```
{int x = 10;
  void uno () {
    x++;
  }
  void due () {
    int y = 2;
    uno();
  }
  due();
  uno()
}
```

Se cambio la y in due con x non succede nulla, ma se lo scope fosse dinamico allora la semantica cambierebbe.

#### Specializzazione

Anche lo scope dinamico obbedisce ad un principio, quello di *specializzazione*. Consideriamo una procedura che *colora* il video ed ha una variabile colore.

```
{...
  void visualizza(text x) {
    ...
    printscreen(x, colore);
  }
}
```

```
colore = rosso;
visualizza("prova", colore);
}
```

Questa visualizza sullo schermo la parola "prova" su sfondo rosso.

### 10.2.1 Scope statico e dinamico

Confrontiamo un po' i due approcci allo scope, cercando di evidenziarne i vantaggi e gli svantaggi. Nel caso dello **scope statico** (detto anche scoping statico, statically scoped, lexical scope) abbiamo

- l'informazione è completa dal testo del programma
- le associazioni sono note a tempo di compilazione
- valgono i principi di indipendenza
- è complesso da implementare (lo vedremo), ma in generale efficiente
- i principali linguaggi lo implementano: Algol, Pascal, C, Java, ML, ecc

Nel caso dello **scope dinamico** (o scoping dinamico, dynamically scoped), abbiamo invece

- l'informazione è derivata dall'esecuzione
- i programmi con scope dinamico sono meno "leggibili"
- è semplice da implementare, ma meno efficiente
- pochi linguaggi, uno è Lisp (alcune versioni), quello più usato è Perl.

I due tipi di scope differiscono solo in presenza congiunta di ambiente non locale e non globale, quindi certamente con le procedure o le funzioni, ma anche in assenza di ambiente globale che coinvolge tutti i nomi non locali.

Ci sono eccezioni e problemi.

Cosa succede se scrivo in Pascal

```
const pippo = valore;
const valore = 0;
```

dovrei derogare alla regola che dice che un nome può essere usato solo dopo che è stato definito, ma se lo inserisco in un contesto come

```
const valore = 1;
procedure pluto
begin
  const pippo = valore;
  const valore = 0;
  ...
end
```

riesco a dare il valore alla costante, che poi ridefinisco, quindi ora la costante valore dovrebbe contenere 0, ma questa dichiarazione di valore arriva dopo il suo effettivo uso. Sempre in Pascal la dichiarazione di una lista viene fatta in questo modo

```
type lista = ^elemento;
type elemento record
  info: integer;
  next: lista
end
```

e quindi il tipo lista è definito in funzione del tipo elemento che contiene un riferimento al tipo lista. In questi casi si decide di aspettare per capire come risolvere il nome. Lo stesso fenomeno si ha con le procedure mutuamente ricorsive.

Altri linguaggi consentono dichiarazioni incomplete (esempio, ADA):

```
type elemento;
type lista = access elemento;
type elemento is record
    info: intero;
    next: lista
end
```

Qui la prima dichiarazione è incompleta, e si adotta la stessa strategia che si usa con la mutua ricorsione in Pascal.

Il C ha lo scope statico e non permette di definire funzioni in blocchi interni. Quindi l'ambiente di una funzione è diviso in ambiente locale e globale e la parte non locale e non globale non esiste. I nomi non locali vengono sempre risolti nell'ambiente più esterno (quello globale).

In C realizzo lo scope dinamico con la direttiva `#define` che corrisponde ad una macro espansione

```
int x=10;
int next_x(int delta) { return x + delta; }
#define NEXT_X(delta) x+delta
main(){
    int x=5;
    printf("%d, □%d\n", next_x(4), NEXT_X(4));
}
```

la chiamata `NEXT_X(4)` nel main viene inserita nel codice.

In Java in generale una variabile è visibile a partire dalla dichiarazione e fino alla fine del blocco, e solo dopo la dichiarazione di una variabile, quindi il seguente frammento di codice non è ammesso

```
{a=1;
  int a;
  ...
}
```

mentre il seguente si

```
{void f(){
    g();
}
void g(){
    f();
}
...
}
```

Java permette metodi mutuamente ricorsivi.

Un ulteriore problema in Pascal si ha con dichiarazioni "normali" e mutuamente ricorsive.

```
procedure pippo;
    ...
end (*pippo*)
```

```

procedure A;
...
  procedure B
  begin
    ...
    pippo;
  end (*B*)
  ...
  procedure pippo;
  begin
    ...
  end)

```

quale pippo devo usare in B? Il compilatore non sa decidere.

### 10.3 Ambiente locale

L'ambiente è l'insieme di legami tra nomi ed oggetti denotabili e le regole di scope determinano come i legami vengono ereditati tra i vari ambienti. Ci sono poi delle regole specifiche per determinare lo scope di nomi in situazioni particolari (alcune situazioni le abbiamo viste, altre le vedremo), per il passaggio dei parametri e per il *binding*, che può essere *shallow* o *deep*, di parametri formali ai quali si associano funzioni.

**Definizione 10.3.1 — Ambiente locale dinamico.** Ricordiamo che l'ambiente è sempre locale (può anche essere globale o non locale), e in effetti ci riferiamo a quelle locali, non locali o globali (appartenenti ad un solo blocco, comuni a vari blocchi, comuni a tutti i blocchi). La creazione di una associazione tra nome e oggetto denotato avviene in generale all'ingresso di un blocco, e viene inserita nell'ambiente, mentre la distruzione di una associazione tra nome e oggetto denotato avviene all'uscita dal blocco in cui l'associazione è stata creata e viene eliminata dall'ambiente. L'associazione è utilizzabile solo all'interno del blocco (a parte eventuali buchi perché siamo entrati in un altro blocco) e se rientriamo nello stesso blocco, rieseguiamo le dichiarazioni e creiamo *nuove* associazioni. Lo chiamiamo *dinamico* perché ogni volta creiamo nuove associazioni.

**Definizione 10.3.2 — Ambiente locale statico.** In questo caso non creiamo associazioni quando si entra in un blocco, ma all'ingresso di questo avviene la riattivazione di un'associazione tra nome e oggetto denotato nell'ambiente, e all'uscita del blocco in cui l'associazione è stata attivata, avviene la disattivazione dell'associazione nell'ambiente. Inoltre l'associazione è utilizzabile solo all'interno del blocco, cioè quando è attiva, e se rientriamo nello stesso blocco, non rieseguiamo le dichiarazioni, ma ci limitiamo a riattivare le associazioni precedenti.

La domanda è quindi chi esegue le dichiarazioni, e queste sono eseguite in generale o dal compilatore (prima dell'inizio dell'esecuzione) oppure alla prima esecuzione del blocco (o simile).

Nei moderni linguaggi di programmazione l'ambiente locale è dinamico, ma alcuni linguaggi hanno (anche) ambiente locale statico. In particolare nel Fortran dove non ci sono blocchi, questa è la regola per l'ambiente locale dei sottoprogrammi, le diverse attivazioni (chiamate) dello stesso sottoprogramma condividono ambiente e memoria locali, e quindi la chiamata *n*-esima trova lo stato lasciato dalla chiamata (*n-1*)-esima. Nell'Algol, nel PL/I o nel C alcune dichiarazioni locali (a blocchi o sottoprogrammi) possono essere dichiarate statiche (*static*, *own*, ...) e per queste particolari associazioni l'ambiente è statico, le altre sono trattate con l'ambiente dinamico. In Java

le dichiarazioni static all'interno di una classe provocano la creazione di ambiente (ed eventuale memoria) che appartengono alla classe e non agli oggetti sue istanze, sono eseguite una sola volta all'atto della esecuzione della dichiarazione di classe e possono costituire uno stato interno comune a tutti gli oggetti della classe.

Perché usare l'ambiente locale statico? Le dichiarazioni statiche sono più efficienti, ma creano comportamenti molto più complessi da comprendere dal punto di vista semantico dato che la presenza di uno stato interno fa sì che la semantica debba tener conto di tutta la sequenza di attivazioni. Queste sono utili se si vogliono definire blocchi (o meglio sottoprogrammi) dotati di stato interno (ad esempio, un generatore di numeri casuali, o un generatore di nomi sempre nuovi, ecc.), inoltre l'implementazione dell'ambiente locale statico è complessa.

## 10.4 Gestione della memoria

La seconda parte dello stato è la **memoria**, che è quella che viene realmente modificata dall'esecuzione di un comando, mentre l'ambiente viene modificato dalla dichiarazione, come vedremo in seguito. Tra gli oggetti denotabili includiamo le locazioni.

**Definizione 10.4.1 — Memoria.** La memoria è un insieme di associazioni tra locazioni ed oggetti memorizzabili.

La memoria può essere **statica**, cioè la memoria viene allocata a tempo di compilazione (ad esempio, le variabili globali), oppure **dinamica**, ad esempio la memoria viene allocata o su una pila (stack), dove tutti gli oggetti sono allocati con politica *LIFO* (*Last In First Out*), oppure su uno heap, dove gli oggetti sono allocati e deallocati in qualsiasi momento (in generale per mezzo di puntatori). Inoltre l'implementazione delle regole di scope è legata all'avere sottoprogrammi e/o meccanismi come blocchi per strutturare l'ambiente (se non li avessimo, la gestione della memoria sarebbe ovviamente sempre statica), non avremmo necessità di realizzare le regole di scope.

### 10.4.1 Allocazione statica

In questo caso un oggetto ha un **indirizzo assoluto** che è mantenuto per tutta l'esecuzione del programma.

Solitamente sono allocate staticamente le

- variabili globali,
- le variabili locali di sottoprogrammi senza ricorsione,
- le costanti determinabili a tempo di compilazione
- le tabelle usate dal supporto a run-time (per type checking, garbage collection, ecc.).

Spesso sono usate zone protette di memoria.

Va osservato che l'allocazione statica ha come conseguenza che non è possibile avere allo stesso tempo incarnazioni diverse della stessa funzione o porzione di programma, quindi la ricorsione non è sostanzialmente implementabile, a parte la ricorsione in codo per motivi che saranno chiari successivamente.

### 10.4.2 Allocazione dinamica: la pila

Ogni istanza di un sottoprogramma a run-time ha una porzione di memoria detta **record di attivazione** (*RdA* o *frame*) contenente le informazioni relative alla specifica istanza (compreso l'indirizzo di ritorno). Anche ogni blocco anonimo ha un suo record di attivazione (che è più semplice). La pila (*LIFO*) è la struttura dati naturale per gestire i RdA perché le chiamate di procedura (anche ricorsive) ed i blocchi sono annidati uno dentro l'altro.

Vediamo nel dettaglio come viene gestita la pila dei RdA, precisando prima cosa in generale contiene un generico RdA.

Un RdA contiene **tutte le informazioni relative alla specifica istanza del blocco in esecuzione**, quindi il record d'attivazione di un blocco anonimo ha spazio per

- i **nomi locali**, quelli cioè che sono dichiarati nel blocco (e che serviranno a capire cosa è associato a quel nome, quindi l'ambiente locale)
- eventuale spazio per memorizzare **risultati intermedi**, cioè quelli che sono i risultati parziali della valutazione di un'espressione complessa (ad esempio  $(x + y)/(2 * x) + x * y$  ha bisogno di memorizzare i valori delle sotto espressioni  $(x + y)/(2 * x)$  e  $x * y$ )
- il riferimento al RdA del blocco (anonimo o no) che racchiude il blocco che stiamo eseguendo, cioè quello in cima alla pila degli RdA, e che si chiama in generale **puntatore di catena dinamica** ed il **puntatore di catena statica**

La **catena statica** consente di identificare il record d'attivazione che contiene l'ambiente non locale nel caso di scope statico.

Il record d'attivazione di un blocco non anonimo, quindi di una **procedura** o **funzione**, contiene, oltre a quello che contiene il RdA di un blocco anonimo, anche :

- spazio per i **parametri**, che fanno parte dell'ambiente locale del blocco
- informazioni sull'**indirizzo di ritorno**
- l'**indirizzo** dove mettere il **risultato** se è una funzione che restituisce un risultato

Nella gestione della pila dei RdA identifichiamo i passi:

- **sequenza di chiamata**: è il codice eseguito dal chiamante immediatamente prima della chiamata, in questa fase si prepara il RdA che deve essere messo sulla pila
- **prologo**: è il codice eseguito all'inizio del blocco (con nome o senza), e qui si eseguono le dichiarazioni e se serve si effettuano le associazioni per il passaggio dei parametri
- **epilogo**: è il codice eseguito alla fine del blocco (con nome o senza), che prepara all'effettivo pop dalla pila del record d'attivazione,
- **sequenza di ritorno**: è il codice eseguito dal chiamante immediatamente dopo la chiamata, e coinvolge non solo il levare il RdA dalla cima della pila, ma anche la comunicazione dei risultati e delle modifiche effettuate dal blocco che era in cima alla pila, cioè in esecuzione.

L'indirizzo in memoria di un RdA non può essere noto a compile-time, quindi c'è un *Puntatore RdA* (o SP, *stack pointer*) che punta al RdA del blocco attivo e le informazioni contenute in un RdA (nomi locali, risultati intermedi, ecc.) sono accessibili per offset rispetto allo SP:

$$\text{indirizzo-info} = \text{contenuto (SP)} + \text{offset}$$

e l'offset è determinabile staticamente.

## 10.5 Memoria e pila degli RdA

Come abbiamo sempre un ambiente locale abbiamo anche sempre una memoria locale, e questa è allocata nel RdA. In particolare troviamo tutta la memoria relativa alle strutture dati statiche (la cui dimensione è determinata al momento della compilazione o scrittura del programma) è allocata nel RdA, mentre nel caso di memoria relativa a strutture dati dinamiche, la cui dimensione è determinata dall'esecuzione, nel RdA si alloca solo il modo per arrivarci (un altro puntatore).



## 10.6 Ambiente e pila degli RdA

I record d'attivazione servono anche ad implementare l'*ambiente* (alla memoria ci accederò normalmente con un nome, e quindi devo capire soprattutto come si implementano le regole di scope).

Ricordiamo che abbiamo una regola di visibilità nella Definizione 10.1.9, quindi se una dichiarazione è *locale* ad un blocco, come la troviamo se siamo in un altro blocco?

Per usarla non possiamo usare l'offset e l'SP ma dobbiamo risalire, all'interno della pila, per trovare il giusto RdA dove la variabile è dichiarata e per questo si possono usare solo i link a disposizione: il *puntatore di catena dinamica* o il *puntatore di catena statica*. Abbiamo visto che nel RdA di un blocco non anonimo abbiamo due puntatori. Quale dei due usiamo? Vale prima la pena sottolineare che che lo scope statico o dinamico hanno senso solo in presenza di ambiente non locale e non globale, dato che l'ambiente globale lo possiamo realizzare all'esterno della pila dei RdA.

Il **puntatore di catena statica** è quello usato per realizzare lo **scope statico** (che abbiamo detto è un po' più complesso da implementare dato che dobbiamo calcolare questo puntatore), mentre per realizzare lo **scope dinamico** basta il **puntatore di catena dinamica**, che sappiamo calcolare facilmente.

### 10.6.1 Implementazione dello scope statico

Ricordiamo che nello scope *statico* si fa riferimento al legame creato nel blocco testualmente più vicino, ed al momento dell'esecuzione, come si trova il RdA del blocco *testualmente* più vicino? Devo fare attenzione, dato che tra il RdA **attivo** (quello in cima alla pila) e quello che contiene il non locale al quale sono interessato, ci possono essere RdA che contengono come nome locale proprio quello. Usiamo l'annidamento dei blocchi.

**Definizione 10.6.1 — Annidamento dei blocchi.** Dato un frammento di programma scritto in un linguaggio  $\mathcal{L}$  ad alto livello, l'annidamento misura il numero di blocchi/sottoprogrammi aperti e non ancora richiusi all'interno del quale si trova l'occorrenza di un certo nome.

Ricordiamo che i nomi che usiamo devono essere in *scope*, che significa che devono essere noti.

Per calcolare il puntatore di catena statico abbiamo a disposizione solo il puntatore di catena statico del chiamante, cioè del blocco dove avviene la chiamata alla funzione o da dove si accede ad un nuovo blocco annidato all'interno. Abbiamo due casi:

- il chiamato si trova all'**interno** del chiamante, e qui è tutto facile: il puntatore di catena statica è il puntatore di catena dinamica coincidono, oppure
- il chiamato si trova all'**esterno** del chiamante, quindi dato che il chiamato è un nome noto, lo avremo trovato utilizzando il puntatore di catena statico del chiamante. In che modo? Siamo risaliti lungo la *catena statica* fino a che non abbiamo trovato il nome corretto, e questa *distanza* è nota al momento della scrittura del programma, quindi basta aggiornare il puntatore di catena statica del nuovo RdA in base a questo calcolo.



# 11. Il linguaggio IMP con i blocchi e le dichiarazioni

## 11.1 Il linguaggio IMP con i blocchi e le dichiarazioni (locali)

Gli oggetti memorizzabili sono tutti quelli che immaginiamo possano essere in qualche modo memorizzati.

Possiamo ora dire cosa è lo stato.

**Definizione 11.1.1 — Stato.** Lo stato è una coppia  $(\delta, \sigma)$  dove  $\delta$  è un ambiente, cioè un'associazione tra nomi e oggetti denotabili, che comprendono le locazioni, e  $\sigma$  è una memoria, cioè un'associazione tra locazioni e oggetti memorizzabili.

Avendo ora una nozione di stato più ricca, possiamo introdurre nel linguaggio imperativo IMP i blocchi e le dichiarazioni locali.

### 11.1.1 Dichiarazioni

Le dichiarazioni sono le seguenti

$$d ::= \text{null} \mid d_1; d_2 \mid \text{var } X$$

null è la dichiarazione vuota.

Lo stato ora sarà composto da un'associazione nomi  $\rightarrow$  oggetti denotabili (che contengono anche le locazioni), cioè l'ambiente, e da una memoria che associa alle locazioni gli oggetti memorizzabili.

Le dichiarazioni servono sostanzialmente a definire l'ambiente.

La **semantica** delle dichiarazioni è:

$$\frac{}{\langle \text{null}, (\rho, \sigma) \rangle \mapsto_d \rho} \quad \frac{\text{loc} = \text{newloc} ()}{\langle \text{var } X, (\rho, \sigma) \rangle \mapsto_d \rho \{ \text{loc} / X \}} \\ \frac{\langle d_1, (\rho, \sigma) \rangle \mapsto_d \rho'' \quad \langle d_2, (\rho'', \sigma'') \rangle \mapsto_d \rho'}{\langle d_1; d_2, (\rho, \sigma) \rangle \mapsto_d \rho'}$$

dove  $\text{newloc}()$  è una funzione che restituisce una nuova locazione.

Se indichiamo con  $\text{Env} = \{ \rho \mid \rho : \text{Name} \rightarrow \text{DenObj} \}$  abbiamo che

$$\mapsto_d \subseteq \text{Dich} \times \text{Env} \times \text{Env}$$

e questa relazione definisce, come prima, una funzione  $\mathcal{D} : Dich \times Env \rightarrow Env$ . *Name* sono i nomi dei programmi (in questo caso sono variabili) mentre *DenObj* sono gli oggetti denotabili che contengono *Loc* che sono le locazioni. Infine *newloc()* è una funzione che restituisce una locazione non usata.

### 11.1.2 Comandi

Ai comandi aggiungiamo  $\text{block}(d, c)$ , ma avendo modificato lievemente la nozione di stato, riscriviamo tutte le regole della *big-step* semantics, che ancora restituisce solo una memoria, che ricordiamo è quella che viene modificata dall'unico comando che modifica realmente lo stato che è l'assegnamento. La memoria è una funzione parziale tra locazioni e oggetti memorizzabili che sono i valori. Quindi

$$Mem = \{\sigma \mid \sigma : Loc \rightarrow \mathbb{B} \cup \mathbb{Z}\}$$

$$\begin{array}{c}
\frac{}{\langle \text{skip}, (\rho, \sigma) \rangle \Rightarrow_{bs} \sigma} \quad \frac{\langle e, (\rho, \sigma) \rangle \rightarrow v}{\langle X := e, (\rho, \sigma) \rangle \Rightarrow_{bs} \sigma\{v/\rho(X)\}} \\
\\
\frac{\langle c_0, (\rho, \sigma) \rangle \Rightarrow_{bs} \sigma'' \quad \langle c_1, (\rho, \sigma'') \rangle \Rightarrow_{bs} \sigma'}{\langle c_0; c_1, (\rho, \sigma) \rangle \Rightarrow_{bs} \sigma'} \\
\\
\frac{\langle e, (\rho, \sigma) \rangle \rightarrow \text{true} \quad \langle c_0, (\rho, \sigma) \rangle \Rightarrow_{bs} \sigma'}{\langle \text{if } e \text{ then } c_0 \text{ else } c_1, (\rho, \sigma) \rangle \Rightarrow_{bs} \sigma'} \quad \frac{\langle e, (\rho, \sigma) \rangle \rightarrow \text{false} \quad \langle c_1, (\rho, \sigma) \rangle \Rightarrow_{bs} \sigma'}{\langle \text{if } e \text{ then } c_0 \text{ else } c_1, (\rho, \sigma) \rangle \Rightarrow_{bs} \sigma'} \\
\\
\frac{\langle b, (\rho, \sigma) \rangle \rightarrow \text{false}}{\langle \text{while } b \text{ do } c, (\rho, \sigma) \rangle \Rightarrow_{bs} \sigma} \\
\\
\frac{\langle b, (\rho, \sigma) \rangle \rightarrow \text{true} \quad \langle c, (\rho, \sigma) \rangle \Rightarrow_{bs} \sigma'' \quad \langle \text{while } b \text{ do } c, \sigma'' \rangle \Rightarrow_{bs} \sigma'}{\langle \text{while } b \text{ do } c, (\rho, \sigma) \rangle \Rightarrow_{bs} \sigma'} \quad (11.1)
\end{array}$$

mentre quella per i blocchi sarà:

$$\frac{\langle d, (\rho, \sigma) \rangle \mapsto_d \rho' \quad \langle c, (\rho', \sigma) \rangle \Rightarrow_{bs} \sigma'}{\langle \text{block}(d, c), (\rho, \sigma) \rangle \Rightarrow_{bs} \sigma'}$$

Ora la semantica è una relazione  $\Rightarrow_{bs} : Com \times Env \times Mem \times Mem$  che è una funzione, che indicheremo ancora con  $\mathcal{C}$ .

Nel caso della semantica *small step* dobbiamo fare un'osservazione. L'ambiente nel quale si esegue il comando di un blocco è ottenuto dall'ambiente che si ha prima del blocco, e l'ipotetica istruzione successiva al blocco dovrà di nuovo essere valutata nell'ambiente precedente al blocco. La regola della *big step* semantics nasconde questo, mentre nel caso della *small step* semantics dobbiamo per forza tenerne esplicitamente conto, in particolare dobbiamo esplicitamente tenere conto della implicita struttura a pila, che dobbiamo esplicitare.

Sostanzialmente ora gli ambienti sono una *pila*(stack) sulla quale abbiamo delle operazioni:

- *top* restituisce l'ambiente in cima alla pila
- *push* riceve una pila ed un ambiente e restituisce una pila con in cima il nuovo ambiente
- *pop* riceve una pila e restituisce una pila priva dell'ambiente in cima
- Infine *empty* è la pila vuota.

Con *EnvQueue* indichiamo la pila degli ambienti. Abbiamo quindi

- *top* :  $EnvQueue \rightarrow Env$
- *push* :  $EnvQueue \times Env \rightarrow EnvQueue$

- $\text{pop} : \text{EnvQueue} \rightarrow \text{EnvQueue}$
- $\text{empty} \in \text{EnvQueue}$

Gli elementi di  $\text{EnvQueue}$  sono indicati con  $\Omega$ .

L'ultima aggiunta da fare è ai comandi, ai quali aggiungiamo il comando  $\mathbf{bl}(c)$  che però non è utilizzabile nella scrittura di un programma, serve sostanzialmente solo per la definizione della semantica *small step* (lo scriviamo in grassetto proprio per distinguerlo).

Ricapitolando, in sostanza definiremo una relazione

$$\rightsquigarrow \subseteq \text{Com}' \times \text{EnvQueue} \times \text{Mem} \times \text{Com}' \times \text{EnvQueue} \times \text{Mem}$$

, mentre

$$\Longrightarrow_{ss} \subseteq \text{Com} \times \text{EnvQueue} \times \text{Mem} \times \text{Mem}$$

$\text{Com}'$  contiene comandi che della forma  $\mathbf{bl}(c)$ .

$$\begin{array}{c} \frac{}{\langle \text{skip}, (\Omega, \sigma) \rangle \Longrightarrow_{ss} \sigma} \quad \frac{\langle a, (\text{top}(\Omega), \sigma) \rangle \longrightarrow v \quad l = \text{top}(\Omega)(X)}{\langle X := a, (\Omega, \sigma) \rangle \Longrightarrow_{ss} \sigma(v/l)} \\[10pt] \frac{\langle c_0, \sigma \rangle \Longrightarrow_{ss} \sigma'}{\langle c_0; c_1, (\Omega, \sigma) \rangle \rightsquigarrow \langle c_1, (\Omega, \sigma') \rangle} \quad \frac{\langle c_0, \sigma \rangle \rightsquigarrow \langle c'_0, (\Omega, \sigma') \rangle}{\langle c_0; c_1, (\Omega, \sigma) \rangle \rightsquigarrow \langle c'_0; c_1, \sigma' \rangle} \\[10pt] \frac{\langle e, (\text{top}(\Omega), \sigma) \rangle \longrightarrow \text{true}}{\langle \text{if } e \text{ then } c_0 \text{ else } c_1, (\Omega, \sigma) \rangle \rightsquigarrow \langle c_0, (\Omega, \sigma) \rangle} \quad \frac{\langle e, (\text{top}(\Omega), \sigma) \rangle \longrightarrow \text{false}}{\langle \text{if } e \text{ then } c_0 \text{ else } c_1, (\Omega, \sigma) \rangle \rightsquigarrow \langle c_1, (\Omega, \sigma) \rangle} \\[10pt] \frac{\langle e, (\text{top}(\Omega), \sigma) \rangle \longrightarrow \text{true}}{\langle \text{while } e \text{ do } c, (\Omega, \sigma) \rangle \rightsquigarrow \langle c; \text{while } e \text{ do } c, (\Omega, \sigma) \rangle} \quad \frac{\langle e, (\text{top}(\Omega), \sigma) \rangle \longrightarrow \text{false}}{\langle \text{while } e \text{ do } c, (\Omega, \sigma) \rangle \Longrightarrow_{ss} \sigma} \end{array}$$

quella per i blocchi diventa

$$\frac{\langle d, \text{top}(\Omega) \rangle \mapsto_d \rho}{\langle \text{block}(d, c), (\Omega, \sigma) \rangle \Longrightarrow_{ss} \langle \mathbf{bl}(c), (\text{push}(\Omega, \rho), \sigma) \rangle}$$

ed infine quelle per  $\mathbf{bl}$  sono

$$\frac{\langle c, (\Omega, \sigma) \rangle \Longrightarrow_{ss} \sigma'}{\langle \mathbf{bl}(c), (\Omega, \sigma) \rangle \rightsquigarrow \langle \text{skip}, (\text{pop}(\Omega), \sigma') \rangle} \quad \frac{\langle c, (\Omega, \sigma) \rangle \rightsquigarrow \langle c', (\Omega, \sigma') \rangle}{\langle \mathbf{bl}(c), (\Omega, \sigma) \rangle \rightsquigarrow \langle \mathbf{bl}(c'), (\Omega, \sigma') \rangle}$$

In questo modo la pila degli ambienti viene gestita dalle regole dalla *small step* semantics e non è lasciata all'implementazione della *big step* semantics.

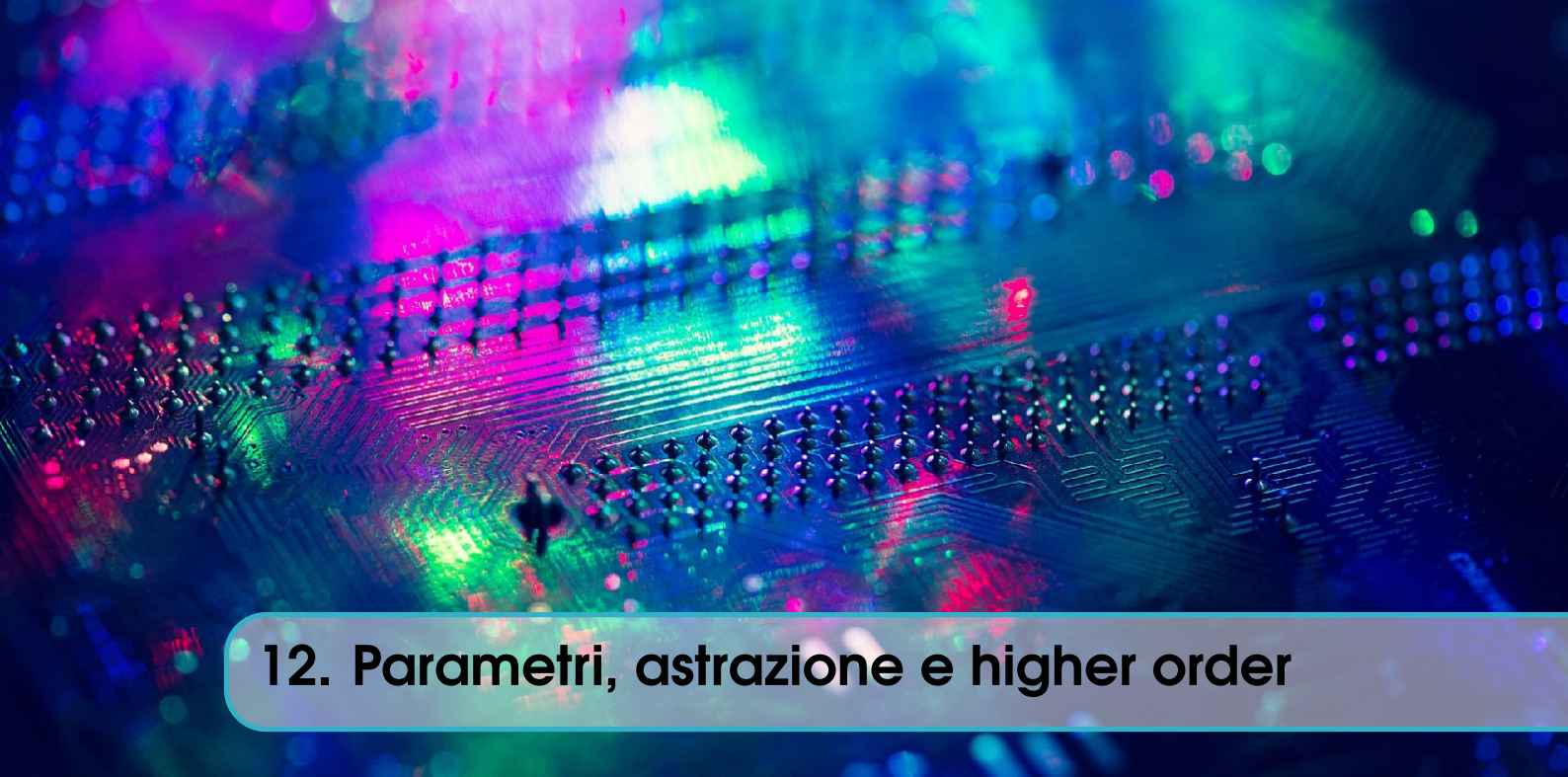
Osserviamo che quando dobbiamo in generale eseguire un comando  $c$  useremo le regole definite per trovare quale memoria è associata a  $\langle c, (\text{empty}, \sigma) \rangle$ .

Di nuovo possiamo vedere che le due semantiche sono equivalenti, tenendo presente che ora lo stato è una coppia formata da un ambiente e una memoria o una pila di ambienti e una memoria.

**Teorema 11.1.1** Per ogni comando  $c \in \text{Com}$ , per ogni ambiente  $\rho \in \text{Env}$  e per ogni memoria  $\sigma \in \text{Mem}$  abbiamo che  $\langle c, (\rho, \sigma) \rangle \Longrightarrow_{bs} \sigma'$  se e solo se  $\langle c, (\text{push}(\text{empty}, \rho), \sigma) \rangle \Longrightarrow_{ss} \sigma'$ .







## 12. Parametri, astrazione e higher order

### 12.1 Programmi strutturati

Abbiamo visto che i programmi di un linguaggio di programmazione sono in generale **strutturati**. Essere strutturati ha una importante conseguenza: gli elementi per capire cosa deve essere eseguito sono tutti contenuti nella porzione di programma che si sta eseguendo.

Nel caso di un programma di un linguaggio appartenente al paradigma funzionale abbiamo che l'applicazione determina cosa dobbiamo eseguire, nel caso della programmazione imperativa e dichiarativa l'istruzione successiva da eseguire è scritta nell'istruzione medesima che viene eseguita, mentre nel paradigma orientato ad oggetti i messaggi sono tra oggetti in quel momento attivi.

Abbiamo visto che il modo di strutturare i programmi è attraverso i **blocchi**, che possono essere o anonimi oppure avere un nome, e questi costituiscono sostanzialmente le funzioni, procedure e moduli che usiamo per strutturare i programmi.

Un blocco, che abbia un nome o meno, è in genere costituito da due parti:

- una **dichiarazione**, che ci serve per costruire l'ambiente  $\delta$ ,
- un **corpo** che va eseguito tenendo conto dell'ambiente costruito dalla dichiarazione.

I vantaggi di questo approccio sono:

- facilita la progettazione *top-down* (gerarchica) dei programmi,
- consente la *modularizzazione* del codice, che quindi può essere diviso,
- consente l'uso di *nomi significativi per denotare oggetti*,
- consente l'uso di tipi di dato strutturati e di costrutti strutturati per il controllo come l'*if* oppure, nel caso del paradigma imperativo, il *while*,
- con i commenti rende il codice comprensibile.

Per poter parlare d'astrazione bisogna anche parlare di ricorsione. Per questo ricordiamo che le **definizioni induttive** sono quelle date dicendo prima cosa si fa nei casi base e poi cosa si fa in nei casi *generici*. L'esempio classico, il solito fattoriale:

$$fatt(x) = \begin{cases} 1 & \text{se } x = 0 \\ x \times fatt(x - 1) & \text{altrimenti} \end{cases}$$

Queste definizioni le posso vedere come delle dichiarazioni su cosa devo fare per calcolare (da qui i linguaggi del paradigma dichiarativo).



Le funzioni ricorsive richiamano se stesse, e il *calcolo* viene eseguito usando la pila dei RdA. Abbiamo un caso particolare che bisogna ricordare: la ricorsione in coda (tailrecursion).

**Definizione 12.1.1 — Chiamata in coda.** Sia  $f$  una funzione che nel suo corpo contiene una chiamata alla funzione  $g$  (non necessariamente diversa da  $f$ ). La chiamata di  $g$  si dice chiamata in coda se la funzione  $f$  restituisce il valore restituito da  $g$  senza dover fare alcuna ulteriore computazione.

**Definizione 12.1.2 — Ricorsione in coda.** Diciamo che  $f$  è tail-recursive (ha la ricorsione in coda) se tutte le chiamate ricorsive in  $f$  sono chiamate in coda.

Se la chiamata ricorsiva è **l'ultima istruzione** allora **non mi devo ricordare i punti di ritorno** e mi basta mettere sulla pila un solo RdA e non tutti quelli delle istanze di chiamata della funzione. Con un po' di trasformazioni molte funzioni possono essere rese tail-recursive.

Osserviamo che non c'è alcuna sostanziale differenza tra ricorsione ed iterazione dal punto di vista teorico. Un programma iterativo può essere reso ricorsivo (l'idea è che un while lo vedo come una funzione a cui passo come parametro la condizione booleana) ed un programma ricorsivo può essere reso iterativo nello stesso modo. Un tempo l'*iterazione* era considerata più *efficiente*, dato che un compilatore la traduceva in codice macchina facilmente, mentre la *ricorsione* era considerata più *elegante* ma meno efficiente dato che si calcola con la pila dei record d'attivazione. Ora questa differenza è considerata risibile, quindi possiamo dire che non c'è reale differenza.

## 12.2 Astrazione

Abbiamo detto che l'astrazione è quel meccanismo che consente *realmente* di avere linguaggi ad alto livello. Con l'astrazione possiamo identificare proprietà importanti di cosa si vuole descrivere, e possiamo focalizzare l'attenzione sulle questioni rilevanti, ignorando quelle meno importanti, infine consente di specificare cosa interessa, facilitando in questo modo la *scrittura* del programma. Ovviamente cosa è rilevante dipende da cosa si vuole ottenere.

Nel caso del paradigma imperativo, astrarre il controllo significa nascondere dettagli procedurali. Vediamo ora come si realizza effettivamente l'astrazione.

### 12.2.1 Sottoprogrammi e parametri

**Definizione 12.2.1 — Sottoprogramma.** Un **sottoprogramma** è una parte di programma che ha un **nome**, ha una **sequenza di parametri** (che vengono chiamati **parametri formali** nella definizione del sottoprogramma e **attuali** quando il sottoprogramma viene effettivamente usato) e può restituire un valore (quindi essere una funzione ed avere un tipo) o un oggetto.

Se non restituisce alcunché, può modificare lo stato della macchina astratta. In generale nel linguaggio c'è la possibilità di definirli e di invocarli.

Ricordiamo che storicamente si usava la keyword `function` per le astrazioni che restituiscono un valore e la keyword `procedure` per quelle che non restituiscono un valore. Nei linguaggi moderni sono tutte funzioni, e le procedure restituiscono l'unico elemento del tipo `void`.

Perché introdurre l'astrazione? In generale un frammento di programma (sequenza di istruzioni) risulta utile in diversi punti del programma, e si riduce il "costo della programmazione" se si può dare un nome al frammento e qualcun altro inserisce automaticamente il codice del frammento ogni

qualvolta nel "programma principale" c'è un'occorrenza del nome<sup>1</sup>. Si riduce anche l'occupazione di memoria se esiste un meccanismo che permette al programma principale di trasferire il *controllo* ad una unica copia del frammento memorizzata separatamente e di riprendere il controllo quando l'esecuzione del frammento termina (ricordiamo che la subroutine è in generale supportata anche dall'hardware (codice rientrante)). Infine il frammento diventa ancora più importante se può essere realizzato in modo parametrico.

Vediamo come funziona in generale. A **livello hardware**, c'è una **operazione primitiva** di return jump. Quando viene eseguita (nel programma chiamante) l'istruzione return jump *a* memorizzata nella cella *b* il controllo viene trasferito alla cella *a* (entry point della subroutine) e l'indirizzo dell'istruzione successiva *b* + 1 viene memorizzato da qualche parte prefissata (punto di ritorno). Quando nella subroutine si esegue una operazione di return il controllo ritorna all'istruzione (del programma chiamante) memorizzata nel punto di ritorno.

Nel Fortran (imperativo) una subroutine è (meglio era) un pezzo di codice compilato, a cui sono associati:

- una cella destinata a contenere (a tempo di esecuzione) i punti di ritorno relativi alle (possibili varie) chiamate
- alcune celle destinate a contenere i valori degli eventuali parametri, oltre all'ambiente e memoria locali (in Fortran l'ambiente locale è statico).

Nel Fortran la semantica delle subroutine si definisce facilmente attraverso la copy rule statica, cioè la macroespansione. Ogni chiamata di sottoprogramma è testualmente rimpiazzata da una copia del codice facendo qualcosa per i parametri e ricordandosi che le dichiarazioni sono eseguite una sola volta

Il sottoprogramma non è *semanticamente* qualcosa di nuovo: è solo un (importante) strumento metodologico e non è compatibile con la ricorsione. Il fatto che le subroutine Fortran siano concettualmente una cosa statica fa sì che non esista di fatto il concetto di attivazione e l'ambiente locale è necessariamente statico.

Noi vogliamo una vera nozione di sottoprogramma e poter ragionare in termini di attivazioni. La semantica può essere ancora definita da una copy rule, ma deve essere dinamica, cioè ogni chiamata di sottoprogramma è rimpiazzata a tempo di esecuzione da una copia del codice e ragionare in termini di *attivazione* ci dà un sottoprogramma che è ora *semanticamente* qualcosa di nuovo, rende naturale la ricorsione e porta ad adottare la regola dell'ambiente locale dinamico

Con l'**astrazione procedurale** si realizza sia l'astrazione di una sequenza di istruzioni che l'astrazione via parametrizzazione. Cosa serve per realizzarla?

Un luogo di controllo per la gestione dell'ambiente e della memoria (pila degli RdA), ed è un'estensione della nozione di blocco (senza funzioni o procedure è staticamente chiaro cosa c'è), ed è in assoluto l'aspetto più interessante dei linguaggi, intorno a cui ruotano tutte le decisioni semantiche importanti.

Invece delle informazioni (punto di ritorno, parametri, ambiente e memoria locale) staticamente associate al codice compilato del Fortran vogliamo avere i record di attivazione, che contengono più o meno le stesse informazioni, e questi devono essere associati dinamicamente alle varie chiamate di sottoprogrammi. Ci possiamo aspettare che i record di attivazione siano organizzati in una pila dato che i sottoprogrammi hanno un comportamento LIFO e l'ultima attivazione creata nel tempo è la prima che *ritorna*. Ecco perché la pila degli RdA è così importante.

C'è flusso d'informazione tra programma e sottoprogramma: quando eseguiamo l'invocazione di un sottoprogramma, questo "comunica" con il chiamante attraverso i seguenti mezzi

---

<sup>1</sup>Attenzione che l'astrazione è diversa da avere le macro e la conseguente macroespansione, che è una sostituzione testuale.

- l'ambiente non locale (il meccanismo meno sofisticato),
- i parametri, che sono passati in vari modi: per valore, per riferimento, per costante, per risultato, per valore-risultato e per nome,
- il valore restituito se è una funzione.

L'invocazione corrisponde a mettere sulla pila degli RdA il record d'attivazione del sottoprogramma e quindi calcolare lo stato della macchina astratta in cui eseguire le istruzioni del sottoprogramma, cioè l'ambiente, la memoria ed eventuali strutture dinamiche come lo heap.

Vediamo i vari modi di passaggio dei parametri:

### Passaggio per valore

Comunicazione **unidirezionale**: dal programma al sottoprogramma. È spesso la modalità di default per cui non ci sono particolari keywords (le mettiamo solo se vogliamo sottolineare la modalità). Il **valore** del parametro attuale (che deve quindi essere di un tipo che ha valori, non void che ha un solo valore) **viene associato al parametro formale**.

Nei linguaggi funzionali l'operazione è sull'ambiente mentre nei linguaggi imperativi l'operazione è sull'ambiente e sulla memoria. **Non c'è comunicazione** tra chiamante e chiamato attraverso il parametro, dato che non c'è alcun legame tra il parametro formale e quello attuale. **È costoso per dati grandi**: se ne creano copie.

In Java, Scheme, Pascal è il modo di default, per C e Java è anche l'unico modo.

### Passaggio per riferimento

Comunicazione **bidirezionale** tra programma e sottoprogramma, tipica dei linguaggi imperativi.

Si associa al parametro formale il **riferimento** al parametro attuale (che non viene valutato). **Il parametro attuale deve essere una variabile** a cui è associata una locazione e l'operazione è sempre sull'ambiente. Il parametro formale è un'*alias* del parametro attuale, ed è efficiente dato che non si valuta e si cambia solo l'ambiente. Gli effetti si ripercuotono all'esterno, quindi attenzione. Nel Pascal (keyword var), in C se passo un puntatore ho lo stesso effetto (quindi è simulabile), e per questo il C ha solo il passaggio per valore. Quello per riferimento è simulato.

### Passaggio per costante

Comunicazione **unidirezionale** e **obsoleta**: da programma a sottoprogramma. Associa al parametro formale la valutazione del parametro attuale (come nel passaggio per valore) ma nel sottoprogramma **non è consentito modificare il parametro formale**. Implementazione a discrezione (per valore nel caso di dati di piccole dimensioni, per riferimento in altri casi), ma il reale controllo è fatto sull'uso del parametro attuale. Presente in Java (keyword final), Modula-3, ANSI C (keyword const)

### Passaggio per risultato

Comunicazione **unidirezionale**: da sottoprogramma a programma. Associa al parametro attuale la valutazione del parametro formale all'uscita dal sottoprogramma. **Il parametro formale non viene manipolato in ingresso ma in uscita**, e il parametro attuale deve essere un nome associato ad una locazione. Nel sottoprogramma non c'è alcun legame tra parametro formale e parametro attuale. Tale metodo è costoso per dati "grandi" (è sempre una copia). Come per costante. Presente in Java (keyword final), Modula-3, ANSI C (keyword const)

### Passaggio per valore-risultato

Comunicazione **bidirezionale** tra programma e sottoprogramma, tipica dei linguaggi imperativi in cui si vogliono restituire dati strutturati.

In ingresso associo al parametro formale il valore del parametro attuale, mentre in uscita faccio il viceversa, associo al parametro attuale il *valore* del parametro formale. Il parametro attuale deve essere un nome associato ad una locazione e l'operazione è sempre sugli ambienti (del chiamato e del chiamante). Il parametro formale in questo caso non è un alias del parametro attuale.

### Passaggio per nome

Metodo di comunicazione bidirezionale tra programma e sottoprogramma. Nasce con Algol-W (era il modo), ed è una sorta di sostituzione testuale: nel sottoprogramma le occorrenze del parametro formale vengono sostituite con il parametro attuale (che deve essere una variabile modificabile, quindi un nome a cui è associata una locazione). Considerate il programma

```
1. int y;
2. void passnome (name int \texttt{x} ) {
3.   x=x+1 ; }
   ...
n. y=1;
   ...
m. passnome (y);
```

In questo frammento abbiamo che, al momento della chiamata della funzione *passnome* *nome* del parametro attuale *y* viene sostituito nel corpo di *passnome* alla linea *m*. Si esegue quindi '*y* = *y* + 1'.

Ora il passaggio per nome ha dei problemi. Considerate il frammento

```
1. int y;
2. void passnome (name int x) {
3.   int y = 1;
   x = x + y; }
   ...
n. y = 2;
   ...
m. passnome(y)
```

se come prima il nome del parametro attuale *y* viene sostituito nel corpo di *passnome* alla linea *m*. Si esegue *y* = *y* + *y* ma il risultato dipende dal contesto di chiamata. Le due variabili (*y* e *y*) dovrebbero essere diverse, ma come fare per distinguerle? Viene introdotto il concetto di chiusura, cioè non si passa solo il nome, ma anche il suo ambiente.

La chiusura è formalmente una coppia  $\langle e, \delta \rangle$  dove  $\delta$  è l'ambiente dove valutare l'espressione *e*.

```
1. int y;
2. void passnome (name int x) {
3.   int y = 1;
   x = x + y; }
   ...
n. y = 2;
   ...
m. passnome(y)
```

come prima il nome del parametro attuale *y* viene sostituito nel corpo di *passnome* alla linea *m*. Si esegue *y* = *y* + **y** ma il risultato non dipende dal contesto di chiamata. Le due variabili (*y* e **y**) sono diverse, una è quella nella linea 1, l'altra nella linea 3, e ogni variabile viene valutata nel suo ambiente (tramite la chiusura), quindi si crea il corretto *aliasing* tra formale e attuale.

Per implementare il passaggio per nome, e quindi passare la coppia  $\langle e, \delta \rangle$  dove  $\delta$  è l'ambiente dove valutare l'espressione  $e$ , bisogna passare un *puntatore* al testo di  $e$  (quindi creare un'associazione con il testo di  $e$ ) e passare il puntatore di catena statica al RdA del chiamante.

Viene chiamata chiusura perché chiude una espressione con il suo ambiente. Osservate che le variabili *libere* vengono legate, quindi è come definire una funzione. Le chiusure servono a passare funzioni come argomenti ad altre funzioni, oltre che a capire come implementare lo scope statico.

### 12.3 Higher order

Abbiamo visto che si possono passare senza problemi oggetti di qualsiasi tipo, spesso e volentieri passando il modo per arrivare all'oggetto. Viene quindi naturale capire cosa succede se passiamo una funzione come argomento.

Consideriamo il programma

```
1. int z = 0;
2. int f (int y ) {
3.     return y + 1; }
4. int g (int -> int h ) {
5.     return h(3);
6. }
...
n. g(f);
```

qui la funzione  $f$ , che ha *tipo*  $\text{int} \rightarrow \text{int}$ , viene passata come argomento alla  $g$  che la usa nel suo *corpo*. Se è anche vero che non abbiamo mai formalmente definito quale è il tipo di un programma, e quindi di una funzione, abbiamo detto che un programma riceve dei dati in ingresso e restituisce dei dati in uscita, quindi intuitivamente il tipo di uno specifico programma è determinato dal tipo dei dati in ingresso e dal tipo dei dati che restituisce. La  $f$  riceve un intero e restituisce un intero, mentre la  $g$  riceve una funzione da interi ad interi e restituisce un intero. Quindi la  $g$  ha tipo  $(\text{int} \rightarrow \text{int}) \rightarrow \text{int}$ . Cosa succede quando chiamo la  $g$  e passo come argomento la  $f$ ? Alla linea 5 viene eseguita una chiamata al parametro  $h$ , a cui è associata la  $f$ , quindi si chiama la  $f$ . In questo caso non ci sono variabili non locali quindi non ci sono problemi con lo scope statico o dinamico, ma in generale non è così. L'esecuzione della chiamata  $h(3)$  fa sì che venga restituita dalla  $f$  il valore 4 e anche dalla  $g$ .

Consideriamo il programma

```
1. {int x = 4;
2. int z = 0;
3. int f (int y){
4.     return y+x;}
5. void g (int -> int h) {
6.     int x = 7;
7.     z = h(3) + x;}
...
m. {int x = 5;
...
1. g(f);
...}
...
}
```

Cosa succede quando chiamo la *g* e passo come argomento la *f* che viene eseguita alla linea 7? A quale definizione della *x* ci si riferisce nell'uso della *f* ?

Ce ne sono 3. Una è quella alla linea 1, nel blocco più esterno, poi ne troviamo una alla linea 6, locale alla *g*, ed infine una locale al blocco che inizia alla linea *m* e all'interno del quale si trova la chiamata alla *g*.

Devo stabilire quali sono le **politiche di binding**. Quando una procedura viene passata come parametro, si crea un riferimento tra un nome, il parametro formale, nel nostro esempio *h*, ed una procedura, nel nostro esempio il parametro attuale *f*. Indichiamo questo legame con  $h \leftrightarrow f$ . Quale ambiente non locale si applica al momento dell'esecuzione di *f* che è stata chiamata via *h* all'interno della *g* ?

Le politiche di binding sono due:

- **deep binding**: ambiente al momento della creazione del legame  $h \leftrightarrow f$ ;
- **shallow binding**: ambiente al momento della chiamata di *f* via *h*.

Domandiamoci quale è la relazione con lo scope.

Se lo **scope è statico**, *shallow* e *deep* binding **non contano**, dato che la regola dello scope ci dice quali sono le variabili non locali della procedura passata come parametro (passeremo sempre una chiusura).

Se lo scope è **dinamico** allora la situazione si fa più complessa. Nel caso in cui si implementa il **deep binding** ci si immagina che il programmatore voglia che l'**ambiente** sia quello **al momento della chiamata della funzione** a cui si passa la funzione come parametro, e si realizza un po' di scope statico, mentre se si implementa lo **shallow binding** vale la solita **regola dello scope statico**.

Nel concreto, nel nostro esempio,

- *se lo scope è statico* la chiamata *h(3)* fa sì che la *x* all'interno della *f* sia quella dichiarata alla linea 1
- *se lo scope è dinamico* e la politica di binding è quella del *shallow binding*, in breve si applica la regola dello **scope dinamico**, quindi la *x* all'interno della *f* è quella dichiarata alla linea 6, nell'ambiente attivo al momento della invocazione della *f*
- infine *se lo scope è dinamico* e la politica di binding è quella del *deep binding*, si applica solo qui limitatamente la regola dello scope statico, quindi la *x* all'interno della *f* è quella dichiarata alla linea *m*, cioè quella nell'ambiente attivo al momento della invocazione della *g* e quindi della creazione del legame  $h \leftrightarrow f$

Nel caso dello scope statico, per passare dinamicamente sia il legame al codice della funzione che il suo ambiente non locale, passiamo una chiusura  $\langle \text{code}, \delta \rangle$ . Alla chiamata di una procedura passata per parametro, si alloca (come sempre) il record di attivazione sulla pila e si prende il puntatore di catena statica dalla chiusura (della funzione), quindi le chiusure servono per mantenere puntatore all'ambiente statico del corpo di una funzione. Tutti i puntatori di catena statica puntano sempre all'indietro nella pila, quindi alcuni RdA vengono saltati per accedere alle variabili non locali e la de-allocazione dei record di attivazione avviene sempre secondo la politica LIFO (abbiamo sempre la pila).

Per implementare lo scope dinamico con la politica del **deep binding** devo usare le chiusure per **congelare uno scope da riattivare dopo**, mentre per lo **shallow binding** non devo **fare nulla di particolare**.

Consideriamo il frammento

```
void A (void -> int f, int x) {
    int B () {
        return x;}
}
```

```

    int z;
    if x==0 then {z = f();}
                else {A(B,0);}
    }
...
int C () {
    return 1;}
...
A(C,1);

```

a quale  $x$  si fa riferimento se chiamo  $A(B, 0)$  in  $A$  (e quindi chiamerò eventualmente la  $B$ )?

Lo scope da solo non basta. Se la politica è quella dello *shallow binding* quello al momento dell'uso, quindi a  $x$  è associato il valore 0, mentre se la politica è quella del *deep binding* quello al momento della creazione del legame tra la  $f$  e la  $B$ , quindi la  $x$  contiene 1.

Abbiamo visto come passare le funzioni come parametri, vediamo cosa succede se vogliamo restituire una funzione come risultato. In molti linguaggi possiamo definire tipi complessi (ad esempio, una funzione) ed è quindi naturale immaginare di poter scrivere sottoprogrammi che definiscono funzioni. Abbiamo la creazione dinamica di funzioni al momento dell'esecuzione. Ricordiamo che quando eseguiamo una funzione dobbiamo conoscere l'ambiente non locale della funzione, quindi questo come lo determino?

Consideriamo

```

1. int x = 1;
2. void -> int F () {
3.     int g () {
4.         return x + 1;
5.     }
6.     return g;
7. }
...
m. void -> int gg = F();
...
1. int z = gg();

```

La  $F$  restituisce una funzione che ha gli stessi riferimenti non locali di dove viene creata ed un suo ambiente, quindi restituisco sempre una chiusura.

Il tipo del valore restituito da  $F$  è  $\text{void} \rightarrow \text{int}$  dato che nel corpo della  $F$  c'è la definizione della funzione che dobbiamo restituire, e quando si invoca la  $F$  (alla linea  $m$ ) si dà un nome alla funzione restituita ( $gg$ ) che viene usata alla linea 1. L'effetto è di incrementare  $x$  della linea 1.

Non ci sono problemi perché, anche se il RdA di  $F$  non è più sulla pila, c'è quello che ci interessa: il riferimento non locale  $x$ . Sostanzialmente è per questo bisogna restituire una chiusura.

Ma consideriamo

```

1. void -> int F () {
2.     int x = 1;
3.     int g () {
4.         return x + 1;
5.     }
6.     return g;
7. }

```



```
...  
m. void -> int gg = F();;  
...  
1. int z = gg();
```

Quando si invoca la *F* (linea *m* ) si dà un nome alla funzione restituita (*gg*) che viene usata alla linea *1* . L'effetto è di incrementare *x* di 1. Ora ci sono problemi perché il RdA di *F* non è più sulla pila e quindi non c'è quello che interessa: il riferimento **non** locale a *x*, e questo anche se si restituisce una chiusura, che contiene un puntatore ad una parte che non esiste più.

Bisogna *derogare* alla politica FIFO per gli RdA e *conservare* quell'ambiente. Bisogna quindi non deallocare gli RdA esplicitamente, ma metterli sullo heap e collegarli con i vari puntatori, indicare quando non sono *in esecuzione* ma il loro ambiente è attivo. La soluzione adottata è che di solito la pila degli RdA viene allocata sullo heap.





sas

## 13. Linguaggio Funzionale



## 14. Esercizi

**Esercizio 14.1** Considerate la grammatica  $G = (\{S, A, B\}, \{a, b\}, \mathcal{P}, S)$  dove  $\mathcal{P}$  contiene le produzioni

$$\mathcal{P} = \begin{cases} S & \rightarrow aSa \mid bSb \mid aAb \\ A & \rightarrow aAb \mid aBb \mid BA \mid AB \mid b \\ B & \rightarrow bAa \mid bAb \mid b \mid a \end{cases}$$

- Scrivere una derivazione per la parola abaaabbba,
- Scrivere una derivazione per la parola abbaababb,
- Scrivere una derivazione per la parola abababbba,
- Scrivere una derivazione per la parola ababbbabb,
- Scrivere una derivazione per la parola abbbabbab,
- Scrivere una derivazione per la parola bbbaabbbb,
- Scrivere una derivazione per la parola baaaabbab,
- Scrivere una derivazione per la parola aaaaabbaa,
- Scrivere una derivazione per la parola aabbbabb,
- Scrivere una derivazione per la parola abbbbbb.

**Soluzione:**

a) soluzione:

|           |                                   |            |
|-----------|-----------------------------------|------------|
| <b>S</b>  | $\xrightarrow{S \rightarrow aSa}$ | <b>aSa</b> |
| aSa       | $\xrightarrow{S \rightarrow bSb}$ | abSba      |
| abSba     | $\xrightarrow{S \rightarrow aAb}$ | abaAbba    |
| abaAbba   | $\xrightarrow{A \rightarrow aBb}$ | abaaBbbba  |
| abaaBbbba | $\xrightarrow{B \rightarrow a}$   | abaaabbba  |

b) soluzione:

|                  |                                   |                  |
|------------------|-----------------------------------|------------------|
| <b>S</b>         | $\xrightarrow{S \rightarrow aSb}$ | <b>aAb</b>       |
| <b>aAb</b>       | $\xrightarrow{A \rightarrow BA}$  | <b>aBAb</b>      |
| <b>aBAb</b>      | $\xrightarrow{B \rightarrow bAa}$ | <b>abAaAb</b>    |
| <b>abAaAb</b>    | $\xrightarrow{A \rightarrow BA}$  | <b>abBAaAb</b>   |
| <b>abBAaAb</b>   | $\xrightarrow{B \rightarrow b}$   | <b>abbAaAb</b>   |
| <b>abbAaAb</b>   | $\xrightarrow{A \rightarrow aBb}$ | <b>abbaBbaAb</b> |
| <b>abbaBbaAb</b> | $\xrightarrow{B \rightarrow a}$   | <b>abbaabaAb</b> |
| <b>abbaabaAb</b> | $\xrightarrow{A \rightarrow b}$   | <b>abbaababb</b> |

c) soluzione:

|                  |                                   |                  |
|------------------|-----------------------------------|------------------|
| <b>S</b>         | $\xrightarrow{S \rightarrow aSa}$ | <b>aSa</b>       |
| <b>aSa</b>       | $\xrightarrow{S \rightarrow bSb}$ | <b>abSba</b>     |
| <b>abSba</b>     | $\xrightarrow{S \rightarrow aAb}$ | <b>abaAbba</b>   |
| <b>abaAbba</b>   | $\xrightarrow{A \rightarrow AB}$  | <b>abaABbba</b>  |
| <b>abaABbba</b>  | $\xrightarrow{A \rightarrow AB}$  | <b>abaABBBba</b> |
| <b>abaABBBba</b> | $\xrightarrow{A \rightarrow b}$   | <b>ababBBbba</b> |
| <b>ababBBbba</b> | $\xrightarrow{B \rightarrow a}$   | <b>ababaBbba</b> |
| <b>ababaBbba</b> | $\xrightarrow{B \rightarrow b}$   | <b>abababbba</b> |

d) soluzione:

|                  |                                   |                  |
|------------------|-----------------------------------|------------------|
| <b>S</b>         | $\xrightarrow{S \rightarrow aAb}$ | <b>aAb</b>       |
| <b>aAb</b>       | $\xrightarrow{A \rightarrow BA}$  | <b>aBAb</b>      |
| <b>aBAb</b>      | $\xrightarrow{B \rightarrow b}$   | <b>abAb</b>      |
| <b>abAb</b>      | $\xrightarrow{A \rightarrow aAb}$ | <b>abaAbb</b>    |
| <b>abaAbb</b>    | $\xrightarrow{A \rightarrow AB}$  | <b>abaABbb</b>   |
| <b>abaABbb</b>   | $\xrightarrow{B \rightarrow bAa}$ | <b>abaAbAabb</b> |
| <b>abaAbAabb</b> | $\xrightarrow{A \rightarrow b}$   | <b>ababbAabb</b> |
| <b>ababbAabb</b> | $\xrightarrow{A \rightarrow b}$   | <b>ababbabb</b>  |

e) soluzione:

|                  |                                   |                  |
|------------------|-----------------------------------|------------------|
| <b>S</b>         | $\xrightarrow{S \rightarrow aAb}$ | <b>aAb</b>       |
| <b>aAb</b>       | $\xrightarrow{A \rightarrow AB}$  | <b>aABb</b>      |
| <b>aABb</b>      | $\xrightarrow{A \rightarrow b}$   | <b>abBb</b>      |
| <b>abBb</b>      | $\xrightarrow{B \rightarrow bAa}$ | <b>abbAab</b>    |
| <b>abbAab</b>    | $\xrightarrow{A \rightarrow BA}$  | <b>abbBAab</b>   |
| <b>abbBAab</b>   | $\xrightarrow{B \rightarrow b}$   | <b>abbbAab</b>   |
| <b>abbbAab</b>   | $\xrightarrow{A \rightarrow aAb}$ | <b>abbbAAbab</b> |
| <b>abbbAAbab</b> | $\xrightarrow{A \rightarrow b}$   | <b>abbbabbab</b> |

f) soluzione:

|                  |                                   |                  |
|------------------|-----------------------------------|------------------|
| <b>S</b>         | $\xrightarrow{S \rightarrow bSb}$ | <b>bSb</b>       |
| <b>bSb</b>       | $\xrightarrow{S \rightarrow bSb}$ | <b>bbSbb</b>     |
| <b>bbSbb</b>     | $\xrightarrow{S \rightarrow aAb}$ | <b>bbaAbbb</b>   |
| <b>bbaAbbb</b>   | $\xrightarrow{A \rightarrow aBb}$ | <b>bbaaBbbbb</b> |
| <b>bbaaBbbbb</b> | $\xrightarrow{B \rightarrow a}$   | <b>bbaaabbbb</b> |

g) soluzione:

|                  |                                   |                  |
|------------------|-----------------------------------|------------------|
| <b>S</b>         | $\xrightarrow{S \rightarrow bSb}$ | <b>bSb</b>       |
| <b>bSb</b>       | $\xrightarrow{S \rightarrow aSa}$ | <b>baSab</b>     |
| <b>baSab</b>     | $\xrightarrow{S \rightarrow aAb}$ | <b>baaAbab</b>   |
| <b>baaAbab</b>   | $\xrightarrow{A \rightarrow aBb}$ | <b>baaaBbbab</b> |
| <b>baaaBbbab</b> | $\xrightarrow{B \rightarrow a}$   | <b>baaaabbab</b> |

h) soluzione:

|                  |                                   |                   |
|------------------|-----------------------------------|-------------------|
| <b>S</b>         | $\xrightarrow{S \rightarrow aSa}$ | <b>aSa</b>        |
| <b>aSa</b>       | $\xrightarrow{S \rightarrow aSa}$ | <b>aaSaa</b>      |
| <b>aaSaa</b>     | $\xrightarrow{S \rightarrow aAb}$ | <b>aaaAbaa</b>    |
| <b>aaaAbaa</b>   | $\xrightarrow{A \rightarrow aBb}$ | <b>aaaaBbbaa</b>  |
| <b>aaaaBbbaa</b> | $\xrightarrow{B \rightarrow a}$   | <b>aaaaabbbaa</b> |

i) soluzione:

|                  |                                   |                  |
|------------------|-----------------------------------|------------------|
| <b>S</b>         | $\xrightarrow{S \rightarrow aAb}$ | <b>aAb</b>       |
| <b>aAb</b>       | $\xrightarrow{A \rightarrow aBb}$ | <b>aaBbb</b>     |
| <b>aaBbb</b>     | $\xrightarrow{B \rightarrow bBa}$ | <b>aabAabb</b>   |
| <b>aabAabb</b>   | $\xrightarrow{A \rightarrow BA}$  | <b>aabBAabb</b>  |
| <b>aabBAabb</b>  | $\xrightarrow{A \rightarrow BA}$  | <b>aabBBAabb</b> |
| <b>aabBBAabb</b> | $\xrightarrow{A \rightarrow b}$   | <b>aabBBbabb</b> |
| <b>aabBBbabb</b> | $\xrightarrow{B \rightarrow b}$   | <b>aabbBbabb</b> |
| <b>aabbBbabb</b> | $\xrightarrow{B \rightarrow b}$   | <b>aabbbbabb</b> |

j) soluzione:



|                      |                                   |                      |
|----------------------|-----------------------------------|----------------------|
| <b>S</b>             | $\xrightarrow{S \rightarrow aAb}$ | <b>aAb</b>           |
| a <b>A</b> b         | $\xrightarrow{A \rightarrow BA}$  | a <b>BA</b> b        |
| aB <b>A</b> b        | $\xrightarrow{A \rightarrow BA}$  | aBB <b>A</b> b       |
| aBB <b>A</b> b       | $\xrightarrow{A \rightarrow BA}$  | aBBB <b>A</b> b      |
| aBBB <b>A</b> b      | $\xrightarrow{A \rightarrow BA}$  | aBBBB <b>A</b> b     |
| aBBBB <b>A</b> b     | $\xrightarrow{A \rightarrow BA}$  | aBBBBB <b>A</b> b    |
| aBBBBB <b>A</b> b    | $\xrightarrow{A \rightarrow BA}$  | aBBBBBB <b>A</b> b   |
| aBBBBBB <b>A</b> b   | $\xrightarrow{B \rightarrow b}$   | abBBBBBB <b>A</b> b  |
| abBBBBBB <b>A</b> b  | $\xrightarrow{B \rightarrow b}$   | abbBBBBBB <b>A</b> b |
| abbBBBBBB <b>A</b> b | $\xrightarrow{B \rightarrow b}$   | abbbbBBBB <b>A</b> b |
| abbbbBBBB <b>A</b> b | $\xrightarrow{B \rightarrow b}$   | abbbbbBB <b>A</b> b  |
| abbbbbBB <b>A</b> b  | $\xrightarrow{B \rightarrow b}$   | abbbbbbb <b>A</b> b  |
| abbbbbbb <b>A</b> b  | $\xrightarrow{A \rightarrow b}$   | abbbbbbb <b>b</b>    |

**Esercizio 14.2** Considerate la grammatica  $G = (\{S, A\}, \{a, b\}, \mathcal{P}, S)$  dove  $\mathcal{P}$  contiene le produzioni

$$\mathcal{P} = \left\{ \begin{array}{l} S \rightarrow aSa \mid bSb \mid aAa \mid bAb \mid \\ A \rightarrow a \end{array} \right.$$

Dite se il linguaggio generato dalle produzioni è quello delle parole palindrome di lunghezza dispari con al centro una  $a$ . La risposta non può essere un semplice sì o no ma deve essere argomentata.

**Soluzione:** Sì, il linguaggio generato dalle produzioni è quello delle parole palindrome di lunghezza dispari con al centro una  $a$ . Questo perché tutte le regole di produzione che passano per  $S$  aggiungono sempre la stessa lettera sia a destra che a sinistra. Inoltre quando si usano le regole  $S \rightarrow aAa \mid bAb$  si passa al simbolo non terminale  $A$ , che permette di aggiungere solo la lettera  $a$  che sarà obbligatoriamente al centro.

**Esercizio 14.3** Considerate la grammatica  $G = (\{S, A\}, \{a, b\}, \mathcal{P}, S)$  dove  $\mathcal{P}$  contiene le produzioni

$$\mathcal{P} = \left\{ \begin{array}{l} S \rightarrow aSb \mid bSa \mid bAa \mid aAb \mid \\ A \rightarrow a \end{array} \right.$$

Dite se il linguaggio generato dalle produzioni è quello delle parole in cui il numero di  $a$  è uguale al numero di  $b$  più uno. La risposta non può essere un semplice sì o no ma deve essere argomentata.

**Esercizio 14.4** Considerate la grammatica  $G = (\{S, A\}, \{a, b\}, \mathcal{P}, S)$  dove  $\mathcal{P}$  contiene le produzioni

$$\mathcal{P} = \begin{cases} S & \rightarrow aSbb \mid abSb \mid baAb \mid abAb \\ A & \rightarrow ab \end{cases}$$

Dite se il linguaggio generato dalle produzioni è quello delle parole in cui il numero di  $a$  è la metà del numero di  $b$  e almeno tante  $b$  quante  $a$  sono un suffisso della parola. La risposta non può essere un semplice sì o no ma deve essere argomentata. ■

**Esercizio 14.5** Considerate la grammatica  $G = (\{S, A\}, \{a, b\}, \mathcal{P}, S)$  dove  $\mathcal{P}$  contiene le produzioni

$$\mathcal{P} = \begin{cases} S & \rightarrow aSb \mid aAb \\ A & \rightarrow aaA \mid aa \end{cases}$$

Dite se il linguaggio generato dalle produzioni è quello delle parole in cui il numero di  $a$  è uguale al numero di  $b$  più un numero pari qualsiasi maggiore di 0. La risposta non può essere un semplice sì o no ma deve essere argomentata. ■

**Esercizio 14.6** Considerate la grammatica  $G = (\{S, A\}, \{a, b\}, \mathcal{P}, S)$  dove  $\mathcal{P}$  contiene le produzioni

$$\mathcal{P} = \begin{cases} S & \rightarrow aAbBa \mid aAb \\ A & \rightarrow ab \\ B & \rightarrow bBa \mid b \end{cases}$$

Dite se il linguaggio generato dalle produzioni è quello delle parole in cui il numero di  $a$  è uguale al numero di  $b$  più uno e se una  $b$  è preceduta da una  $a$  allora prima della  $a$  ci sono solo  $a$ , e se una  $b$  è seguita da una  $a$  allora dopo la  $a$  ci sono solo  $a$ . La risposta non può essere un semplice sì o no ma deve essere argomentata. ■

**Esercizio 14.7** Considerate il seguente  $\lambda$ -termine  $(\lambda x.\lambda y.xy)(\lambda z.z)(\lambda y.yy)$ . Identificate i redexes del termine ed eseguite un passo di  $\beta$ -riduzione.

**Soluzione:**

I redex sono 2

1.  $(\lambda x.\lambda y.xy)(\lambda z.z)(\lambda y.yy)$
2.  $(\lambda x.\lambda y.xy)(\lambda z.z)(\lambda y.yy)$

Una applicazione di un passo di  $\beta$ -calcolo è il seguente:

$$(\lambda x.\lambda y.xy)(\lambda z.z)(\lambda y.yy) \rightarrow_{\beta} (\lambda y.(\lambda z.z)y)(\lambda y.yy)$$

**Esercizio 14.8** Considerate il seguente  $\lambda$ -termine  $(\lambda x.\lambda y.xxy)(\lambda z.z)(\lambda y.y)$ . Identificate i redexes del termine ed eseguite un passo di  $\beta$ -riduzione.

**Soluzione:**

I redex sono 2

1.  $(\lambda x.\lambda y.xxy)(\lambda z.z)(\lambda y.y)$
2.  $(\lambda x.\lambda y.xxy)(\lambda z.z)(\lambda y.y)$

Una applicazione di un passo di  $\beta$ -calcolo è il seguente:

$$\underline{(\lambda x.\lambda y.xxy)(\lambda z.z)(\lambda y.y)} \rightarrow_{\beta} (\lambda y.(\lambda z.z)(\lambda z.z)y)(\lambda y.y)$$

**Esercizio 14.9** Considerate il seguente  $\lambda$ -termine  $(\lambda x.\lambda y.yxy)(\lambda z.z)(\lambda y.y)$ . Identificate i redexes del termine ed eseguite un passo di  $\beta$ -riduzione.

**Soluzione:**

I redex sono 2:

1.  $(\lambda x.\lambda y.yxy)(\lambda z.z)(\lambda y.y)$
2.  $(\lambda x.\lambda y.yxy)(\lambda z.z)(\lambda y.y)$

Una applicazione di un passo di  $\beta$ -calcolo è il seguente:

$$\underline{(\lambda x.\lambda y.yxy)(\lambda z.z)(\lambda y.y)} \rightarrow_{\beta} (\lambda y.y(\lambda z.z)y)(\lambda y.y)$$

**Esercizio 14.10** Considerate il seguente  $\lambda$ -termine  $(\lambda x.\lambda y.xyy)(\lambda z.z)(\lambda y.y)$ . Identificate i redexes del termine ed eseguite un passo di  $\beta$ -riduzione.

**Soluzione:**

I redex sono 2:

1.  $(\lambda x.\lambda y.xyy)(\lambda z.z)(\lambda y.y)$
2.  $(\lambda x.\lambda y.xyy)(\lambda z.z)(\lambda y.y)$

Una applicazione di un passo di  $\beta$ -calcolo è il seguente:

$$\underline{(\lambda x.\lambda y.xyy)(\lambda z.z)(\lambda y.y)} \rightarrow_{\beta} (\lambda y.(\lambda z.z)yy)(\lambda y.y)$$

**Esercizio 14.11** Considerate il seguente  $\lambda$ -termine  $(\lambda x.\lambda y.xvy)(\lambda z.wz)(\lambda y.yv)$ . Quali sono le variabili libere del termine? Eseguite, se possibile, un passo di  $\beta$ -riduzione.

**Soluzione:**

I redex sono 2:

1.  $(\lambda x.\lambda y.xvy)(\lambda z.wz)(\lambda y.yv)$
2.  $(\lambda x.\lambda y.xvy)(\lambda z.wz)(\lambda y.yv)$

Le variabili libere sono  $v$  nel primo termine,  $w$  nel secondo termine e  $v$  nel terzo termine.

Una applicazione di un passo di  $\beta$ -calcolo è il seguente:

$$\underline{(\lambda x.\lambda y.xvy)(\lambda z.wz)(\lambda y.yv)} \rightarrow_{\beta} (\lambda y.(\lambda z.wz)vy)(\lambda y.yv)$$

**Esercizio 14.12** Considerate il seguente  $\lambda$ -termine  $(\lambda x.xvy)(\lambda z.wz)(\lambda y.yv)$ . Quali sono le variabili libere del termine? Eseguite, se possibile, un passo di  $\beta$ -riduzione.

**Soluzione:**

I redex sono 2:

1.  $(\lambda x.xvy)(\lambda z.wz)(\lambda y.yv)$
2.  $(\lambda x.xvy)(\lambda z.wz)(\lambda y.yv)$

Le variabili libere sono  $v$  e  $y$  nel primo termine,  $w$  nel secondo termine e  $v$  nel terzo termine.

Una applicazione di un passo di  $\beta$ -calcolo è il seguente:

$$(\lambda x.xvy)(\lambda z.wz)(\lambda y.yv) \rightarrow_{\beta} ((\lambda z.wz)vy)(\lambda y.yv)$$

**Esercizio 14.13** Considerate il seguente  $\lambda$ -termine  $(\lambda x.xy)(\lambda z.\lambda w.wz)(\lambda y.yv)$ . Quali sono le variabili libere del termine? Eseguite, se possibile, un passo di  $\beta$ -riduzione.

**Soluzione:**

I redex sono 2:

- $(\lambda x.xy)(\lambda z.\lambda w.wz)(\lambda y.yv)$
- $(\lambda x.xy)(\lambda z.\lambda w.wz)(\lambda y.yv)$

Le variabili libere sono la  $y$  nel primo termine e la  $v$  nel terzo termine.

Una applicazione di un passo di  $\beta$ -calcolo è il seguente:

$$(\lambda x.xy)(\lambda z.\lambda w.wz)(\lambda y.yv) \rightarrow_{\beta} ((\lambda z.\lambda w.wz)y)(\lambda y.yv)$$

**Esercizio 14.14** Eseguite le SECD elencate di seguito, dove eseguire significa che dovete fare tutti i passaggi per arrivare al risultato usando le regole per la SECD. Le pile sono indicate come le liste in Ocaml.

- a)  $[([], \perp, [\text{sum}(\text{apply}(\text{fun}(g, y, \text{sum}(y, 1)), 4), \text{apply}(\text{fun}(f, x, x), 2))])]$
- b)  $[([], \perp, [\text{apply}(\text{fun}(f, x, x), \text{fun}(h, z, \text{sum}(3, z)))])]$
- c)  $[([], \perp, [\text{sum}(\text{apply}(\text{fun}(f, x, \text{apply}(x, 4)), \text{fun}(h, z, \text{sum}(3, z))), 3)])]$
- d)  $[([], \perp, [\text{sum}(\text{apply}(\text{fun}(f, x, \text{sum}(x, 3)), 5), \text{apply}(\text{fun}(g, y, \text{sum}(4, y)), 1)])]$
- e)  $[([], \perp, [\text{minus}(\text{apply}(\text{fun}(f, x, \text{sum}(x, x)), 8), \text{apply}(\text{fun}(f, x, \text{sum}(x, 5)), 1)])]$
- f)  $[([], \perp, [\text{apply}(\text{fun}(f, x, \text{apply}(x, \text{apply}(\text{fun}(g, y, \text{sum}(y, 2)), 1))), \text{fun}(h, z, \text{sum}(3, z))])]$

**Soluzioni:**

**Secd a**

```
[([], \perp, [sum(apply(fun(g, y, sum(y, 1)), 4), apply(fun(f, x, x), 2)])])
[([], \perp, [apply(fun(g, y, sum(y, 1)), 4); apply(fun(f, x, x), 2); Prim(sum)])]
[([], \perp, [fun(g, y, sum(y, 1)); 4; app; apply(fun(f, x, x), 2); Prim(sum)])]
[[[fun(g, y, sum(y, 1))], \perp, [4; app; apply(fun(f, x, x), 2); Prim(sum)]]]
[[[4; fun(g, y, sum(y, 1))], \perp, [app; apply(fun(f, x, x), 2); Prim(sum)]]]
[([], \perp {g/f1, y/4}, [sum(y, 1)]); ([[], \perp, [apply(fun(f, x, x), 2); Prim(sum)]])]
[([], \perp {g/f1, y/4}, [y; 1; Prim(sum)]); D1]
[[[4], \perp {g/f1, y/4}, [1; Prim(sum)]]; D1]
[[[1; 4], \perp {g/f1, y/4}, [Prim(sum)]]; D1]
```

```

[[([5], ⊥, {g/f1, y/4}, [])]([[], ⊥, [apply(fun(f, x, x), 2); Prim(sum)])]
[[([5], ⊥, [apply(fun(f, x, x), 2); Prim(sum)])]
[[([5], ⊥, [fun(f, x, x); 2; app; Prim(sum)])]
[[([fun(f, x, x); 5], ⊥, [2; app; Prim(sum)])]
[[([2; fun(f, x, x); 5], ⊥, [app; Prim(sum)])]
[[([], ⊥, f/fun(f, x, x), x/2, [x]); ([5], ⊥, [Prim(sum)])]
[[([2], ⊥, f/fun(f, x, x), x/2, []); ([5], ⊥, [Prim(sum)])]
[[([2; 5], ⊥, [Prim(sum)])]
[[([7], ⊥, [])]

```

**Esercizio 14.15** Considerate il seguente pezzo di pseudo-codice di un linguaggio imperativo (quindi non corrisponde ad un preciso linguaggio di programmazione). I parametri, quando non indicato diversamente in una domanda, sono passati per valore. Ogni linea di codice è identificata da un numero, le parentesi graffe indicano i blocchi ed il '\*' indica la moltiplicazione tra interi. Le funzioni restituiscono un valore, che può anche essere una funzione, ed il valore restituito è indicato dalla keyword `return`. Infine la valutazione di espressioni complesse viene fatta da sinistra a destra e la valutazione delle sottoespressioni può influenzare le espressioni che le seguono, sempre da sinistra a destra, ad esempio se considerate  $f(x) + g(y)$  può accadere che i side effects dell'esecuzione della  $f(x)$  si ripercuotano sull'esecuzione della  $g(y)$  che avviene dopo l'esecuzione della  $f(x)$ .

```

01 {int x = 4;
02   int y = 4;
03   int z = 2;
04   int f (int y) {
05     int m (int z) {
06       return z + x;
07     }
08     return z;
09   }
10   (int -> int) g ((int -> int) n) {
11     int k (int w) {
12       return 2 * w;
13     }
14     x = n(z) + f(z);
15     return k;
16   }
17   int h (int w) {
18     int x = 2;
19     x = f(w) + z;
20     return x;
21   }
22   {int z = 5;
23     x = g(h)(x) + h(z);
24   }

```

- che valore contiene la variabile  $x$  alla linea 20 se lo scope è statico
- che valore contiene la variabile  $x$  alla linea 20 se lo scope è dinamico e la politica è quella del shallow binding
- che valore contiene la variabile  $x$  alla linea 20 se lo scope è statico e il parametro  $y$  della funzione  $f$  è passato per riferimento
- che valore contiene la variabile  $x$  alla linea 20 se lo scope è dinamico, la politica è quella dello shallow binding e il parametro  $y$  della funzione  $f$  è passato per riferimento
- che valore è associato alla variabile  $x$  nella funzione  $h$  quando viene chiamata alla linea 20
- che valore contiene la variabile  $x$  alla linea 20 se lo scope è dinamico e la politica è quella del deep binding
- quali sono i legami nell'ambiente al momento dell'esecuzione della chiamata della funzione  $n$  alla linea 13, assumendo che lo scope sia statico
- quali sono i legami nell'ambiente al momento dell'esecuzione della chiamata della funzione  $n$  alla linea 13, assumendo che lo scope sia dinamico e la politica quella del

**Esercizio 14.16** Considerate il seguente pezzo di pseudo-codice di un linguaggio imperativo (quindi non corrisponde ad un preciso linguaggio di programmazione). I parametri, quando non indicato diversamente in una domanda, sono passati per valore. Ogni linea di codice è identificata da un numero, le parentesi graffe indicano i blocchi ed il '\*' indica la moltiplicazione tra interi. Le funzioni restituiscono un valore, che può anche essere una funzione, ed il valore restituito è indicato dalla keyword return. Infine la valutazione di espressioni complesse viene fatta da sinistra a destra e la valutazione delle sottoespressioni può influenzare le espressioni che le seguono, sempre da sinistra a destra, ad esempio se considerate  $f(x) + g(y)$  può accadere che i side effects dell'esecuzione della  $f(x)$  si ripercuotano sull'esecuzione della  $g(y)$  che avviene dopo l'esecuzione della  $f(x)$ .

```

1 {int x = 1;
2   int y = 1;
3   int z = 2;
4   int f (int y) {
5     y = x + z;
6     return y};
7 int m (int x) {;
8   return y + x};
9 int h1 (int w) {
10   int x = 3;
11   x = f(x) * w;
12   return x};
13 int h2 (int x) {
14   int w = 1;
15   w = f(x) + w;
16   return w};
17 int -> int g (int -> int n) {
18   int k (int w) {
19     return m(w) + 1};
20   int x = 2;
21   int z = 3;
22   y = n(x) + f(z);
23   return y};
24 {int x = 2;
25   int z = 2;
26   x = g(h1)(x) + h2(y) + z;}}
```

- che valore contiene la variabile  $x$  alla linea 26 se lo scope è statico
- che valore contiene la variabile  $x$  alla linea 26 se lo scope è dinamico e la politica è quella del shallow binding
- che valore contiene la variabile  $x$  alla linea 26 se lo scope è statico e il parametro  $y$  della funzione  $f$  è passato per riferimento
- che valore contiene la variabile  $x$  alla linea 26 se lo scope è dinamico, la politica è quella dello shallow binding e il parametro  $y$  della funzione  $f$  è passato per riferimento
- che valore è associato alla variabile  $x$  nella funzione  $h2$  quando viene chiamata alla linea 26
- che valore contiene la variabile  $x$  alla linea 26 se lo scope è dinamico e la politica è quella del deep binding
- quali sono i legami nell'ambiente al momento dell'esecuzione della chiamata della funzione  $n$  alla linea 22, assumendo che lo scope sia statico
- quali sono i legami nell'ambiente al momento dell'esecuzione della chiamata della funzione  $n$  alla linea 22, assumendo che lo scope sia dinamico e la politica quella del shallow binding

**Esercizio 14.17** Considerate il seguente pezzo di pseudo-codice di un linguaggio imperativo (quindi non corrisponde ad un preciso linguaggio di programmazione). I parametri, quando non indicato diversamente in una domanda, sono passati per valore. Ogni linea di codice è identificata da un numero, le parentesi graffe indicano i blocchi ed il '\*' indica la moltiplicazione tra interi. Le funzioni restituiscono un valore, che può anche essere una funzione, ed il valore restituito è indicato dalla keyword `return`. Infine la valutazione di espressioni complesse viene fatta da sinistra a destra e la valutazione delle sottoespressioni può influenzare le espressioni che le seguono, sempre da sinistra a destra, ad esempio se considerate  $f(x) + g(y)$  può accadere che i side effects dell'esecuzione della  $f(x)$  si ripercuotano sull'esecuzione della  $g(y)$  che avviene dopo l'esecuzione della  $f(x)$ .

```

01 {int x = 1;
02   int y = 2;
03   int z = 3;
04   int f (int y) {
05       y = (x * z) + y;
06       return y};
07   int m (int x) {;
08       return y + x};
09   int h1 (int w) {
10       int x = 2;
11       x = f(x) * w;
12       return x};
13   int h2 (int x) {
14       int w = 2;
15       w = f(x) + x;
16       return w};
17   int -> int g (int -> int n) {
18       int k (int w) {
19           return m(w) + 1};
20       int y = 1;
21       int z = 2;
22       x = n(x) + f(z);
23       return k};
24   {int x = 0;
25     int z = 1;
26     x = g(h1)(x) + h2(y) + y;}}

```

- a) che valore contiene la variabile  $x$  alla linea 26 se lo scope è statico
- b) che valore contiene la variabile  $x$  alla linea 26 se lo scope è dinamico e la politica è quella del shallow binding
- c) che valore contiene la variabile  $x$  alla linea 26 se lo scope è statico e il parametro  $y$  della funzione  $f$  è passato per riferimento
- d) che valore contiene la variabile  $x$  alla linea 26 se lo scope è dinamico, la politica è quella dello shallow binding e il parametro  $y$  della funzione  $f$  è passato per riferimento
- e) che valore è associato alla variabile  $x$  nella funzione  $h2$  quando viene chiamata alla linea 26
- f) che valore contiene la variabile  $x$  alla linea 26 se lo scope è dinamico e la politica è quella del deep binding
- g) quali sono i legami nell'ambiente al momento dell'esecuzione della chiamata della funzione  $n$  alla linea 22, assumendo che lo scope sia statico
- h) quali sono i legami nell'ambiente al momento dell'esecuzione della chiamata della funzione  $n$  alla linea 22, assumendo che lo scope sia dinamico e la politica quella del shallow binding



# Indice analitico

## A

|                                                |    |
|------------------------------------------------|----|
| Albero di sintassi astratta .....              | 45 |
| Albero Sintattico .....                        | 23 |
| Allocazione dinamica .....                     | 89 |
| Allocazione statica .....                      | 89 |
| Ambiente .....                                 | 80 |
| globale .....                                  | 82 |
| locale .....                                   | 82 |
| non locale .....                               | 82 |
| operazioni .....                               | 82 |
| Ambiente di tipi .....                         | 69 |
| Ambiente locale dinamico .....                 | 88 |
| Ambiente locale statico .....                  | 88 |
| Annidamento dei blocchi .....                  | 91 |
| Automa a stati finiti deterministico .....     | 17 |
| Automa a stati finiti non-deterministico ..... | 17 |

## B

|                            |    |
|----------------------------|----|
| Beta-riduzione piena ..... | 47 |
| Big-step semantic .....    | 41 |
| Binder .....               | 46 |
| Binding .....              | 80 |
| Binding time .....         | 80 |
| dinamico .....             | 81 |
| statico .....              | 81 |
| Blocco .....               | 81 |

## C

|                                |    |
|--------------------------------|----|
| call-by-name .....             | 48 |
| call-by-need .....             | 48 |
| call-by-value .....            | 48 |
| Chiamata in coda .....         | 98 |
| Contesto di tipizzazione ..... | 69 |
| Currying .....                 | 49 |

## D

|                             |     |
|-----------------------------|-----|
| Deep binding .....          | 103 |
| Derivazione di parole ..... | 21  |
| DFA .....                   | 17  |

|                     |    |
|---------------------|----|
| Dichiarazione ..... | 81 |
| esplicita .....     | 81 |
| implicita .....     | 81 |

## E

|                              |    |
|------------------------------|----|
| Equivalenza di comandi ..... | 76 |
| Espressione regolare .....   | 14 |

## F

|                     |    |
|---------------------|----|
| FA .....            | 17 |
| Forma normale ..... | 38 |

## G

|                  |    |
|------------------|----|
| Grafo .....      | 22 |
| Grammatica ..... | 20 |

## I

|                                          |    |
|------------------------------------------|----|
| Indipendenza dai nomi locali .....       | 85 |
| Indipendenza dalla posizione .....       | 85 |
| Istanza di una regola di inferenza ..... | 36 |

## L

|                                                       |    |
|-------------------------------------------------------|----|
| Linguaggio accettato da un DFA .....                  | 17 |
| Linguaggio accettato da un NFA .....                  | 18 |
| Linguaggio associato a una espressione regolare ..... | 15 |
| Linguaggio generato da una grammatica .....           | 22 |
| Linguaggio regolare .....                             | 15 |

## M

|                             |    |
|-----------------------------|----|
| Memoria .....               | 89 |
| Multi-step evaluation ..... | 39 |

## N

|                             |    |
|-----------------------------|----|
| NFA .....                   | 17 |
| Nomi di un linguaggio ..... | 79 |
| numerali di Church .....    | 51 |

## O

|                           |    |
|---------------------------|----|
| Oggetti denotabili .....  | 80 |
| Oggetto                   |    |
| operazioni .....          | 82 |
| One-step evaluation ..... | 36 |
| Operazioni su i linguaggi |    |
| Chiusura di Kleene .....  | 13 |
| Concatenazione .....      | 13 |
| Differenza .....          | 13 |
| Intersezione .....        | 13 |
| Negazione .....           | 13 |
| Unione .....              | 13 |
| Operazioni sulle parole   |    |
| Concatenazione .....      | 12 |
| Infisso .....             | 13 |
| Lunghezza .....           | 12 |
| Prefisso .....            | 13 |
| Suffisso .....            | 13 |
| Ordine normale .....      | 47 |

## P

|                                    |     |
|------------------------------------|-----|
| Paradigma .....                    | 28  |
| funzionale .....                   | 28  |
| imperativo .....                   | 28  |
| logico .....                       | 28  |
| object oriented .....              | 28  |
| procedurale .....                  | 28  |
| Parametro attuale .....            | 98  |
| Parametro formale .....            | 98  |
| Parola vuota .....                 | 12  |
| Passaggio                          |     |
| per costante .....                 | 100 |
| per nome .....                     | 101 |
| per riferimento .....              | 100 |
| per risultato .....                | 100 |
| per valore .....                   | 100 |
| per valore-risultato .....         | 100 |
| Passo di derivazione .....         | 21  |
| Preservazione .....                | 66  |
| Produzione su un alfabeto .....    | 20  |
| Progresso .....                    | 66  |
| Puntatore di catena dinamica ..... | 90  |
| Puntatore di catena statica .....  | 90  |

## R

|                             |    |
|-----------------------------|----|
| Record di attivazione ..... | 89 |
| Redex .....                 | 47 |

|                            |    |
|----------------------------|----|
| Regola di calcolo .....    | 36 |
| Regola di congruenza ..... | 36 |
| Ricorsione in coda .....   | 98 |

## S

|                               |        |
|-------------------------------|--------|
| Scope .....                   | 46, 80 |
| dinamico .....                | 84, 86 |
| statico .....                 | 84, 86 |
| Semantica assiomatica .....   | 34     |
| Semantica denotazionale ..... | 34     |
| Semantica operativa .....     | 34     |
| shallow binding .....         | 103    |
| Sintassi .....                | 33     |
| Sintassi astratta .....       | 45     |
| Sintassi concreta .....       | 45     |
| Small-steps semantic .....    | 34     |
| Specializzazione .....        | 85     |
| Stato .....                   | 73, 93 |

## T

|                        |    |
|------------------------|----|
| Tailrecursion .....    | 98 |
| Termine bloccato ..... | 41 |

## V

|                                             |    |
|---------------------------------------------|----|
| Variabile bound .....                       | 46 |
| Variabile libera .....                      | 46 |
| Visibilità delle dichiarazioni locali ..... | 83 |
| Vita corta .....                            | 83 |
| Vita di un oggetto .....                    | 83 |
| Vita di una associazione .....              | 83 |
| Vita lunga .....                            | 83 |